

Pact

Developer & Integrator Handbook

Verified against commit [ad31d55](#) · July 2026

Contents

Copyright and disclaimer	7
I Overview	8
1 About this Handbook	9
1.1 Who this is for	9
1.2 How this handbook is organised	9
1.3 Conventions	10
1.4 Source revision and status	10
1.5 Where to get help	11
2 What Pact Is	12
2.1 The crate map	12
2.2 The bitcoind analogy	13
2.3 What Pact deliberately is <i>not</i>	13
2.4 The market metaphor	14
2.5 The first pair	14
3 Architecture & Trust Boundaries	15
3.1 The three moving parts	15
3.2 The hard wall	15
3.3 Data flow of an offer	16
3.4 Data flow of a swap	16
3.5 The picture	17
II Running Pact	18
4 Building from Source	19
4.1 Prerequisites	19
4.2 Building and testing the engine	19
4.3 Running the end-to-end harness	20
4.4 Building Corkboard	20
4.5 Where the binaries land	20
5 Running pactd	22
5.1 Synopsis	22
5.2 Flags	22
5.3 Logging	26
5.4 RPC authentication	27

5.5	The scheduler tick model	27
5.6	Example invocations	27
6	Coins, Pairs & Capabilities	29
6.1	Built-in coins	29
6.2	Adding coins with <code>coins.toml</code>	29
6.2.1	Nodeless (Electrum-only) coins	30
6.2.2	Per-coin minimum feerate	31
6.2.3	The <code>%NODEDIR%</code> <code>datadir</code> token	31
6.3	Attaching backends with <code>--coin</code>	31
6.4	Confirmation depth with <code>--coin-confs</code>	32
6.5	Capabilities and how pairs are derived	32
6.6	Inspecting coins and pairs	33
7	Seeds, Wallets & Merchants	34
7.1	The seed lifecycle	34
7.2	The encrypted seed format	35
7.3	The merchant model	35
7.4	What lives on disk	35
7.5	Seed-only swap rescue (#54)	36
7.5.1	Restore is gated, never automatic	37
7.5.2	Anchored participant keys	37
8	The <code>pact-cli</code>	38
8.1	Direct dispatch and help	38
8.2	Global flags	38
8.3	Structured subcommands	39
8.4	The explicit <code>call</code> spelling	40
8.5	Worked example: a v1 swap with two CLIs	40
III	The JSON-RPC API	42
9	JSON-RPC Conventions	43
9.1	Transport	43
9.2	Authentication	43
9.3	Request shape	44
9.4	Response shape	44
9.5	Examples	45
10	API: Node, Seed, Merchants, Coins	47
10.1	Node / info	47
10.1.1	Fee policy	48
10.2	Seed lifecycle	49
10.2.1	Seed-only rescue (#54)	50
10.3	Merchants	50
10.4	Coins / pairs	51
10.5	Wallet helpers	52
11	API: v1 HTLC Swaps	54
11.1	Swap state machine	54
11.2	The <code>coin:amount</code> convention	55
11.3	Read methods	55

11.4	Lifecycle methods	55
11.5	Diagnostics	57
12	API: v2 Adaptor Swaps	58
12.1	Swap state machine	58
12.2	Handshake order	59
12.3	Cancelling a v2 handshake	59
12.4	Read method	59
12.5	Handshake & lifecycle methods	60
13	API: Board, Private Offers & Fees	61
13.1	Board (Corkboard / Nostr)	61
13.2	Private (off-market) offers	63
13.3	Fees	63
IV	The Swap Protocol	65
14	The Swap Lifecycle	66
14.1	The two roles	66
14.2	The scheduler-driven model	67
14.3	v1 state machine — <code>swap::State</code>	68
14.4	v2 state machine — <code>AdaptorState</code>	69
14.5	Where the secret lives	70
15	v1 HTLC Swaps	71
15.1	Key derivation	71
15.2	The HTLC witness script	73
15.2.1	Per-leg keys and locktimes	73
15.3	Funding	74
15.4	Redeem	74
15.5	Refund	75
15.6	Preimage extraction	75
15.7	Message flow	76
15.8	vsize reference	76
16	v2 Taproot/MuSig2 Adaptor Swaps	77
16.1	Keys and secrets	77
16.2	The Taproot output	78
16.3	Cooperative redeem (key path)	79
16.4	Refund (script path)	79
16.5	The adaptor mechanism	80
16.6	Nonce safety	80
16.7	The recovery contract	81
16.8	Taproot tweak parity	81
16.9	vsize reference	82
17	Timelocks & Action Deadlines	83
17.1	The structural rule: $T2 < T1$	83
17.2	Network-profile minimums	83
17.3	The §7.4 action-deadline margins	84
17.3.1	The deadline clock	85
17.4	Why the margins exist: the reveal-too-late race	85

17.5 Offer-offset validation at post time	85
17.6 Recommended values: the UI presets	86
18 Fees, Fee-Bumping & Auto-Refund	87
18.1 The unified fee-bump strategy (market-tracking)	87
18.2 v1 fee-bumping (RBF)	89
18.3 v2 fee-bumping: a split design	89
18.3.1 The refund is RBF-bumpable	89
18.3.2 The cooperative redeem is NOT bumpable	89
18.4 The funding-bump nurse	90
18.5 Auto-refund	92
18.5.1 v1 auto-refund	92
18.5.2 v2 auto-refund	93
18.6 Confirmation depth as the reorg-finality knob	93
18.7 Configurable fee policy	93
19 Network Support, Reorgs & Safety	96
19.1 Network support	96
19.2 Network isolation: the foreign-pactd guard	96
19.3 Protocol selection prefers HTLC	97
19.4 Confirmation depth and reorg protection	97
19.5 The wallet-lock funding gate	98
19.6 Safety properties	99
19.7 Summary	99
V Transports	100
20 The Noticeboard Abstraction	101
20.1 The Noticeboard trait	101
20.1.1 Sealed relays, signed offers	102
20.2 The two implementations	102
20.2.1 BoardClient — HTTP Corkboard	102
20.2.2 NostrBoard — local buffer over Nostr	103
20.3 Fan-out vs browse	103
20.3.1 POST fans out to all boards	103
20.3.2 Browse selects one board	103
20.4 The key property: one board in common	104
20.5 The relay_poll cursor model	104
21 Wire Format (pact-proto)	105
21.1 The Envelope struct	105
21.2 Message types	106
21.2.1 OfferBody	106
21.3 Canonical JSON	107
21.4 BIP340 signing	107
21.5 swap_id derivation	108
21.6 The PACTSEALED1 sealed-blob format	108
21.7 The private-offer slip codec	108
22 Nostr Transport (pact-nostr)	110
22.1 Event kinds and constants	110
22.1.1 Offers — kind 31510	110

22.1.2	Gift wraps — kind 1059	111
22.1.3	Rescue snapshots — kind 31512	111
22.2	NIP-40 rolling expiration	112
22.3	Revocation — NIP-09	112
22.4	Subscription filters	113
22.5	Identity equals npub	113
22.6	The sync / inbox model	113
22.7	Default relay list	114
23	Corkboard (self-hostable board)	115
23.1	Running it	115
23.2	HTTP API	115
23.3	Storage model	116
23.4	Limits and errors	117
23.5	Blind relay	117
23.6	Reset hygiene	117
23.7	Auth model	118
23.8	Self-hosting walkthrough	118
24	Private (Off-Market) Offers	119
24.1	The slip	119
24.2	Engine surface	119
24.3	RPCs	120
24.4	The relay-assisted flow	120
24.5	Bearer-slip security	120
24.6	Fully-manual CLI fallback	121
VI	Building Against Pact	122
25	Building Your Own Front-End	123
25.1	The integration contract	123
25.2	A recommended app flow	123
25.3	What the front-end owns vs what pactd owns	124
25.4	Threading and polling	124
25.5	Error handling	125
26	Testing & the Regtest Harness	126
26.1	The cargo test suite	126
26.2	The Python harness	126
26.2.1	v1 HTLC scenarios — test_swap_e2e.py	127
26.2.2	v2 adaptor scenarios — test_adaptor_swap.py	128
26.2.3	Nodeless wallet scenarios — test_nodeless_e2e.py	129
26.3	Running the suites	130
26.4	The playground scripts	130
27	Reference	132
27.1	Default ports & paths	132
27.2	RPC method index	133
27.3	Error format	133
27.4	BIP32 derivation paths	134
27.5	Tagged-hash tags	135
27.6	Spec & vectors file map	135

27.7 Glossary 135

Copyright and disclaimer

Pact — Developer & Integrator Handbook Verified against commit ad31d55 · July 2026

This handbook tracks the code by commit hash rather than a release version: the hash above is the pact source revision its contents were verified against. When the code moves, the hash is bumped and the affected pages updated.

Pact is open-source software. This handbook is distributed alongside the software and may be reproduced and shared under the same terms. See the `LICENSE` file in the project repository for details.

Disclaimer — Pact is the swap engine that powers trustless, peer-to-peer atomic swaps. It holds keys and signs and broadcasts transactions on live networks. This handbook documents the engine for developers, integrators, and operators; it is **not** financial advice and makes no guarantees. The software is provided “as is”, without warranty of any kind. The code is the ultimate source of truth; consult the official project repository for the most up-to-date information.

Part I

Overview

Chapter 1

About this Handbook

This is the **Pact Developer & Integrator Handbook** — the reference for the software that actually executes atomic swaps in the Satchel project. It is written for people who run, drive, or build against the swap engine, not for end users trading from the desktop app. If you are an end user, read the *Satchel User Handbook* instead; this handbook assumes you are comfortable with Bitcoin script, key derivation, JSON-RPC, and the Rust toolchain.

1.1 Who this is for

You will get the most out of this handbook if you are one of the following:

- An **operator** running `pactd` — the local swap daemon — as a long-lived service: wiring it to your Bitcoin-Core-compatible nodes, configuring coins and confirmation depths, managing seeds and merchants, and keeping the auto-refund scheduler alive.
- A **front-end or tooling developer** building against the engine's JSON-RPC 2.0 API: a custom UI, a bot, a monitoring dashboard, or an integration that posts and takes offers programmatically.
- A **protocol implementer** who wants to understand — or independently reimplement — the on-chain swap construction: the HTLC scripts, the Taproot/MuSig2 adaptor scheme, the key-derivation paths, the timelock margins, and the message flow.

Note — The canonical, machine-checkable description of the wire format and on-chain construction lives in the `spec/` directory of the repository, together with deterministic test vectors. This handbook explains and contextualises that material; where the two ever disagree, the spec and the code win.

1.2 How this handbook is organised

The handbook is grouped into six parts that move from orientation to deep detail:

1. **Overview** — what Pact is, the crate map, and the architecture and trust boundaries. Start [here](#).

2. **Running Pact** — building from source, running `pactd`, configuring coins and pairs, and managing seeds, wallets, and merchants.
3. **The JSON-RPC API** — the complete method surface: `node/info`, the seed/wallet lifecycle, coins and pairs, v1 and v2 swaps, board operations, private offers, and fees.
4. **The Swap Protocol** — the v1 HTLC and v2 adaptor state machines step by step, the on-chain scripts, timelocks and action deadlines, fee management, and auto-refund.
5. **Transports** — how identity-signed offers and sealed coordination blobs move over Nostr relays and the self-hostable Corkboard.
6. **Building Against Pact** — driving swaps with `pact-cli`, the generic `call` escape hatch, and end-to-end worked examples.

You do not have to read straight through. Operators can jump to the chapter *Running pactd*; integrators to *The pact-cli* and the API part; implementers to the protocol part.

1.3 Conventions

This handbook follows a few consistent conventions:

- **Callouts** highlight things worth pausing on:
 - Tip** — a shortcut or a recommended way to do something.
 - Note** — context, a clarification, or a cross-reference.
 - Warning** — something that can lose funds, corrupt state, or fail boot if you get it wrong.
- **Text styles** carry meaning. monospace marks commands, flags, RPC method names, field names, file paths, and any exact text you type. **Bold** marks component names and UI labels. *Italics* mark a new term on first use.
- **Cross-references** name the target chapter by its title — for example, *see the chapter “Running pactd”* — never by number, because numbers shift as the handbook grows.
- **Code blocks** are fenced with a language hint (`sh`, `json`, `rust`, `text`) and are meant to be copy-pasteable.

1.4 Source revision and status

Status — Both swap protocols — v1 (HTLC) and v2 (Taproot/MuSig2 adaptor) — are reviewed and live on mainnet.

Rather than a release-version number, this handbook tracks the **source revision** it was checked against: it was verified against commit `ad31d55` (July 2026), the hash printed on the copyright page. The **code is the ultimate source of truth** — when a precise detail matters for your integration, confirm it against the code at that revision and the pinned test vectors before you rely on it. When the engine moves, the hash is bumped and the affected pages are updated.

1.5 Where to get help

- The repository `README.md` and the `docs/` directory carry the architecture and per-feature design notes.
- The `spec/` directory is the authoritative protocol description plus the `htlc_v1.json` / `htlc_v2.json` test vectors.
- The end-to-end harness under `pact/harness/` (notably `test_swap_e2e.py` and `test_adaptor_swap.py`) is the best executable reference for how a full swap is driven — when in doubt, read the harness.

Chapter 2

What Pact Is

Pact is the swap engine at the heart of the Satchel project: the software that builds, signs, broadcasts, and watches the on-chain transactions that make a trustless cross-chain trade *atomic*. It is the component that holds your keys, enforces the timelocks, and refunds you if a counterparty walks away. Everything else in the project — the desktop app, the transports — exists to feed offers to Pact and render what Pact reports back.

An *atomic swap* is a trade between two chains in which either both legs settle or neither does. Pact implements this two ways: *v1*, a classic hash-locked HTLC swap, and *v2*, a Taproot/MuSig2 *adaptor* swap. Both are covered in detail in the protocol part of this handbook.

2.1 The crate map

Pact is a small Rust workspace plus two supporting crates. Knowing which crate owns what is the fastest way to navigate the codebase:

Crate	Role
libswap	The engine proper: the HTLC and adaptor logic, the on-chain script construction, the key derivation, and the two swap state machines . All swap intelligence lives here; everything else is a shell around it.
pactd	The local daemon. Exposes libswap over JSON-RPC 2.0 , persists state in SQLite , and runs the background scheduler that auto-refunds, auto-redeems, and RBF-fee-bumps with no human present.
pact-cli	A thin RPC client — a hand-rolled HTTP caller that maps subcommands (and a generic <code>call</code>) onto pactd methods. The integrator's command line.

Crate	Role
<code>pact-proto</code>	The wire format and crypto primitives: signed envelopes, canonical JSON, BIP340 signatures, recipient-sealed relay blobs (PACTSEALED1), and private-offer slips.
<code>pact-nostr</code>	Pure mapping between Pact envelopes and Nostr events (public offer adverts as kind 31510, gift-wrapped relay messages as kind 1059). Mapping and crypto only — no relay I/O.

Note — `libswap` is where you look to verify how a swap is constructed; `pactd` is where you look to verify how it is *driven* and *persisted*. The two state machines (`swap::State` for v1, `AdaptorState` for v2) are defined in `libswap` and stepped by the engine’s scheduler tick inside `pactd`.

2.2 The bitcoind analogy

If you have run Bitcoin Core, the shape of Pact will be familiar — it is deliberately the same:

- `pactd` $\approx$ `bitcoind` — a headless daemon you run and leave running, with a data directory, a `.cookie` file for RPC auth, and a JSON-RPC endpoint.
- `pact-cli` $\approx$ `bitcoin-cli` — a thin command-line client that reads the cookie and talks to the daemon.
- **Satchel** $\approx$ `bitcoin-qt` — the optional desktop GUI that bundles and supervises the daemon and renders its RPC. (Satchel is documented in its own handbook.)

The data layout, the cookie auth, the HTTP Basic credentials, and the loopback-only listener are all modelled on Core. If you know how to run and script `bitcoind`, you already know most of how to run and script `pactd`.

2.3 What Pact deliberately is *not*

Pact is a swap *executor*, and nothing more. It is important to be precise about the boundaries:

- **No custody.** Pact never holds your coins on your behalf in any account. Your funds are either in your own node’s wallet or locked in a swap output that only you or your counterparty can spend — enforced by the chain, not by Pact.
- **No matching engine.** Pact does not pair buyers with sellers. It posts an offer to a transport and takes an offer from one; *finding* the offer is the transport’s job (see the chapter *Architecture & Trust Boundaries*).
- **No fees.** There is no platform fee anywhere. The `platform_fee_sat` field reported by `estimateswapfees` is hard-wired to 0; the only costs in a swap are the on-chain miner fees of its transactions.

- **No order book of record.** Offers live on transports (Nostr relays or a Corkboard), which are untrusted and replaceable. Pact derives which trading *pairs* are possible from coin capabilities; it does not curate a market.

2.4 The market metaphor

The project's naming follows a village-market-square theme, and it maps cleanly onto the components: a *pact* is the trustless deal itself, *posted on the corkboard* (the transport), and *settled into your satchel* (the app and its balances). There is deliberately no "exchange" or "DEX" branding — Pact is a protocol and a daemon, not a venue.

2.5 The first pair

The first supported trading pair is **BTCX** ↔ **BTC**, where *BTCX* is Bitcoin-PoCX. More coins follow — Litecoin (LTC) was the first added third coin — and new Bitcoin-Core-compatible chains can be added as data without recompiling the engine (see the chapter *Coins, Pairs & Capabilities*). For the shipped BTCX ↔ BTC pair the engine selects the v1 HTLC protocol by default; the v2 adaptor protocol is used for Taproot pairs that lack an HTLC option.

Chapter 3

Architecture & Trust Boundaries

Pact's architecture is built around one rule: **the things that can lose your money never leave your machine, and the things that leave your machine can never lose your money.** Understanding where that wall sits is the single most important thing for anyone integrating against, operating, or auditing the system.

3.1 The three moving parts

A running swap involves three categories of component, with a hard trust wall between the first two (yours) and the third (hosted and untrusted):

1. **Your machine — fully trusted.** This is the stack you run and control:
 - **Satchel** (or any RPC client) — the face. It renders the engine's RPC and never touches swap logic.
 - **pactd** — the swap engine. It owns the BIP39 *seed*, derives every key, builds and broadcasts the swap transactions, and runs the scheduler that auto-refunds if a counterparty disappears.
 - Your **coin backends** — a BTCX node and a BTC node (or Electrum backend), reached over their own RPC. These hold the actual coins and the funding wallet.
2. **Hosted, untrusted transport.** A **Nostr relay** (the default) or a **Corkboard** instance (self-hostable). Its only job is to carry identity-signed offers and forward sealed coordination blobs between counterparties. It is replaceable, runs no swap logic, and is assumed hostile.

Note — "Three moving parts" counts the transport as one part even though you can point Pact at several relays or boards at once. They are interchangeable couriers, not authorities.

3.2 The hard wall

The boundary is precise and worth stating in operational terms:

- **Keys and refunds are local, always.** The seed, every derived key, the preimage (v1) and adaptor secret (v2), the pre-signed refund transactions, and the entire decision to redeem or refund live inside `pactd` on your machine. None of this is ever sent to a transport.
- **The transport sees only signed offers and sealed blobs.** What leaves your machine is (a) an *identity-signed offer* — an advert that you are willing to trade, signed by your BIP340 identity key so it cannot be forged or altered — and (b) *recipient-sealed coordination blobs* (PACTSEALED1), encrypted to the counterparty so the relay operator sees ciphertext only. A relay cannot read your coordination, cannot move your funds, and cannot match, execute, custody, or charge.
- **The RPC is loopback-only.** `pactd` binds `127.0.0.1` by default and **enforces** that its listen address is a loopback address — a non-loopback `--listen` aborts boot. The JSON-RPC surface is for local clients (the CLI, Satchel) reading the on-disk cookie; it is never meant to face a network.

Warning — Because the RPC has no remote-access design, do not put `pactd` behind a reverse proxy or expose its port. Anything that can reach the RPC and read the `.cookie` can drive your swaps and your wallet. Keep it loopback.

3.3 Data flow of an offer

At a high level, posting and discovering an offer looks like this:

1. You call an offer-posting RPC on `pactd` with the coins, amounts, and timelocks.
2. `pactd` builds an offer envelope and **signs it with your identity key** (the BIP340 key at `m/7228'/0'/0'`, which never appears in any on-chain script).
3. `pactd` hands the signed envelope to each configured transport, which publishes it (Nostr kind 31510, or a Corkboard row).
4. A counterparty's `pactd`, browsing that transport, sees the signed advert, verifies the signature, and — if they want it — *takes* it, which kicks off the coordination flow.

The transport stored and forwarded a signed advert. It never learned a key.

3.4 Data flow of a swap

Once an offer is taken, the two daemons coordinate to build and settle the swap:

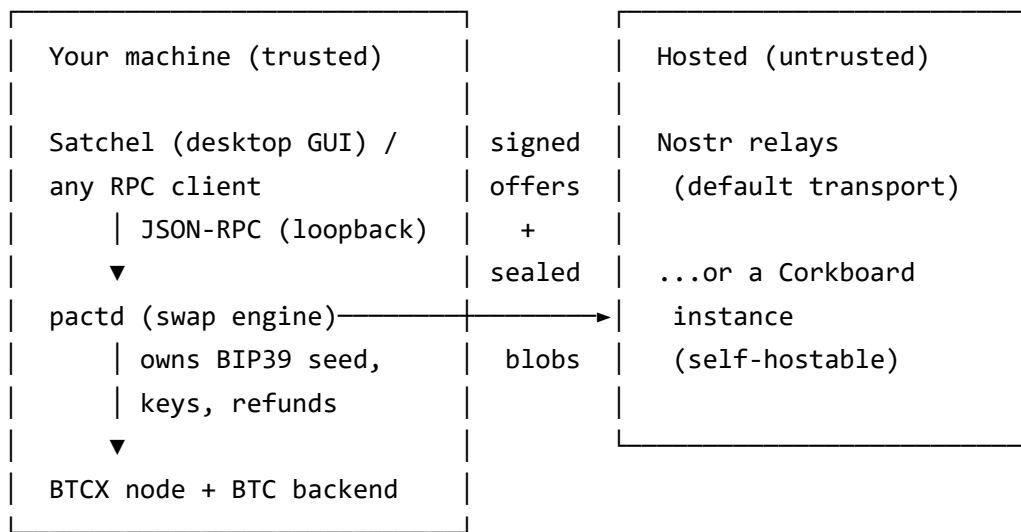
1. The takers exchange a short sequence of **sealed messages** through the transport — funding details, public keys, sweep addresses, and (for v2) the MuSig2 nonces and adaptor signatures. Each message is sealed to its recipient (PACTSEALED1), so the relay forwards opaque ciphertext.
2. Each side independently **reconstructs the on-chain output** (the HTLC P2WSH for v1, or the Taproot output for v2) and byte-compares it before any funds move — neither side trusts the other's claim about the script.
3. The maker funds first; the taker verifies the funding on-chain, then funds its leg. Redemption on one chain reveals the secret (the preimage for v1, the adaptor secret `t` for v2) that

lets the counterparty claim the other chain.

4. Throughout, each `pactd` scheduler tick watches both chains and, if a deadline passes with the swap unfinished, broadcasts the pre-signed (v1) or re-derived (v2) **refund** — no human and no transport required.

The detailed message sequence, the exact scripts, and the timelock margins are covered in the protocol part of this handbook — see the chapters “What Pact Is” and the *swap-protocol chapters*. The point here is structural: all of step 2 through step 4 happen between two local engines and two sets of nodes; the transport only relayed sealed blobs in step 1.

3.5 The picture



Everything left of the arrow is yours and trusted. Everything right of it is hosted, replaceable, and assumed hostile. The arrow carries only signed adverts and sealed ciphertext — never a key, never custody, never a fee.

Part II

Running Pact

Chapter 4

Building from Source

Pact is a Rust workspace. Building the engine and running its tests needs only the Rust toolchain; running the full end-to-end harness additionally needs Python 3 and a regtest bitcoin-pocx build. This chapter covers building the engine, running the test suites, building the Corkboard transport, and where the binaries land.

4.1 Prerequisites

- **Rust** (stable) with cargo. The whole engine — libswap, pactd, and pact-cli — builds with a standard stable toolchain; no nightly features are required.
- **Python 3**, for the end-to-end harness scripts under pact/harness/.
- A **bitcoin-pocx build** providing the regtest BTCX and BTC nodes the harness spins up. The harness drives real bitcoind-shaped nodes on regtest; it does not mock the chain.

Note — The desktop app **Satchel** adds a Node/npm + Tauri toolchain on top of this. That build is covered in the *Satchel User Handbook*; this handbook concerns the engine and its transports only.

4.2 Building and testing the engine

From the pact/ directory:

```
cd pact
cargo build          # build libswap, pactd, pact-cli
cargo test           # unit tests + protocol-vector tests (v1 + v2)
```

cargo test runs the unit tests across the workspace and the deterministic **protocol-vector** tests. Those vector tests (pact/libswap/tests/vectors.rs for v1 and vectors_v2.rs for v2) pin the on-chain construction against the JSON vectors in spec/vectors/ (htlc_v1.json, htlc_v2.json). If you change anything that touches script construction, key derivation, or the adaptor mechanism, these are the tests that must stay green.

4.3 Running the end-to-end harness

The harness performs complete swaps on regtest against real nodes — the highest-fidelity check that the engine works end to end:

```
python harness/test_swap_e2e.py      # full BTCX↔BTC v1 (HTLC) swap on regtest
python harness/test_adaptor_swap.py  # full v2 (Taproot/MuSig2 adaptor) swap
```

`test_swap_e2e.py` exercises the v1 flow: the manual two-CLI happy path, refund (including premature- and late-reveal negatives), the daemon autopilot (scheduler-driven auto-redeem and RBF fee-bump, and auto-refund on both sides), chain-watched funding, balance validation, encrypted-seed create/import, coin setup, and the Corkboard, Nostr-relay, and private-offer flows.

`test_adaptor_swap.py` exercises the v2 flow: the happy path, the single-key CLTV-tapleaf refund and its fee-bump, the CPFP redeem-bump (on BTC and on litecoind), the reveal depth-gate, and the v2 Corkboard flow.

Tip — Both scripts share `regtest_harness.py`, which always brings up BTCX and BTC nodes and only adds an LTC node when constructed with `Harness(with_ltc=True)`. The harness uses `setmocktime` to advance median time on both chains, so timelock-dependent paths (refund, deadline gates) can be tested deterministically without waiting in real time.

4.4 Building Corkboard

Corkboard is the self-hostable transport — a single axum + SQLite/Postgres binary that stores signed offers and blind-relays sealed blobs. Build and run it from its own directory:

```
cd corkboard
cargo build
cargo run -- --listen 127.0.0.1:9780 --db corkboard.sqlite
```

Its default listen address is `127.0.0.1:9780`. Corkboard is optional: the default transport is **Nostr**, which needs no infrastructure — you simply point `pactd` at relays. See the transports part of this handbook for how to wire each one.

4.5 Where the binaries land

`cargo build` produces unoptimised binaries under the workspace `target/debug/` directory; `cargo build --release` produces optimised ones under `target/release/`. From the `pact/` workspace you get `pactd` and `pact-cli`; from `corkboard/` you get the `corkboard` binary. You can run any of them in place with `cargo run -p <crate> -- <args>` without copying the binary out — for example:

```
cargo run -p pactd -- --data-dir ./data --coin btcx=<rpc-url> --coin btc=<rpc-or-electrum-url>
cargo run -p pact-cli -- --data-dir ./data getinfo
```

The next chapter, *Running pactd*, covers the daemon's full command line, RPC authentication, and the scheduler in detail.

Chapter 5

Running pactd

pactd is the swap daemon — the long-lived process that exposes the libswap engine over JSON-RPC 2.0, persists state in SQLite, and runs the scheduler that auto-redeems, auto-refunds, and RBF-fee-bumps with no human present. This chapter is the operator’s reference: the full command line, every flag, RPC authentication, and the scheduler model.

5.1 Synopsis

pactd is a JSON-RPC 2.0 daemon served over **HTTP POST** / (plus an unauthenticated GET /health), loopback-only:

```
pactd [--data-dir <DIR>] [--coins-file <coins.toml>] [--coin <id>=<url[,url]> ...]
      [--coin-confs <id>=<N> ...] [--listen <addr:port>] [--network <net>]
      [--board-url <url[,url]>] [--nostr-relay <wss://...[,...]>] [--auto-fund]
      [--tick-secs <s>] [--once] [--auto-init] [--merchants]
```

No flag is required. With no `--coin`, the daemon starts but cannot move funds on any chain until a backend is attached.

5.2 Flags

Flag	Type	Default	Meaning
<code>--data-dir</code>	path	platform default	Data directory holding the seed, SQLite state, .cookie, and pact.conf. In <code>--merchants</code> mode this is the <i>parent</i> of <code>merchants/<id>/</code> . Defaults bitcoind-style to <code>%APPDATA%\Pact</code> (Windows), <code>~/Library/Application Support/Pact</code> (macOS), <code>~/.pact</code> (elsewhere), with <code>mainnet</code> at the root and <code>testnet/regtest</code> nested beneath — <code>pact-cli</code> autodiscovers the same location.
<code>--coins-file</code>	path	none	A <code>coins.toml</code> adding coins beyond the two built-ins (<code>btcx</code> , <code>btc</code>); merged at startup. A file coin whose id collides with a built-in is dropped. A bad file logs and falls back to the built-ins rather than refusing boot.

Flag	Type	Default	Meaning
--coin	repeatable id=url[,url]	none	Per-coin chain backend. An http:// first URL is the wallet-qualified Core-RPC primary that funds swaps, and any further Electrum URLs (tcp:// / ssl://) are light chain views. A list with no http:// URL (Electrum-only) runs the coin nodeless — a bdk wallet on the Pact seed, with the first Electrum URL serving the wallet (see the chapter <i>Coins, Pairs & Capabilities</i>). The coin id must be in the registry; the last --coin for a given id wins.
--coin-confs	repeatable id=N	network/spacing default	Per-coin confirmation depth (reorg finality, $N \geq 1$). Gates auto-redeem and completion in v1 and v2. Omitted coins use the default heuristic (see the chapter <i>Coins, Pairs & Capabilities</i>).
--listen	addr:port	127.0.0.1:9737	JSON-RPC listen address. Must be loopback — this is enforced; a non-loopback address aborts boot.

Flag	Type	Default	Meaning
<code>--network</code>	string	regtest	One of regtest, testnet, mainnet.
<code>--board-url</code>	string	none	Corkboard base URL(s), comma-separated (the HTTP transport).
<code>--nostr-relay</code>	string	none	Nostr relay wss://... URL(s), comma-separated. Runs alongside any <code>--board-url</code> ; empty or absent disables it.
<code>--auto-fund</code>	flag	false	Auto-fund our leg of swaps (the engine mechanism). The standalone CLI flag is opt-in; Satchel always launches pactd with it on . v2 (adaptor) swaps auto-fund regardless via the autopilot. The end-to-end harness uses this flag to drive manual-vs-auto funding.
<code>--tick-secs</code>	u64	30	Scheduler interval in seconds; 0 disables the background loop entirely.
<code>--once</code>	flag	false	Run a single scheduler pass (<code>sync_board + tick</code>), print the resulting events as JSON, and exit. Exit code is 1 if any event has <code>action == "error"</code> .

Flag	Type	Default	Meaning
<code>--auto-init</code>	flag	false	Create the seed + state on first run (flat layout). No-op if a seed already exists.
<code>--merchants</code>	flag	false	Use the nested <code>merchants/<id>/</code> layout with an in-process registry. Without it, the legacy flat layout (one seed in the <code>data-dir</code> root) is used. Ignored once a flat seed already exists.

Note — The default `--network` is `regtest`. For a real deployment you must set `--network mainnet` (or `testnet`) explicitly; the network selects the chain params, the timelock-margin profile, and the confirmation-depth defaults.

Warning — `--listen` is loopback-enforced by design. Do not try to bind a routable address to share a daemon — it will refuse to start. The RPC has no remote-access model; see the chapter *Architecture & Trust Boundaries*.

Note — The standalone `--auto-fund` flag is opt-in (off by default), but **Satchel always launches `pactd` with it on** — auto-funding is the single, always-on behaviour for app users. It is safe as a default because offers are one-shot: a taken offer cannot be re-taken, so a maker’s exposure is bounded by the size of the offers they posted. (v2 adaptor swaps auto-fund regardless, via the autopilot.)

5.3 Logging

`pactd` writes its tracing output to **both** stdout and a rolling daily file at `<data-dir>/logs/pactd.log.<date>` (for example `logs/pactd.log.2026-06-20`). The file exists because a managed daemon spawned by Satchel has no stdout capture, so the file is the only record of what the engine did.

- The scheduler tags every swap event (`tracing::info!(swap, action, detail)`), so swap narration is captured per swap. The `dumpswap` RPC extracts the lines for one `swap_id` from these files.
- Verbosity follows `RUST_LOG` (an `EnvFilter`); the default level is `INFO`.
- **No secrets are ever logged.** Seeds, passphrases, the v1 preimage, and MuSig2 nonces are never passed to tracing, so the log file is safe to share for diagnostics.

5.4 RPC authentication

Authentication is HTTP Basic, modelled on bitcoind:

- **Cookie (always on).** On startup `pactd` writes `<data-dir>/cookie` containing `__cookie__:<32-byte hex>`, regenerated per run, and removes it on clean shutdown. Local clients (the CLI, Satchel) read this file — this is the zero-config default.
- **pact.conf (optional).** A `<data-dir>/pact.conf` file of `key = value` lines (`#` for comments) may set `rpcuser` and `rpcpassword`; those credentials are accepted *alongside* the cookie.
- Both forms are expanded to Basic `<base64(user:pass)>` and compared in constant time. The `GET /health` endpoint is unauthenticated.

Note — To open an *encrypted* seed at boot without an interactive unlock, set the `PACT_PASSPHRASE` environment variable; the daemon uses it to decrypt the seed on startup. See the chapter *Seeds, Wallets & Merchants*.

The default RPC port is **9737**. (Coin peer-to-peer ports live in each coin’s chain params and are unrelated to the RPC port.)

5.5 The scheduler tick model

`pactd` runs a background loop that fires every `--tick-secs` seconds (default 30). Each pass does two things: it syncs configured boards/relays (pulling new offers and inbound sealed messages) and it runs the engine `tick`, which advances every live swap’s state machine — funding when due, redeeming when the secret is available, fee-bumping a stuck transaction, and broadcasting a refund when a timelock deadline passes. This is the mechanism that makes swaps *unattended*: once a swap is underway, the scheduler will redeem your winnings and refund your losses without any further RPC calls.

Setting `--tick-secs 0` disables the loop; the engine then advances only when you call `tick` (or run with `--once`). That is useful for tests and for externally orchestrated setups, but for a normal deployment leave the scheduler running — a disabled scheduler means **no auto-refund**, which can lose funds if a counterparty disappears.

Warning — Do not run a mainnet maker with `--tick-secs 0` and walk away. The auto-refund safety net only fires on scheduler ticks; with the loop off, a stalled swap will sit unrefunded until you manually tick it.

5.6 Example invocations

A regtest daemon that auto-creates its seed and runs over a local Corkboard:

```
pactd --data-dir ./data \  
      --network regtest \  
      --auto-init \  
      --tick-secs 0
```

```
--coin btcx=http://user:pass@127.0.0.1:19443/wallet/swap \  
--coin btc=http://user:pass@127.0.0.1:18443/wallet/swap \  
--board-url http://127.0.0.1:9780
```

A more realistic mainnet-shaped daemon: nested merchant layout, an encrypted seed opened from the environment, the default Nostr transport, and an explicit per-coin confirmation override:

```
PACT_PASSPHRASE='...' pactd \  
  --data-dir /var/lib/pact \  
  --network mainnet \  
  --merchants \  
  --coin btcx=http://user:pass@127.0.0.1:9332/wallet/swap \  
  --coin btc=http://user:pass@127.0.0.1:8332/wallet/swap \  
  --coin-confs btc=6 --coin-confs btcx=10 \  
  --nostr-relay wss://relay.example.com,wss://relay2.example.com \  
  --tick-secs 30
```

Here `--merchants` puts each seed under `merchants/<id>/`, `PACT_PASSPHRASE` unlocks the encrypted seed at boot, the two `--coin` flags attach wallet-qualified Core-RPC backends that fund the swap legs, and `--coin-confs` overrides the per-coin finality depth. The defaults for coins, pairs, and confirmation depth are explained in the next chapter.

Chapter 6

Coins, Pairs & Capabilities

Pact trades between *coins*. A coin is a Bitcoin-Core-compatible chain the engine knows how to verify, derive keys for, and build swap outputs on. Trading *pairs* are not curated — they are **derived** from what each pair of coins can do. This chapter covers the built-in coins, adding coins as data, attaching backends with `--coin`, setting confirmation depths, and how capabilities turn into pairs.

6.1 Built-in coins

Two coins ship in the engine code itself, fully trusted: `btcx` (Bitcoin-PoCX) and `btc` (Bitcoin). These are the engine's *registry* coins — their consensus parameters are compiled in and cannot be redirected by a stray file.

6.2 Adding coins with `coins.toml`

Any Bitcoin-Core-compatible chain can be added **as data**, with no recompile, through a `coins.toml` passed with `--coins-file`. The file is loaded at startup and merged with the built-ins; a file coin whose id collides with a built-in is dropped (so a stray file can never redirect `btc` or `btcx`), and a malformed file logs an error and falls back to the built-ins rather than refusing to boot.

Only the **consensus** fields the engine needs are read — enough to verify genesis, build and parse addresses, and derive keys. A minimal entry looks like:

```
schema_version = 1

[[coin]]
coin_id        = "ltc"
display_name    = "Litecoin"
symbol         = "LTC"
decimals       = 8
bip32_coin_type = 2
```

```
target_spacing_secs = 150
```

[coin.capabilities]

```
cltv      = true  
segwit_v0 = true  
taproot   = true
```

[coin.mainnet.consensus]

```
genesis_hash = "12a765e31ffd4059bada1e25190f6e98c99d9714d334efa41a195a7e7e04bfe2"  
bech32_hrp   = "ltc"
```

[coin.regtest.consensus]

```
genesis_hash = "530827f38f93b43ed12af0b3ad25a288dc02ed74d6d7857862df51fc56c416f9"  
bech32_hrp   = "r1tc"
```

Note — The consensus values in `coins.toml` are trusted by whoever edits the file, but they are not taken on faith at runtime: before any funds move, the engine validates each coin's `genesis_hash` against the live node (`getblockhash 0`). A wrong genesis hash fails the chain check, not a swap. A `coins.toml` entry may also carry a connection sub-table with RPC defaults; that is Satchel's concern and is ignored by the engine.

6.2.1 Nodeless (Electrum-only) coins

A coin whose backend list has **no** `http://primary` runs **nodeless** (epic #58): the first `tcp://ssl://` URL becomes a bdk wallet on the Pact seed's BIP-86 branch (`m/86' /<bip32_coin_type>' /0'`), synced over the same raw Electrum calls the chain-data backends use; any further URLs join as independent chain views. All nine `wallet_*` operations — funding, the v2 two-phase build, CFP, RBF bump — are served by that wallet, and the `listtransactions` RPC exposes its activity feed. Rules and guarantees:

- **Mainnet requires ≥ 2 Electrum servers** — a single lying or withholding server must never be the only view of the chain while funds move (spec §10).
- Every server passes a **capability handshake** before use: `server.version` (protocol 1.4+), `server.features` cross-checks (`genesis_hash` must match; **pruned servers are refused** — a restored seed's history scan would be silently incomplete), and a deep check that fetches header 0 and hashes it locally (which also validates PoCX's 286-byte headers against that server).
- **Address handout is capped** (`MAX_UNUSED_AHEAD = 20`, `wallet_bdk.rs`): `getnewaddress` reveals fresh external addresses only while fewer than 20 revealed-but-unused ones are outstanding; past the cap it recycles the oldest unused address instead. The restore scan's `STOP_GAP = 25` therefore always covers the real address gap by construction — a wallet restored from the seed alone finds every coin, with no deep-rescan affordance needed.
- A locked or absent seed degrades to **chain-reads-only** and surfaces as `wallet_locked` —

exactly like an encrypted, locked Core wallet.

- Default server lists ship per coin in `coins.toml` (`connection.electrum = [...]`) and pre-fill Satchel's setup form.
- An Electrum-FIRST list must be Electrum-only; with a Core-RPC primary, Electrum URLs remain plain chain-data views as before.

6.2.2 Per-coin minimum feerate

`pactd` derives its funding and spend feerate from the node's `estimatesmartfee`, falling back to 1 sat/vB (Bitcoin's floor) on a quiet or fresh chain. Some chains bake a higher wallet floor: litecoind's `-mintxfee` defaults to ~10 sat/vB, and because that floor is exposed by **no RPC**, the engine cannot discover it at runtime — a spend below it is simply rejected (-6, "lower than the minimum fee rate setting"), so swaps on that coin can't fund at all.

Carry the floor as data instead. Each `[coin.<network>]` block takes an optional `min_feerate_sat_vb` (integer sat/vB, default 1); the engine floors every funding and spend feerate at it, so a coin's own floor always wins:

[coin.mainnet]

```
min_feerate_sat_vb = 10    # litecoind's wallet floor; optional, defaults to 1
consensus = { genesis_hash = "12a7...", bech32_hrp = "ltc", ... }
```

The bundled `ltc` coin ships `min_feerate_sat_vb = 10` on mainnet, testnet, and regtest; the built-in `btc` and `btcx` coins use the 1 default.

6.2.3 The %NODEDIR% datadir token

A connection sub-table's `datadir` value may use a `%NODEDIR%/<Name>` token. Satchel expands it to the node's **real per-OS default data directory** for a Bitcoin-Core-family node named `<Name>`, which is where that node writes its `.cookie`:

OS	%NODEDIR%/<Name> resolves to
Windows	%LOCALAPPDATA%\<Name>
macOS	~/Library/Application Support/<Name>
Linux	~/.<name> (lowercased)

This is why the bundled templates use `%NODEDIR%/Bitcoin-PoCX`, `%NODEDIR%/Bitcoin`, and `%NODEDIR%/Litecoin`: each resolves to the correct cookie path on every OS without per-platform editing. The `datadir` field also understands `~`, `%LOCALAPPDATA%`, and `%APPDATA%`; anything else is left literal.

6.3 Attaching backends with --coin

A coin in the registry is *configured* by attaching a chain backend with `--coin id=url[,url]`:

- The **first URL is the wallet-qualified Core-RPC primary** — the bitcoind-style RPC, including the wallet path, that actually funds and broadcasts swap transactions. For example: `--coin btcx=http://user:pass@127.0.0.1:9332/wallet/swap`.
- Any **additional URLs may be Electrum** backends (`tcp://` or `ssl://`), used for light chain queries.
- A list with **no** `http://` URL at all runs the coin **nodeless** — see “Nodeless (Electrum-only) coins” above.

The coin id must already be in the registry (built-in or `coins.toml`-added); the flag is repeatable, once per coin, and the last `--coin` for a given id wins. This is the single, uniform way every coin is wired — there is no per-coin special-casing.

6.4 Confirmation depth with `--coin-confs`

Each coin has a *confirmation depth* N — how many confirmations a funding or redeem needs before the engine treats it as final. This gates auto-redeem and swap completion in both v1 and v2, and it is your reorg-safety knob. Override it per coin with `--coin-confs id=N` ($N \geq 1$); coins you don’t override use a default heuristic based on the network and the chain’s block spacing:

Chain profile	Default N
Regtest (any coin)	1
Fast chain — block spacing under 5 minutes (e.g. BTCX, 120 s)	10
Slow chain — block spacing 5 minutes or more (e.g. BTC, 600 s)	6

Note — The fast-chain default is deliberately higher than the slow-chain one: a faster chain produces more blocks in the same wall-clock reorg window, so it needs more confirmations to reach comparable finality. Whatever you set, the floor is 1.

6.5 Capabilities and how pairs are derived

Each coin declares three capabilities: `c1tv` (OP_CHECKLOCKTIMEVERIFY, i.e. usable timelocks), `segwit_v0` (P2WSH outputs), and `taproot` (P2TR / v1 segwit outputs). These capabilities determine which *protocols* a pair of coins can use, and therefore which pairs are tradable at all — the engine **derives** the pair list, it never curates one:

- A pair where both coins have `c1tv` + `segwit_v0` can run the **v1 HTLC** protocol.
- A pair where both coins have `taproot` can run the **v2 adaptor** protocol.

When both protocols are possible, the engine **prefers HTLC**; the v2 adaptor protocol is selected only for Taproot pairs that lack an HTLC option. The shipped BTCX ↔ BTC pair therefore defaults to HTLC. (Note that v2 adaptor swaps are enabled on every network, including mainnet.)

6.6 Inspecting coins and pairs

Three RPC methods report the live state of your coin and pair configuration — covered in full in the API part of this handbook, but worth knowing here:

- `listcoins` — every registry coin with its capabilities, whether it is configured, a live status probe, tip height, genesis hash, bech32 HRP, and its effective and default confirmation depth.
- `listpairs` — every derivable pair with its protocol list (htlc / adaptor), whether both legs are configured, and whether it is currently available.
- `validatecoin` — a genesis check of a *proposed* backend for a coin, without touching the engine's live configuration; use it to confirm a node before you attach it.

See the chapter on coins-and-pairs RPCs for the full field lists and return shapes.

Chapter 7

Seeds, Wallets & Merchants

Every key Pact ever uses — identity, swap keys, preimages, adaptor secrets, refund keys — is derived from a single BIP39 *seed*. This chapter covers the seed's lifecycle (create, import, encrypt, unlock), the encrypted-seed format, the *merchant* model (one seed per data directory), and what lives on disk.

7.1 The seed lifecycle

A data directory starts with **no seed**. From there, exactly one seed is established, by one of these routes:

- `createseed` — generate and persist a fresh seed; the mnemonic is returned **once** and never again.
- `importseed` — install an existing BIP39 mnemonic.
- `--auto-init` — at boot, `pactd` creates the seed and state on first run (flat layout) if none exists. Used by launchers like Satchel.
- `PACT_PASSPHRASE` — supplies the passphrase to *open an encrypted seed* at boot; it does not create one.

Once established, the seed is either **plaintext** (the regtest intent) or **encrypted** — created with a passphrase, in which case it lands *locked* and stays unusable until you `unlock` it (or supply `PACT_PASSPHRASE` at boot).

The daemon reports exactly where it stands through `WalletStatus`:

```
{ "seed_exists": true, "encrypted": true, "locked": false }
```

`locked` is true precisely when the seed is encrypted *and* the passphrase is not currently held in memory. `unlock` verifies a passphrase by trial-decryption and then holds it in memory for the process lifetime; `walletstatus` and `getinfo` both surface these three flags.

Note — A plaintext seed is appropriate for regtest and disposable test setups. For testnet or mainnet, create the seed with a passphrase so it is encrypted at rest, and supply `PACT_PASSPHRASE` (or call `unlock`) to open it.

7.2 The encrypted seed format

The seed file is `seed.mnemonic`. In plaintext form it holds the BIP39 mnemonic directly. Encrypted, it is a single line:

```
PACTSEEDv1:<salt>:<nonce>:<ciphertext>
```

The mnemonic is encrypted with **ChaCha20-Poly1305** under a key derived by **scrypt** ($N = 2^{15}$, $r = 8$, $p = 1$) from the passphrase and per-file salt.

Warning — Seed installation is non-overwriting and atomic: `install_seed` **refuses to overwrite** an existing seed, and writes via a temp file with `fsync` and `rename` so a crash mid-write cannot corrupt or truncate the seed. To replace a seed you must remove the data directory's seed deliberately — the engine will never clobber it for you.

7.3 The merchant model

A *merchant* is one identity backed by one seed in one data directory. Pact supports two layouts:

- **Flat (default).** A single seed lives in the data-dir root. This is the layout the harness and `pact-cli` rely on, and what `--auto-init` produces. It is internally a single default merchant.
- **Nested (`--merchants`).** Each merchant lives under `<data-dir>/merchants/<id>/`, with a `merchants.json` manifest in the parent. `pactd` owns an in-process registry; one merchant is *active* at a time, and the merchant RPCs (`createmerchant`, `listmerchants`, `loadmerchant`, `renamemerchant`, `unloadmerchant`, `getmerchantinfo`) create, rename, and switch between them at runtime. This is the layout Satchel uses to manage several trading identities.

Warning — Switching identities has a fund-safety guard: `loadmerchant` and `unloadmerchant` **refuse to switch away from a merchant that has a live (non-terminal) swap**. This prevents you from orphaning a swap that still needs its scheduler ticks to redeem or refund. Finish, refund, or let the swap reach a terminal state before switching away.

The `--merchants` flag is ignored once a flat seed already exists in the data-dir root — an existing flat install stays flat.

7.4 What lives on disk

Per merchant data directory (the root in flat mode, `merchants/<id>/` in nested mode):

File	Contents
<code>pact.sqlite</code>	All swap, offer, nonce, and Nostr state (see below).

File	Contents
seed.mnemonic	The BIP39 seed — plaintext, or PACTSEEDv1:... if encrypted.
.cookie	The per-run RPC cookie (data-dir root only).
pact.conf	Optional rpcuser / rpcpassword for RPC auth.
merchants.json	The merchant manifest (parent data dir, nested mode only).
logs/pactd.log.<date>	Rolling daily log files (data-dir root). Secret-free; see the chapter “Running pactd”.

The SQLite database is the engine’s durable state. At a high level its tables are: `swaps` and `adaptor_swaps` (the v1 and v2 swap records), `meta` (counters, the board `relay_cursor`, and private offers), `pending_takes`, `nonce_sessions` (the v2 MuSig2 use-once nonce state machine), `my_offers`, and the Nostr `nostr_outbox` / `nostr_inbox` / `nostr_offer_cache` tables.

Note — Records are strict: a missing required field fails the load rather than silently defaulting. There is no `serde`-default migration path — this is a deliberate “no backward compatibility” stance, so do not hand-edit the SQLite state and expect old shapes to be tolerated.

7.5 Seed-only swap rescue (#54)

Losing the data directory entirely — a dead machine, a wiped disk — no longer strands an in-flight swap. `pactd` opportunistically backs up in-flight swap state to the configured Nostr relays, **encrypted to its own identity key** (sealed-to-self, kind 31512; see the chapter “Nostr Transport”), so a fresh install restored from the seed alone can rediscover and finish or refund a swap the local database has no record of — no separate backup step, and nothing readable by anyone but you.

Snapshots are minimal and sparse, published at exactly these points:

- **v1** — once, at accepted. Any funding either side commits is then refundable (from the derived keys) and, once the counterparty’s leg is spent on-chain, completable (the preimage extracts from the redeem witness).
- **v2** — at accepted, and again at signed. The signed snapshot additionally carries the assembled adaptor signatures — the *only* datum that is neither seed- nor chain-derivable — so a rescued swap can be **completed**, not just refunded.

Everything after that is re-derived from chain-watching once the snapshot is adopted; the scheduler drives a rescued swap exactly like any other. A swap that reaches a terminal state (completed, refunded, aborted) publishes a NIP-09 tombstone so a machine restored later never resurrects a finished swap; even a missed tombstone is safe, since re-driving a settled swap just detects the spend and idempotently finalizes it.

7.5.1 Restore is gated, never automatic

On boot, on unlock, and on every merchant load, `pactd` runs a **read-only** preview (`Engine::rescue_preview`) against the configured relays and only **warns** if rescuable snapshots exist — it never adopts one on its own. Two live machines driving the same swap from one seed can double-fund it, so adoption is an explicit, human decision:

- `restorefromrelay` — adopts every rescuable snapshot the relays hold that isn't already a local record, returning `{ restored, seen }`. Only call this once the machine that ran those swaps is genuinely retired.
- `rescuestatus` — the read-only twin: reports `{ pending, seen, warning }` without adopting anything, for a status check or a UI badge.
- CLI: `pact-cli restore / pact-cli rescue-status`.

A restored swap's local record always wins over a snapshot (adoption is skipped if we already hold that `swap_id`), and restoring also raises the seed's next-swap-index high-water mark from the snapshots, so a reissued index can never reuse a completed swap's keys.

7.5.2 Anchored participant keys

Restoring on a new machine has to re-derive the **participant's** swap keys too — not just the initiator's. Since rc8, the two roles index their keys differently (spec §4.2): the initiator still uses its local counter `i`, but the participant derives its swap and refund keys from the swap's own public anchor (H for v1, the adaptor point T for v2) via a hardened tagged-hash path, needing no counter at all. Two machines holding the same seed therefore derive the identical key for the identical swap, and can never issue one key for two different swaps. `swap_index` is `Option<u32>` in both record types — `None` means "participant, anchored." Existing pre-rc8 records keep their original counter-based derivation; this is fully backward compatible with databases created before this change.

Warning — Only swaps **started on rc8 or later** are covered — a swap already in flight when you upgrade was never snapshotted and is not retroactively rescuable. And a v2 **maker** wiped in the narrow window between accepted and signed cannot complete the swap even after restoring: the assembled adaptor signatures are the one datum that isn't seed- or chain-derivable, and re-running the MuSig2 handshake is structurally forbidden (persisted nonce sessions refuse to sign twice, by design). The timelocked refund is the exit in that case.

See the chapter "Nostr Transport" for the wire format (kind 31512, the opaque per-swap d-tag, the tombstone event), and the Satchel handbook's "Backup, Seeds & Safety" chapter for the rescue story told for end users.

Chapter 8

The pact-cli

`pact-cli` is the thin command-line client for `pactd` — the `bitcoin-cli` of this stack. It is a hand-rolled HTTP caller (a raw `TcpStream` with a 120-second read timeout and chunked-response decoding) that maps subcommands onto JSON-RPC methods. This chapter covers its global flags and auth discovery, direct method dispatch, the structured subcommand table, and a worked end-to-end v1 swap.

8.1 Direct dispatch and help

Every RPC method is a subcommand of its own name — the first token that is not a structured subcommand is sent to the daemon as the method, with the remaining tokens as params (each JSON-parsed if it parses, else passed as a string):

```
pact-cli getinfo
pact-cli getbalance btc
pact-cli adaptorinit btcx:1.0 btc:0.95 86400 43200
```

`pact-cli help` asks the daemon for its full method catalog by category; `pact-cli help <method>` explains one method; `pact-cli listmethods` returns a machine-readable name array. A typo'd method name is answered with the nearest real one (unknown method `'getblance'` — did you mean `'getbalance'`?). Clap's own usage text stays on `pact-cli --help`.

8.2 Global flags

These apply to every invocation:

Flag	Default	Meaning
<code>--rpc</code>	derived from where the auth was found (see below)	The <code>pactd</code> JSON-RPC endpoint.
<code>--data-dir</code>	autodiscovered	Data directory to read the <code>.cookie</code> (or <code>pact.conf</code>) from.

Flag	Default	Meaning
<code>--network</code>	regtest	Network subdir the auth discovery looks under (mirrors pactd).
<code>--rpcuser</code>	—	Explicit RPC username (overrides cookie auth).
<code>--rpcpassword</code>	—	Explicit RPC password.

Authentication mirrors `bitcoin-cli`: explicit `--rpcuser/--rpcpassword` win; an explicit `--data-dir` is read strictly (its `.cookie`, else the `rpcuser/rpcpassword` in its `pact.conf`). With neither flag, the CLI searches, in order:

1. the pactd platform default — `%APPDATA%\Pact` (Windows), `~/Library/Application Support/Pact` (macOS), `~/ .pact` (elsewhere) — mainnet at the root, testnet/regtest nested per `--network`;
2. Satchel's managed pactd dir (`<app-local-data>/org.pocx.satchel/[net]/pactd`).

When `--rpc` is omitted, the default URL follows **where the auth was found**: a cookie in the platform-default dir means a hand-run pactd, which binds `http://127.0.0.1:9737` whatever the network; a cookie in Satchel's managed dir means Satchel's daemon, which offsets its listen port per network — 9737 (mainnet) / 9738 (testnet) / 9739 (regtest). So `pact-cli --network regtest getbalance btcx` reaches Satchel's regtest daemon with no `--rpc` at all, and can never dial the mainnet port by accident. With an explicit `--data-dir` or `--rpcuser/--rpcpassword`, the default stays `http://127.0.0.1:9737`.

So against a default local pactd — or Satchel's — `pact-cli getbalance btc` works with no flags at all.

8.3 Structured subcommands

A handful of operations additionally get structured subcommands — they wrap the RPC plus the file I/O of the manual v1 handshake, or add flag-style arguments:

Subcommand	Flags / args	RPC method
<code>call <method> [params...]</code>	each param JSON-parsed, else treated as a string	any (generic passthrough)
<code>offer</code>	<code>--give --get --t1 --t2 --out</code>	<code>offer</code>
<code>accept</code>	<code>--in --out</code>	<code>acceptoffer</code>
<code>recv</code>	<code>--in</code>	<code>recv</code>
<code>fund</code>	<code>--swap --out</code>	<code>fund</code>
<code>redeem</code>	<code>--swap</code>	<code>redeem</code>
<code>refund</code>	<code>--swap</code>	<code>refund</code>

Subcommand	Flags / args	RPC method
abort	--swap --reason	abort
status	--swap (optional)	getswap / listswaps
restore	—	restorefromrelay
rescue-status	—	rescuestatus
walletstatus	—	walletstatus
transactions <coin>	positional coin id — nodeless (Electrum-backed) coins only	listtransactions
coins	—	listcoins
pairs	—	listpairs
validatecoin	--coin --backend	validatecoin
createseed	--passphrase (optional), --words (12 default 24)	createseed
importseed	--mnemonic --passphrase (optional)	importseed
unlock	--passphrase	unlock
board post	--give --get --t1_secs (default 12h) --t2_secs (default 6h)	boardpostoffer
board offers	—	boardlistoffers
board take	--offer	boardtake
board revoke	--offer	boardrevoke
board sync	—	tick

8.4 The explicit call spelling

`pact-cli call <method> [params...]` is the explicit passthrough spelling of direct dispatch — byte-for-byte the same request. It exists for scripts that want to make “this token is an RPC method, not a CLI subcommand” unambiguous (a structured subcommand name always wins over a same-named method in the bare form).

8.5 Worked example: a v1 swap with two CLIs

This is the manual happy path the harness drives in `test_swap_e2e.py`, with two daemons — **Alice** (the initiator, giving BTCX, getting BTC) and **Bob** (the participant). Each `pact-cli` here is assumed pointed at its own daemon’s `--rpc / --data-dir`; messages are passed between the two as JSON files.

1. **Alice makes the offer.** She gives `btcx`, gets `btc`, with timelocks `t1` (her chain-A refund) and `t2 < t1` (Bob’s chain-B refund). This writes an init envelope:

```
alice> pact-cli offer --give btcx:1.0 --get btc:0.95 \
```

```
--t1 <T1> --t2 <T2> --out init.json
```

2. **Bob accepts.** He reads the init envelope, reconstructs the HTLC scripts locally, and emits an accept envelope:

```
bob> pact-cli accept --in init.json --out accept.json
```

3. **Alice receives the acceptance** and funds her chain-A leg first (the maker funds first), producing a funded message that names the funding output:

```
alice> pact-cli recv --in accept.json
alice> pact-cli fund --swap <swap_id> --out funded_a.json
```

4. **Bob verifies Alice's funding on-chain, then funds his chain-B leg:**

```
bob> pact-cli recv --in funded_a.json
bob> pact-cli fund --swap <swap_id> --out funded_b.json
```

5. **Alice receives Bob's funding, then redeems chain B** — this is the step that reveals her preimage s on the BTC chain:

```
alice> pact-cli recv --in funded_b.json
alice> pact-cli redeem --swap <swap_id>      # reveals  $s$  on chain B
```

6. **Bob redeems chain A.** His engine extracts s from Alice's chain-B spend and uses it to claim the BTCX leg:

```
bob> pact-cli redeem --swap <swap_id>      # engine extracted  $s$  from chain B
```

Both HTLCs are now spent and the swap is completed. The `swap_id` is the 16-hex identifier returned by offer (and visible via status).

Tip — In a real deployment you do not perform steps 5 and 6 by hand. With the scheduler running (the default `--tick-secs 30`), each side's daemon redeems automatically once the required confirmations are reached, and auto-refunds if a deadline passes — see the chapters *"Running pactd"* and *the swap-protocol chapters*. The manual flow above is the way to understand and test the protocol, not the way to operate it.

Warning — The timelock ordering is load-bearing: t_2 (Bob's refund) must be earlier than t_1 (Alice's refund), and both must respect the network's action-deadline margins. The engine validates these offsets at offer time and will reject an unsafe pair; do not work around the rejection. See the timelock material in the protocol part of this handbook.

Part III

The JSON-RPC API

Chapter 9

JSON-RPC Conventions

pactd exposes the libswap engine over **JSON-RPC 2.0**. This chapter covers the transport, authentication, request/response envelope, and the common error shapes that every method shares. The per-method reference lives in the chapters that follow ("API: Node, Seed, Merchants, Coins", "API: v1 HTLC Swaps", "API: v2 Adaptor Swaps", and "API: Board, Private Offers & Fees").

Note — pactd is to the swap engine what bitcoind is to Bitcoin: a long-running daemon driven entirely over RPC. pact-cli is the thin command-line client (the bitcoin-cli analog), and Satchel is the desktop GUI (the bitcoin-qt analog).

9.1 Transport

Property	Value
Protocol	JSON-RPC 2.0 over HTTP
Method endpoint	POST /
Health endpoint	GET /health (returns ok, unauthenticated)
Default listen address	127.0.0.1:9737
Binding	Loopback only — a non-loopback --listen aborts boot

All RPC calls are HTTP POST to the root path /. The daemon enforces that its listen address is a loopback address (127.0.0.1/: :1); binding to a routable interface is rejected at startup. To reach pactd from another host, front it with your own authenticated tunnel — never expose the port directly.

GET /health is the only unauthenticated route. It returns the literal string ok and is intended for liveness probes.

9.2 Authentication

pactd uses **HTTP Basic** auth, exactly like bitcoind. Two credential sources are accepted (either works):

1. **Cookie (always written).** On startup `pactd` writes `<datadir>/cookie` containing `__cookie__:<hex>`, where `<hex>` is a fresh 32-byte random value rendered as hex. The file is rewritten per run and removed on a clean shutdown. Use `__cookie__` as the username and the hex as the password.
2. **pact.conf (optional).** A `<datadir>/pact.conf` file with `key = value` lines (`#` introduces a comment). If it sets `rpcuser` and `rpcpassword`, those credentials are accepted **alongside** the cookie.

Both sources are normalized to a Basic `<base64(user:password)>` header and compared in constant time. `pact-cli` reads `cookie` automatically when you pass `--data-dir`; pass `-rpcuser/--rpcpassword` to override.

Tip — The passphrase that unlocks an encrypted seed is **not** an RPC credential. Supply it via the `unlock` method or the `PACT_PASSPHRASE` environment variable at boot. See the chapter “Wallet & Seed Lifecycle”.

9.3 Request shape

Every request is a JSON object:

```
{ "jsonrpc": "2.0", "id": 1, "method": "getinfo", "params": [] }
```

Field	Required	Notes
<code>jsonrpc</code>	no	Ignored on input; the response always sets "2.0".
<code>id</code>	no	Echoed verbatim in the response.
<code>method</code>	yes	One of the methods in the following chapters.
<code>params</code>	no	Defaults to null. See below.

Params accept a positional array OR a named object. These two calls are equivalent:

```
{ "method": "offer", "params": ["btcx:1.0", "btc:0.5", 144, 72] }
{ "method": "offer",
  "params": { "give": "btcx:1.0", "get": "btc:0.5", "t1": 144, "t2": 72 } }
```

Numeric params also accept numeric strings (e.g. "144"), matching `bitcoin-cli` behaviour. A missing required param fails with `missing param '<name>' (positional form adds (position <i>))`.

9.4 Response shape

A successful call returns:

```
{ "jsonrpc": "2.0", "id": 1, "result": { "...": "..." } }
```

An error returns an error object instead of result:

```
{ "jsonrpc": "2.0", "id": 1,
  "error": { "code": -32601,
            "message": "unknown method 'getblance' – did you mean 'getbalance'? (see 'help'

```

Error	Code	Message text
Unknown method	-32601	unknown method '<method>' – did you mean '<nearest>'? (see 'help') (the <i>did-you-mean</i> hint appears only when the name is plausibly a typo)
Missing param	-1	missing param '<name>'
No merchant loaded	-1	no active merchant – create or load one first

An unknown method returns JSON-RPC's standard *method not found* code -32601; every other error is code -1, with the human-readable detail in message.

Note — All swap, board, offer, and seed RPCs operate on the **active merchant's** engine. If no merchant is loaded (fresh nested-mode datadir with nothing selected), they fail with no active merchant – create or load one first. Use `createmerchant / loadmerchant` first. See the chapter “API: Node, Seed, Merchants, Coins”.

9.5 Examples

A raw `curl` call against the cookie file:

```
# Cookie file holds "__cookie__:<hex>"; pass it straight to --user.
curl -s --user "$(cat ~/.pact/.cookie)" \
  -H 'content-type: application/json' \
  -d '{"jsonrpc":"2.0","id":1,"method":"getinfo","params":[]}' \
  http://127.0.0.1:9737/
```

The same call through `pact-cli` (which autodiscovers the `.cookie` — an explicit `--data-dir` also works):

```
pact-cli getinfo
```

Any method is callable this way — `pact-cli <method> [params...]` — with each argument parsed as JSON if possible, otherwise treated as a string. `pact-cli help` lists the daemon's full catalog — all **64 public methods**, the same list `listmethods` returns as a name array — and an unknown method is refused with JSON-RPC code -32601 and a *did-you-mean* suggestion (all other errors are code -1 today).

Note — `platform_fee_sat` is always 0. There are no platform fees anywhere in the engine; the field exists only so fee previews have a consistent shape. See the chapter

"API: Board, Private Offers & Fees".

Chapter 10

API: Node, Seed, Merchants, Coins

This chapter documents the non-swap RPC surface: node introspection, the seed lifecycle, the merchant model, the coin/pair registry, and the wallet helper methods. Conventions (transport, auth, request/response shape, the *no active merchant* error) are covered in the chapter “JSON-RPC Conventions”.

10.1 Node / info

Method	Params	Returns	Mutates
getinfo	—	{ name, version, protocol, wire_epochs, network, identity?, seed_exists, encrypted, locked, coins, watch_only }	no
walletstatus	—	{ seed_exists, encrypted, locked }	no
setwatchonly	on	{ watch_only }	yes (live + persisted)
help	method?	plain-text catalog (string)	no
listmethods	—	[name, ...]	no
stop	—	"pactd stopping"	yes (lifecycle)

- `getinfo` — name is always "pactd"; version is the crate version; protocol is the swap protocol version; wire_epochs maps each protocol family to the wire-compatibility epoch this build speaks (rc10: { "pact-htlc-v1": 1, "pact-htlc-v2": 2 }) — a UI badges

offers whose signed wire differs as un-takeable; network is the lowercased network name (regtest/testnet/mainnet); coins is the list of configured coin ids; watch_only is the active merchant's watch-only flag (see below). Tolerates a missing or locked seed — identity is null until a seed is present **and** unlocked.

- walletstatus — the seed state triple. locked is true only when the seed is encrypted **and** its passphrase is not held in memory.
- setwatchonly — enters (on: true) or leaves (on: false) **watch-only mode** for the active merchant. A watch-only session may browse the board and withdraw its own offers, but the engine **blocks** posting, taking, and funding (and no-ops offer-liveness management for another session). The flag is persisted per-merchant in pactd's store and applied live (no relaunch); getinfo.watch_only reports it, letting a UI skip the ≥ 2 -coin first-run gate. Returns the new value.
- help — with no param, the daemon's full method catalog grouped by category, rendered as plain text (the CLI prints string results raw, so it reads like a man page); with a method name, that one method's arguments and summary. This catalog is the authoritative live method list — the same one that drives the CLI's *did-you-mean* suggestion.
- listmethods — the same catalog as a machine-readable JSON array of method names.
- stop — requests a graceful shutdown and returns immediately.

10.1.1 Fee policy

The active merchant's local fee-bump policy — the knobs that drive funding-nurse bumps, the v2 committed-redeem feerate, and the market-tracking refund/redeem bumps. Both methods are scoped to the active merchant.

Method	Params	Returns	Mutates
getfeepolicy	—	the policy object (below)	no
setfeepolicy	positional, all optional (below)	the full updated policy	yes (live + persisted)

The policy object is a flat shape:

```
{ "max_feerate_sat_vb": 500, "reservation_mult": 3, "committed_mult": 1 }
```

- getfeepolicy — read-only; returns the active merchant's current policy.
- setfeepolicy — **positional** params, all optional, in order [max_feerate_sat_vb?, reservation_mult?, committed_mult?]. Only the fields you supply change; the rest keep their current values. The new values are validated server-side, applied live, and persisted per-merchant (they survive a restart). Returns the full updated policy (same shape).

Field	Default	Range	What it does
<code>max_feerate_sat_vb</code>	500	1..=500	Local ceiling on any bump's feerate (sat/vB).
<code>reservation_mult</code>	3	1..=1000	Funding-nurse target multiplier over the old feerate.
<code>committed_mult</code>	1	1..=1000	v2 committed-redeem multiplier over live market (1 = commit at market; a CPFP child lifts it if the market climbs).

Every spend and bump is market-derived (`target_feerate = min(market, value-at-risk, max_feerate_sat_vb)`); there is no minimum-fee floor. The former `min_fee_sat` and per-step `step_pct` knobs were removed.

Note — The old `step_pct` escalation knob is **retired**: the unified bump strategy is market-tracking, not a fixed per-tick percentage step, so `step_pct` is no longer part of the `getfeepolicy` / `setfeepolicy` surface. (It survives in the on-disk struct only for `serde` back-compat.) These knobs are the *local* policy; `max_feerate_sat_vb` is distinct from the protocol-negotiated redeem-feerate bound. See the chapter “Fees, Fee-Bumping & Auto-Refund” for what each value does.

10.2 Seed lifecycle

Method	Params	Returns	Mutates
<code>createseed</code>	<code>passphrase?</code>	{ <code>mnemonic</code> , <code>encrypted</code> }	yes (writes seed)
<code>generateseed</code>	—	{ <code>mnemonic</code> }	no
<code>importseed</code>	<code>mnemonic</code> , <code>passphrase?</code>	{ <code>mnemonic</code> , <code>encrypted</code> , <code>identity</code> }	yes (writes seed)
<code>unlock</code>	<code>passphrase</code>	{ <code>unlocked</code> , <code>identity</code> }	yes (in-memory)

- `createseed` — generates and **persists** a new BIP39 seed. The `mnemonic` is returned exactly once, for the user to back up. `encrypted` is true iff a non-empty `passphrase` was supplied.
- `generateseed` — generates a `mnemonic` but does **not** persist it. Used by the onboarding flow to preview-then-confirm a phrase before committing it with `importseed`.

- `importseed` — installs a supplied mnemonic (optionally encrypted under passphrase). Echoes the normalized phrase plus the derived identity (npub-style pubkey). Refuses to overwrite an existing seed.
- `unlock` — verifies passphrase by trial-decrypt and holds it in memory for the process lifetime, returning the derived identity.

Warning — A persisted mnemonic is shown only once by `createseed` / `importseed`. Back it up immediately; there is no recovery path if it is lost.

10.2.1 Seed-only rescue (#54)

A machine restored from the seed alone — a wiped data directory, a fresh install — can rediscover and resume in-flight swaps it has no local record of, from encrypted-to-self snapshots on the configured Nostr relays. See the chapter “Seeds, Wallets & Merchants” for the full mechanics.

Method	Params	Returns	Mutates
<code>restorefromrelay</code>	—	{ restored, seen }	yes (adopts records)
<code>rescuestatus</code>	—	{ pending, seen, warning? }	no

- `restorefromrelay` — fetches this identity’s rescue snapshots from the configured relays and adopts every one that (a) isn’t already a local record and (b) isn’t terminal. `restored` is how many were adopted; `seen` is how many snapshot events were fetched in total. Errors if the seed is locked/unreadable or no relay transport is configured.
- `rescuestatus` — the read-only preview: `pending` is how many snapshots `restorefromrelay` *would* adopt right now, without adopting anything. `warning` is present (a fixed advisory string) whenever `pending` > 0, cautioning that driving the same swap from two live machines on one seed can double-fund it.

Note — `pactd` never adopts a snapshot on its own. Boot, unlock, and merchant-load each trigger a **read-only** rescue preview and log a warning if snapshots are pending, but adoption is always the explicit `restorefromrelay` call (or `pact-cli restore`) — call it only once the machine that ran those swaps is genuinely retired.

10.3 Merchants

A *merchant* is one seed bound to one data directory — the engine’s wallet analog. The RPC surface is merchant-scoped: all swap/board/seed calls target the **active** merchant. Nested mode (`--merchants`) lays out `merchants/<id>/`; the flat layout has a single seed in the `data-dir` root.

Method	Params	Returns	Mutates	Mode
<code>createmerchant</code>	<code>label?</code>	{ id, label }	yes	nested only

Method	Params	Returns	Mutates	Mode
listmerchants	—	{ merchants:[...], active }	no	any
loadmerchant	id	{ id, label }	yes	any
renamemerchant	id, label	{ id, label }	yes	nested only
unloadmerchant	—	{ unloaded }	yes	nested only
getmerchantinfo	id?	merchant metadata	no	any

- `createmerchant` — allocates the next free id ($m \ll N$) and makes it active.
- `listmerchants` — each entry is { `id`, `label`, `identity?`, `created`, `encrypted`, `active`, `locked` }; `active` names the currently selected id.
- `loadmerchant` — switches the active merchant in-process.
- `renamemerchant` — changes a merchant's user-facing `label`; the label is trimmed and an empty one is rejected. The label is the only mutable field (`id`, `identity`, and the seed are immutable), so it touches only the manifest with no engine reload — renaming is safe even for the active merchant mid-swap.
- `unloadmerchant` — clears the active merchant.
- `getmerchantinfo` — metadata for one merchant, defaulting to the active one.

Warning — `loadmerchant` and `unloadmerchant` **refuse** to switch away from a merchant that has a live (non-terminal) swap, so an in-flight swap is never orphaned. Drive the swap to a terminal state first.

10.4 Coins / pairs

Method	Params	Returns	Mutates
listcoins	—	{ <code>network</code> , <code>coins:[...]</code> }	no
listpairs	—	{ <code>network</code> , <code>pairs:[...]</code> }	no
validatecoin	<code>coin_id</code> , <code>chain_data</code>	{ <code>ok</code> , <code>tip_height</code> , <code>genesis_hash?</code> }	no

`listcoins` returns every coin in the shipped registry that is defined on the active network. Each entry:

Field	Meaning
<code>id</code>	Canonical coin id (e.g. <code>btcx</code> , <code>btc</code> , <code>ltc</code>).
<code>display_name</code>	Human name.

Field	Meaning
symbol	Ticker.
decimals	Smallest-unit precision.
capabilities	{ cltv, segwit_v0, taproot } booleans.
configured	True if a chain backend is wired for this coin.
nodeless	True when the coin runs nodeless (Electrum-only backend list; the wallet is the Pact seed's bdk wallet). This is the field a UI keys its send/receive/activity surface off.
wallet	The Core wallet name the coin's RPC is scoped to, parsed from the configured URL (/wallet/<name>); null when none is set (node default wallet, or a nodeless coin).
status	Live probe: "ok", "unconfigured", or "error: ...".
tip_height	Chain tip from the probe (null if unconfigured/errored).
genesis_hash	Expected genesis hash for this network.
bech32_hrp	Address HRP.
confirmations	Effective confirmation depth in force.
default_confirmations	The network/spacing default depth.

- `listpairs` — derived (never curated). Each `PairInfo` is { `coin_a`, `coin_b`, `protocols`, `selectable?`, `both_configured`, `available` }, where `protocols` lists `htlc` and/or `adaptor`.
- `validatecoin` — genesis-hash checks a *proposed* backend (`chain_data`) before Satchel saves it. Builds an ephemeral backend; the running engine config is untouched.

10.5 Wallet helpers

Method	Params	Returns	Mutates
<code>getbalance</code>	<code>chain</code>	{ <code>balance_sat</code> }	no
<code>getnewaddress</code>	<code>chain</code>	{ <code>address</code> }	yes (advances HD index)
<code>estimatesendfee</code>	<code>chain</code>	{ <code>min_sat_per_vb</code> , <code>fast</code> , <code>normal</code> , <code>slow</code> }	no
<code>sendtoaddress</code>	<code>chain</code> , <code>address</code> , <code>amount</code> , <code>conf_target?</code> , <code>fee_rate?</code>	{ <code>txid</code> }	yes (broadcasts)

Method	Params	Returns	Mutates
bumpfee	chain, txid, fee_rate	{ txid }	yes (broadcasts replacement)
listtransactions	chain	{ transactions: [...]}	no

chain is a coin id (e.g. btc). amount for sendtoaddress is a decimal string in whole coin units — or the literal string "all", which **sweeps the wallet**: the fee comes out of the swept amount (Core-RPC via subtractfeefromamount; bdk via drain_wallet), so the recipient receives balance – fee and the wallet ends empty. UTXOs reserved by a built-but-unbroadcast v2 funding are not spendable, so a sweep can never claw back a reservation. getnewaddress advances the HD derivation index (on a nodeless coin the handout is capped — see the chapter “Coins, Pairs & Capabilities”); sendtoaddress constructs and broadcasts a payment, always BIP125-replaceable (sweeps included). The fee is priced by fee_rate (explicit sat/vB — the send form’s Custom field) when given, else by a market estimate at conf_target blocks (default 6, the Normal preset; Slow/Fast are 144/1), floored to the coin’s min_feerate_sat_vb with the usual 1 sat/vB fallback; both fee params apply to a sweep as usual.

estimatesendfee backs the send form’s fee presets: raw estimator answers in sat/vB at the 1/6/144-block targets — null where the estimator has no data (fresh chain, quiet mempool, regtest), which the form maps to disabled presets and a custom-rate fallback at the coin floor (min_sat_per_vb), mirroring phoenix’s send dialog.

bumpfee RBF-replaces an unconfirmed wallet send at the given fee_rate (sat/vB — must beat what the tx pays now plus the incremental-relay margin the wallet enforces). Satchel offers it on pending sent rows of the Activity dialog for nodeless coins; a node-backed wallet is bumped with the node’s own tooling instead. A txid that **funds a live swap** is refused — v1 HTLC and v2 funding txids alike — with <txid> funds live swap <id> – the swap engine manages its fee (see get/setfeepolicy), bumpfee must not replace it. Those fees belong to the funding nurse: v1 fundings are re-RBF’d under the swap’s own fee policy, and a v2 funding is deliberately CPFP’d, because replacing it would change its txid and invalidate the pre-signed MuSig2 redeems (see the chapter “Fees, Fee-Bumping & Auto-Refund”).

listtransactions serves the activity feed of an **Electrum-connected (nodeless) coin** — each entry carries txid, direction ("sent" / "received"), the net amount_sat (fee excluded on sends), fee_sat (absent when the wallet doesn’t own every input), vsize (with fee_sat this yields the feerate a bump has to beat), confirmations, and timestamp (block time; first-seen for mempool entries; absent for a built-but-unreleased v2 funding). A self-transfer nets to "sent" with amount_sat 0. Newest first. Node-backed coins refuse (wallet activity requires a nodeless (Electrum-backed) coin) — the node wallet keeps its own history.

Chapter 11

API: v1 HTLC Swaps

This chapter documents the **v1 (HTLC)** swap RPCs. These drive a classic hashlock/timelock atomic swap between two coins. The cross-chain mechanics — why two timelocks, what each transaction does — are covered in the protocol chapters; here we document the RPC surface only. For shared conventions see the chapter “JSON-RPC Conventions”. For v2 (Taproot/MuSig2 adaptor) swaps see the chapter “API: v2 Adaptor Swaps”.

Note — *v1 (HTLC) and v2 (Taproot/MuSig2 adaptor) swaps are reviewed and live on mainnet.*

11.1 Swap state machine

A v1 swap is a `SwapRecord` whose state advances through this enum (snake_case in JSON):

created → accepted → funded_a → funded_b → redeemed_b → completed

State	Meaning
created	Offer created by the initiator; no on-chain action yet.
accepted	Counterparty accepted; both sides committed to terms.
funded_a	The first HTLC leg is funded on-chain.
funded_b	The second HTLC leg is funded on-chain.
redeemed_b	The B-side HTLC has been redeemed (secret revealed).
completed	Both legs settled; swap finished successfully.

Terminal failure states:

State	Meaning
refunded	A funded leg timed out and was refunded.
aborted	Swap cancelled before our HTLC funded.

11.2 The coin:amount convention

give and get are strings of the form coin:amount, where coin is a registry coin id and amount is a decimal in whole coin units — for example "btcx:1.0" and "btc:0.5". t1/t2 are the two relative timelocks in blocks (initiator and counterparty legs respectively).

11.3 Read methods

Method	Params	Returns	Mutates
listswaps	—	[SwapRecord]	no
getswap	swap_id	one SwapRecord	no
listpendingtakes	—	outstanding takes awaiting maker init	no
listmyoffers	—	[{ offer_id, offer, state, created, valid_for, current_expiry, final_expiry, now }]	no

- listswaps — every v1 swap record for the active merchant.
- getswap — a single record by swap_id.
- listpendingtakes — takes that have arrived but for which the maker has not yet initiated a swap (no SwapRecord exists yet — the UI's "initiating" pre-swaps).
- listmyoffers — the maker's own offers (the My-offers view). current_expiry is the rolling expiry (last refresh + relay TTL, capped at final_expiry); final_expiry is the maker-set hard expiry (created + valid_for); now is the server timestamp for client-side countdown rendering.

11.4 Lifecycle methods

Method	Params	Returns	Mutates	Purpose
offer	give, get, t1, t2	{ record, envelope }	yes	Start a swap as initiator.
acceptoffer	envelope	{ record, envelope }	yes	Accept a received offer envelope.
recv	envelope	{ record }	yes	Ingest a counterparty reply envelope.

Method	Params	Returns	Mutates	Purpose
fund	swap_id	{ record, envelope }	yes (broadcasts)	Broadcast our HTLC funding tx.
redeem	swap_id	{ record }	yes (broadcasts)	Redeem the counterparty HTLC (reveals secret).
refund	swap_id	{ record }	yes (broadcasts)	Reclaim our funded HTLC after timeout.
abort	swap_id, reason?	{ record } or { yes can-celled_pending_take }		Cancel an unfunded swap, a v2 adaptor swap, or an unanswered pending take.
tick	—	{ events:[...] }	yes	Advance the scheduler one pass.

- offer — initiates a swap with the given terms and returns the signed envelope to hand to the counterparty.
- acceptoffer — accepts a counterparty's offer envelope, returning a reply envelope.
- recv — ingests a counterparty envelope (e.g. an acceptance reply).
- fund — builds and broadcasts our HTLC funding transaction, then **relays the funded envelope to the counterparty** (via the engine's notify path), so a manually or hand-recovered swap notifies the maker exactly like the automatic auto-fund path does. (Previously the RPC just returned the envelope without relaying it.)
- redeem — spends the counterparty's funded HTLC, revealing the preimage.
- refund — reclaims our own funded HTLC once its timelock has expired.
- abort — cancels the given swap_id; reason defaults to "user aborted". **One method covers every in-flight card the UI shows:** it looks up swap_id as a v1 record first, then a v2 (Taproot) adaptor record, and — if neither exists — treats it as an *offer id* for a still-unanswered pending take (listpendingtakes) and cancels that instead, returning { cancelled_pending_take: <offer_id> } rather than a record. A cancelled pending take best-effort notifies the maker with an abort envelope keyed by the offer id, so the maker's served-offer marker resolves it even though no SwapRecord ever existed on that side.
- tick — runs one scheduler pass (board sync + engine tick) and returns the resulting events, each { swap_id, action, detail }.

Warning — abort is **refused once our HTLC has funded** (v1) or once our leg is funded (v2). After funding, the only safe exits are redeem (if you can claim the counterparty's leg) or refund (after your timelock expires). Aborting a funded swap is not

an option because the coins are already committed on-chain.

Note — A v2 handshake that stalls before either leg is funded — stuck at created, accepted, or nonces_exchanged — also **self-aborts after 15 minutes** (PRE_FUNDING_TIMEOUT_SECS) without any abort call or relay message: both sides' schedulers independently time out their own copy of the stalled handshake. signed is excluded from this auto-timeout, since funding may already be in flight by then. See the chapter "API: v2 Adaptor Swaps".

Note — The funded envelope fund relays is an *accelerator*, not a requirement. Even if the relay message never reaches the maker, the swap still completes via chain-watching: both sides detect each other's legs on-chain and drive to redemption regardless. The relayed funded / redeemed messages only shave latency off that chain-watched path.

11.5 Diagnostics

Method	Params	Returns	Mutates
dumpswap	swap_id	{ swap_id, pactd_version, record, log }	no

- dumpswap — returns a developer-shareable diagnostics bundle for one swap: its current record (the v1 SwapRecord, or the v2 AdaptorSwapRecord if the id is a v2 swap) plus log, the array of pactd log lines that mention that swap_id (the scheduler tags every event with swap=<id>). pactd_version is the engine's crate version. The dispatch tries the v1 store first, then the v2 adaptor store, and — if swap_id matches neither — falls back to a pending take by offer id, so an "initiating" pre-swap that never became a SwapRecord can still be dumped for support.

Note — dumpswap is **secret-safe by construction**. The record is passed through scrub_secrets, which redacts the v1 preimage and any secret-named field; the v2 adaptor record stores no secret (t is never persisted); and seeds, passphrases, and MuSig2 nonces never appear in a record or in the log. The bundle is safe to paste into a bug report. It backs Satchel's per-swap **Dump logs** button.

Chapter 12

API: v2 Adaptor Swaps

This chapter documents the **v2 (Taproot/MuSig2 adaptor)** swap RPCs. A v2 swap settles on the cooperative key-path (indistinguishable from an ordinary Taproot spend) and uses a MuSig2 two-of-two with an adaptor signature instead of an on-chain hashlock. The cryptographic rationale — why an adaptor, how the MuSig2 nonce exchange binds the secret — is covered in the protocol chapter “v2 Taproot/MuSig2 Adaptor Swaps”. For shared RPC conventions see the chapter “JSON-RPC Conventions”; for the simpler HTLC route see “API: v1 HTLC Swaps”.

Note — v2 adaptor swaps are **live on every network, including mainnet**. *v1 (HTLC) and v2 (Taproot/MuSig2 adaptor) swaps are reviewed and live on mainnet.*

12.1 Swap state machine

A v2 swap is an `AdaptorSwapRecord` whose state advances through this enum (snake_case in JSON):

created → accepted → nonces_exchanged → signed
→ funded_a → funded_b → redeemed_b → completed

State	Meaning
created	Initiator created the adaptor swap; terms set.
accepted	Counterparty accepted the terms.
nonces_exchanged	MuSig2 nonces exchanged for both legs.
signed	Adaptor + partial signatures produced.
funded_a	The first leg’s funding tx is confirmed.
funded_b	The second leg’s funding tx is confirmed.
redeemed_b	The B-side has been redeemed (adaptor secret revealed).
completed	Both legs settled successfully.

Terminal failure states: refunded, aborted.

12.2 Handshake order

The methods below are **not** interchangeable — they must be driven in this order to complete the MuSig2 / adaptor handshake:

```
adaptorinit → adaptoraccept → adaptorrecv → adaptorfundingready  
            → adaptornonces → adaptorsign → adaptorassemble  
            → adaptorfund → adaptorredeem | adaptorrefund
```

init/accept/recv establish terms; fundingready registers the funding output; nonces/sign/assemble complete the MuSig2 adaptor exchange; fund broadcasts; and the swap settles with redeem (cooperative path) or refund (timeout path). See “v2 Taproot/MuSig2 Adaptor Swaps” for why each step is required.

Note — The cooperative key-path **redeem is NOT RBF-bumpable**; it commits at the live market rate, is broadcast non-replaceable, and is dragged through by a CPFP child. The single-key **refund IS bumpable**. Budget fees accordingly when settling near a deadline.

12.3 Cancelling a v2 handshake

There is no separate adaptorabort RPC — v2 shares the abort method documented in the chapter “API: v1 HTLC Swaps”. Dispatch tries the v1 store first, then falls back to the v2 adaptor store, so calling abort with a v2 swap_id cancels the adaptor record instead (internally Engine::adaptor_abort). The same funding gate applies: **abort is refused once our own leg is funded** (funding_a_txid for the initiator, funding_b_txid for the participant) — past that point the timelocked refund is the only exit. An explicit abort is also the one case that sends a best-effort abort envelope to the counterparty; persisted MuSig2 nonce sessions are deliberately **kept**, since the store’s overwrite-refusal on an existing session is what guarantees an aborted swap can never be signed again.

Note — A handshake stalled in created, accepted, or nonces_exchanged — i.e. before either leg has funded — also **times out on its own after 15 minutes** (PRE_FUNDING_TIMEOUT_SECS, engine.rs), with no abort call and no relay message: each side’s scheduler independently retires its own stalled copy of the handshake. signed is deliberately excluded from this auto-timeout, because by then a counterparty’s funding may already be in flight. This closes a gap where a v2 handshake that stalled before signed was previously inert to the ticker — neither cancellable (abort only read the v1 table) nor self-expiring.

12.4 Read method

Method	Params	Returns	Mutates
listadaptorswaps	—	[AdaptorSwapRecord]	no

12.5 Handshake & lifecycle methods

give/get use the same coin:amount string convention as v1 (e.g. "btcx:1.0"); t1/t2 are the two relative timelocks in blocks.

Method	Params	Returns	Mutates
adaptorinit	give, get, t1, t2	{ record, envelope }	yes
adaptoraccept	envelope	{ record, envelope }	yes
adaptorrecv	envelope	{ record }	yes
adaptorfundingready	swap_id, txid, vout	{ envelope }	yes
adaptornonces	swap_id	{ envelope }	yes
adaptorsign	swap_id	{ envelope }	yes
adaptorassemble	swap_id	{ record }	yes
adaptorfund	swap_id	{ envelope }	yes (broadcasts)
adaptorredeem	swap_id	{ record }	yes (broadcasts)
adaptorrefund	swap_id	{ record }	yes (broadcasts)

- `adaptorinit` — initiate a v2 swap with the given terms; returns the offer envelope.
- `adaptoraccept` — counterparty accepts; returns a reply envelope.
- `adaptorrecv` — ingest a counterparty envelope into the local record.
- `adaptorfundingready` — register the funding output (txid, vout) once the funding UTXO exists; returns an envelope for the counterparty.
- `adaptornonces` — produce and exchange the MuSig2 nonces.
- `adaptorsign` — produce the adaptor and partial signatures.
- `adaptorassemble` — assemble the partials into the final signature set.
- `adaptorfund` — broadcast our funding transaction. v2 fundings go out **non-replaceable** (no BIP125 signal): the funding txid is committed into the pre-signed redeems, so a fee bump is CFPF-via-change, never RBF (see the chapter "Fees, Fee-Bumping & Auto-Refund").
- `adaptorredeem` — settle on the cooperative key-path, revealing the adaptor secret.
- `adaptorrefund` — reclaim our funded leg via the single-key timeout path.

Chapter 13

API: Board, Private Offers & Fees

This chapter documents the board (Corkboard / Nostr) RPCs, the private ("trade-with-a-friend") offer RPCs, and the fee-estimation method. For shared conventions see the chapter "JSON-RPC Conventions"; for the swap lifecycle those offers feed into, see "API: v1 HTLC Swaps" and "API: v2 Adaptor Swaps".

13.1 Board (Corkboard / Nostr)

An offer posted to the board fans out to every configured transport (HTTP Corkboard URLs and/or Nostr relays). Reads are per-board: a board selector chooses one transport rather than merging.

Method	Params	Returns	Mutates
boardlistoffers	board?	{ offers }	no
boardstatus	—	{ relays:[{ url, connected }] }	no
boardpostoffer	give, get, t1_secs, t2_secs, protocol?, ttl_secs?	{ offer_id }	yes
boardtake	offer_id	{ taken }	yes
boardrevoke	offer_id	{ revoked }	yes

- `boardlistoffers` — lists offers from one board. The optional board is an HTTP Corkboard URL **or** the literal "nostr"; omitted, it defaults to the first configured board.
- `boardstatus` — relay connectivity for the header indicator: one { url, connected } entry per configured Nostr relay. Empty when the Nostr transport is not configured.
- `boardpostoffer` — posts a listing. give/get are coin:amount strings; t1_secs/t2_secs are the two timelocks **in seconds**. protocol (param 4) optionally forces "pact-htlc-v1" or "pact-htlc-v2"; omitted, the engine picks the default for the pair. ttl_secs (param 5) sets the listing validity; omitted, the engine default applies.

- boardtake — takes a posted offer by offer_id.
- boardrevoke — revokes one of your own posted offers.

Note — Cumulative funds gate. boardpostoffer rejects an offer the core wallet could not fund — and “fund” means the **sum** of the give-leg amounts across **all** your still-live offers in that coin, not just this one offer, plus per-offer funding-fee headroom (Engine::ensure_can_fund_new_offer → committed_give_for_coin). Because each offer is funded only when *taken*, advertising several offers whose give-legs together exceed your balance would let two of them be taken and only one funded; the gate stops that at post time. It is a best-effort pre-flight — in-flight takes are not subtracted and balances are not netted across coins. On failure the maker sees:

```
insufficient {coin_id} balance to advertise this offer: have {balance} sat,
need ~{needed} sat ({amount} for this offer + {committed} already committed
across {n_live} live offer(s) + ~funding-fee headroom). Withdraw or let some
offers expire first.
```

Note — On Nostr the listing’s expiry is **rolling**: min(now + 1800s, created + ttl_secs). It is refreshed as the offer is republished, not pinned to ttl_secs from creation.

Note — Wire epochs (rc10): the signed offer body and the take envelope both carry wire — the protocol family’s wire-compatibility epoch (v1 = 1, v2 = 2; **absent parses as 1**, the pre-rc10 era). The taker refuses an offer whose wire differs from what its build speaks (boardtake errors up-front), and the maker refuses a mismatched take with a take-rejected reason (“incompatible release”) while the offer stays live. Handshake bodies (init/accept, both protocols) carry and validate the same field. Bump the epoch for any wire-breaking protocol amendment — mixed-epoch peers then gate cleanly instead of failing mid-handshake.

Warning — Breaking wire change: the take envelope now carries a signed taken_at timestamp, and this field is **REQUIRED** — a take from a pre-rc8 build has none and is treated as stale (age saturates to “very old”), so it is silently dropped rather than served. Both parties must run rc8+ for boardtake to work. This closes a mainnet incident where a take that had sat queued for hours (e.g. the maker’s node was unreachable) was served anyway after the taker’s own pending-take entry had already self-pruned — burning the offer via revoke-on-commit and stranding an uncancelable record on a counterparty nobody was driving. The maker now compares taken_at against its own PRE_FUNDING_TIMEOUT_SECS (15 minutes) **before** serving or revoking the offer: a take older than that is dropped with no reply (the taker’s own card pruned itself long ago, so nothing is listening), and the offer stays live for the next taker. A future taken_at (clock skew) saturates to age zero rather than being rejected, so ordinary clock drift within the window is tolerated.

13.2 Private (off-market) offers

A private offer is built and signed locally but **never posted to a board**. It travels to a counterparty as a *slip* string (paste it into your own chat). The methods mirror the board ones, but no board is touched.

Method	Params	Returns	Mutates
makeprivateoffer	give, get, t1_secs, t2_secs, protocol?, ttl_secs?	{ slip }	yes
takeoffer	slip	{ taken }	yes
listprivateoffers	—	{ offers }	no
cancelprivateoffer	offer_id	{ cancelled }	yes

- `makeprivateoffer` — builds and signs an offer, returning the `slip` string to hand to a friend. As with `boardpostoffer`, `protocol` is **param 4** and `ttl_secs` is **param 5** (both optional).
- `takeoffer` — takes a private offer from a pasted `slip`: decodes and verifies the signed offer, then relays a take to the maker.
- `listprivateoffers` — your outstanding private offers.
- `cancelprivateoffer` — cancels one by `offer_id`.

13.3 Fees

Method	Params	Returns	Mutates
estimateswapfees	give_coin, get_coin	fee preview (see below)	no

`estimateswapfees` previews the on-chain fees for a prospective swap. It takes **only** `give_coin` and `get_coin` (coin ids).

Warning — Older docs list `protocol?` and `role?` params for `estimateswapfees`. Those are **not parsed** — passing them has no effect. Legs are keyed solely off `give/get` (you fund what you give, redeem what you get).

The result shape:

```
{
  "platform_fee_sat": 0,
  "give": {
    "coin_id": "btcx",
    "fee_rate_sat_per_vb": 5,
    "fee_rate_is_fallback": false,
```



```

    "legs": [
      { "name": "fund",    "vbytes": 0, "fee_sat": 0 },
      { "name": "refund", "vbytes": 0, "fee_sat": 0 }
    ]
  },
  "get": {
    "coin_id": "btc",
    "fee_rate_sat_per_vb": 5,
    "fee_rate_is_fallback": false,
    "legs": [
      { "name": "redeem", "vbytes": 0, "fee_sat": 0 }
    ]
  }
}

```

- platform_fee_sat is always 0 — the board takes nothing.
- The give side covers the fund and refund legs; the get side covers the redeem leg. Each leg reports { name, vbytes, fee_sat }.
- fee_rate_is_fallback is true when a live fee estimate was unavailable and a built-in fallback rate was used.

Part IV

The Swap Protocol

Chapter 14

The Swap Lifecycle

Pact settles a trade as an *atomic swap*: two on-chain transactions, one per chain, bound so that either both parties get paid or both get their money back. Nobody can run off with the funds. The chains — not a custodian, not a relay, not Pact itself — enforce the deal. This chapter introduces the two state machines that drive a swap to completion (v1 HTLC and v2 adaptor), the roles each party plays, and the scheduler that walks every live swap forward on a clock.

Two protocol versions ship today, both running on every network:

- **v1 (pact-htlc-v1)** — classic hash-timelock contracts (HTLCs) over P2WSH outputs. See the chapter “v1 HTLC Swaps”.
- **v2 (pact-htlc-v2)** — Taproot outputs with a MuSig2 adaptor-signature binding. See the chapter “v2 Taproot/MuSig2 Adaptor Swaps”.

The lifecycle, roles, and scheduler model described here are shared by both; the difference is what the on-chain commitment *looks like* and how the secret that unlocks it is revealed.

14.1 The two roles

Every swap has exactly two parties, and the roles are fixed for the duration:

Role	Code name	Who	Locks	Refund deadline	Holds the secret
<i>Initiator</i>	Initiator (Alice)	the maker — posts the offer	chain A	T1 (later)	yes
<i>Participant</i>	Participant (Bob)	the taker — accepts the offer	chain B	T2 < T1 (earlier)	no

Role is defined in `swap.rs:50-57` (v1) and the same Alice-funds-A / Bob-funds-B convention is documented in `adaptor_swap.rs:5-8` (v2).

The asymmetry is the whole point of the design:

- **The maker funds first.** Alice locks her coin on chain A before Bob locks his on chain B. The party who reveals the secret (Alice) is the one who must commit first, so she cannot strand Bob's funds without exposing her own.
- **Alice holds the secret.** In v1 it is a SHA256 preimage s ; in v2 it is an adaptor scalar t . Either way, only the initiator can derive it from her seed, and revealing it on chain B is what lets Bob claim chain A.
- **$T_2 < T_1$.** Bob's refund unlocks *before* Alice's. Bob must always have a safe window to refund leg B after Alice's window to redeem it closes, before his own funds are at risk on the other side. This single inequality is the structural backbone of every Pact timelock; see the chapter "Timelocks & Action Deadlines".

Note — "Maker funds first" is a protocol invariant, not a UI choice. The chain enforces the deal once both legs are funded; until then, the only party with capital at risk is the one who controls when the secret is revealed.

14.2 The scheduler-driven model

A Pact swap is not a request/response RPC dance. Once a swap is Accepted, the daemon owns it: `pacstd` runs a background scheduler that re-examines every live swap on a fixed interval and takes whatever action the current state and the chain clock allow. The operator does not poll, click "redeem", or watch the mempool — the engine does.

The loop is `Engine::tick`, which fans out to one handler per swap (`engine.rs:2581`). For v1 each swap is advanced by `tick_one` (`engine.rs:2654`); for v2 by `adaptor_tick_one` (`engine.rs:2614-2620`). The interval is set by the `--tick-secs` flag.

On each tick, for each swap, the engine:

1. Reads the current state and the party's role.
2. Checks the chains it watches: have the funding outputs confirmed? has the counterparty redeemed and thereby revealed the secret? has a refund timelock matured (measured against the chain's median-time-past)?
3. Arms exactly the action the state machine permits — fund, redeem, refund, fee-bump, or nothing — and persists the new state.

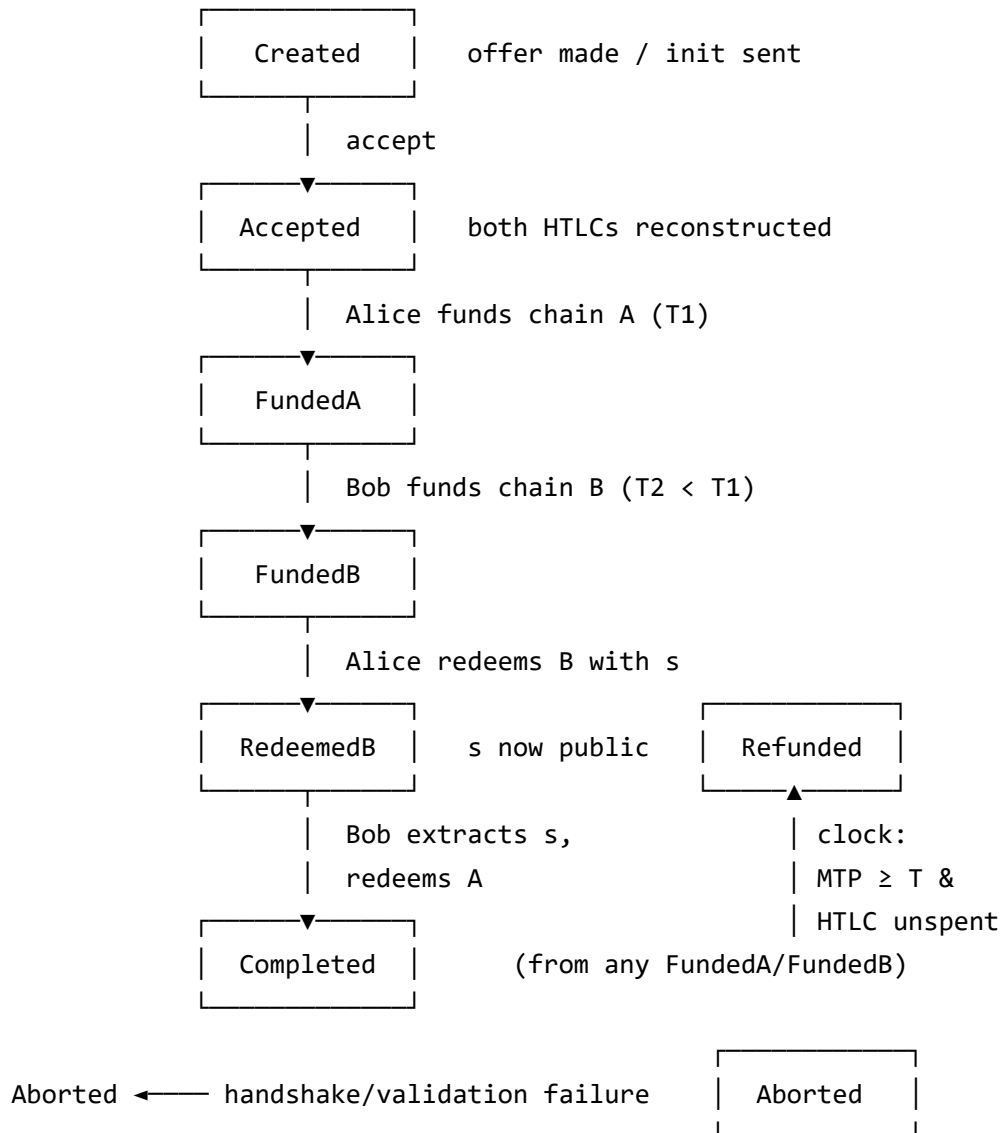
This is what makes **auto-redeem and auto-refund clock-driven**. Alice's daemon redeems leg B the moment it is safe; Bob's daemon extracts the revealed secret and redeems leg A; and either party's daemon fires the timelock refund the moment its deadline passes and the output is still unspent — all without human intervention, and all while the operator is offline-tolerant within the timelock margins (see the chapter "Timelocks & Action Deadlines"). The per-role, per-state arming lives in `engine.rs:2666-2774` for v1.

Warning — Because the engine acts on a clock, a daemon that is *down* across a critical deadline cannot fire its refund. The action-deadline margins (fund 3h / reveal 2h / redeem-A 1h on mainnet) exist precisely to give a recovering daemon slack. Keep

pactd running for the life of any swap you have funded. See the chapter “Timelocks & Action Deadlines”.

14.3 v1 state machine — swap::State

The v1 lifecycle is swap::State (swap.rs:61-72). The happy path is a straight line; the two refund/abort states branch off from any funded state, driven by the clock rather than by a message.

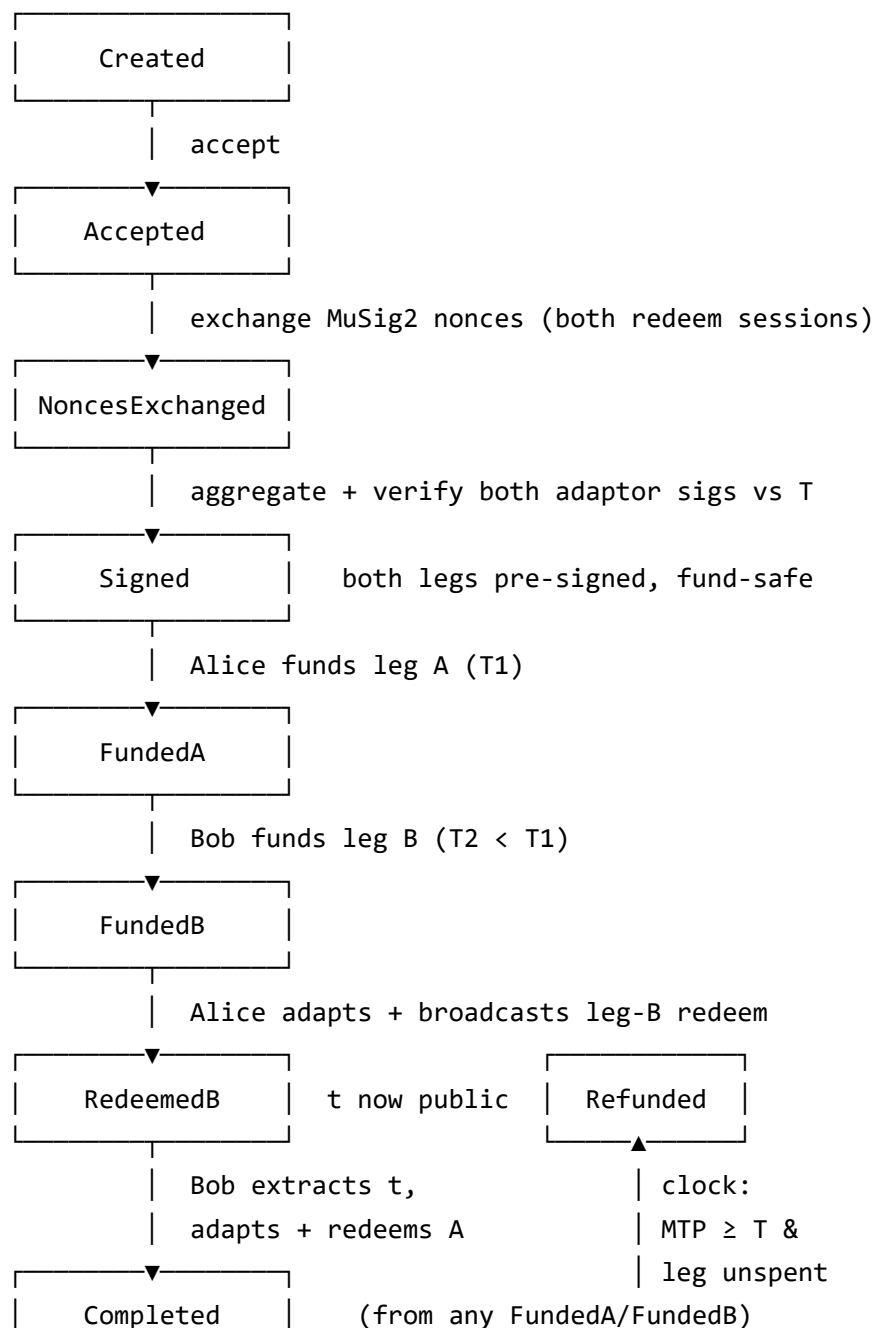


- Created → Accepted: the offer is taken; both parties now hold enough parameters (SwapParams, swap.rs:74-91) to reconstruct **both** HTLCs byte-for-byte and verify the counterparty’s script locally.
- Accepted → FundedA: Alice broadcasts the chain-A funding transaction.
- FundedA → FundedB: Bob, having verified leg A on chain, funds leg B.
- FundedB → RedeemedB: Alice redeems leg B by revealing the preimage s in the spending witness. This is the irreversible step.

- RedeemedB → Completed: Bob's daemon scans the chain-B spend, extracts s , and redeems leg A.
- Refunded is reachable from FundedA or FundedB when a refund timelock matures and the corresponding HTLC is still unspent. Aborted covers pre-funding handshake or validation failures.

14.4 v2 state machine — AdaptorState

The v2 lifecycle is AdaptorState (adaptor_swap.rs:34-48). It is the same shape as v1 with **two extra pre-funding states** for the MuSig2 adaptor ceremony that must complete *before* either party puts coins on chain.



Aborted ← handshake/validation failure

Aborted

The two added states are the heart of v2's safety:

- Accepted → NoncesExchanged: both parties exchange fresh MuSig2 secret nonces for the two redeem sessions (one per leg). Nonces are generated from a CSPRNG, never the seed, and persisted write-ahead — reuse is structurally impossible. See the chapter "v2 Taproot/MuSig2 Adaptor Swaps".
- NoncesExchanged → Signed: both adaptor signatures are aggregated and **verified against the adaptor point T** before anyone funds. Reaching Signed is the guarantee that the cooperative redeems will work and that Alice's broadcast will reveal t to Bob.

Note — In v2 the legs are pre-signed (state Signed) before any funding transaction is broadcast. By the time a party commits coins, the redeem messages already exist and have been checked against T. From FundedB onward, v2 behaves like v1: Alice's redeem reveals the secret on chain, Bob's daemon extracts it and claims his leg.

14.5 Where the secret lives

Both versions hinge on a single secret that only the initiator can produce, and both derive it from the seed so that losing the state database never loses the secret:

	v1	v2
Secret	preimage s (32 bytes)	adaptor scalar t
Public commitment	$H = \text{SHA256}(s)$, on both legs	point $T = t \cdot G$, binds both redeem sigs
Revealed by	the redeem witness on leg B	the adapted leg-B redeem signature
Extracted by Bob	SHA256-scan of the leg-B witness	subtracting his adaptor sig from the broadcast sig

The mechanics of derivation, the exact witness layouts, and the extraction procedure are covered in the v1 and v2 chapters that follow. The lifecycle to hold in mind is the same in both: maker funds first, the chain enforces the deal, and the scheduler redeems or refunds on a clock.

Chapter 15

v1 HTLC Swaps

The v1 protocol (`pact-htlc-v1`) settles a trade with two *hash-timelock contracts* (HTLCs), one per chain. A single SHA256 preimage *s* unlocks both: the initiator reveals *s* to claim her leg, which publishes *s* on chain so the participant can claim his. Each leg also has a timeout branch that refunds the funder after an absolute Unix-time deadline. This chapter walks the protocol end to end — key derivation, the witness script, funding, redeem, refund, and preimage extraction — reproducing the exact byte-level constructions from `libswap`.

Roles are fixed (see the chapter “The Swap Lifecycle”): *Alice* is the Initiator, holds the preimage, locks chain A, and refunds at T_1 ; *Bob* is the Participant, locks chain B, and refunds at $T_2 < T_1$. The maker funds first.

15.1 Key derivation

All key material derives from one BIP39 seed via BIP32, under the Pact *purpose* 7228' (7228 is “PACT” on a phone keypad, keys `.rs:20-21`). Every level is hardened.

Material	Path	Type & use
<i>Identity key</i>	<code>m/7228'/0'/0'</code>	BIP340 x-only; signs handshake envelopes. Never appears in an HTLC.
<i>Swap key, initiator (Alice)</i>	<code>m/7228'/1'/coin(c)'/i'</code>	compressed secp256k1, ECDSA; the redeem/refund key, indexed by her local counter <i>i</i> .

Material	Path	Type & use
<i>Swap key, participant (Bob), anchored</i>	<code>m/7228'/1'/coin(c)'/a'/b'/c</code>	Same key type; a, b, c, d are the first four masked-31-bit words of <code>TaggedHash("pact/swap-key-anchor/v1", H)</code> (spec §4.2) — no counter, re-derivable from the seed plus H alone (which sits in both on-chain HTLC scripts).
<i>Preimage source</i>	<code>m/7228'/2'/i'</code>	feeds the preimage tagged hash (initiator only).

One swap key per chain per swap either way: on the chain where a party locks funds it is their refund key, on the chain where they claim it is their redeem key.

Note — The two roles index their keys differently because they learn the swap's identity at different times. Alice's counter *i* is the *root* of the swap's identity — H (and through it `swap_id`) derives from the preimage at index *i*, so it cannot itself be derived from the swap. Bob learns the public anchor H from the `init` message before deriving any key, so his keys need no counter — and two machines sharing one seed can never issue Bob the same key for two different swaps, since the anchor is swap-specific. This is also what makes Bob's participant-side rescue possible after a data-loss restore (see "Seeds, Wallets & Merchants"): his keys re-derive from the seed plus the on-chain script alone, with no local state needed.

The `coin(c)` index identifies the **asset**, not the network (`keys.rs:23-25`):

Asset	<code>coin(c)</code>
BTC	<code>0</code>
PoCX (BTCX)	<code>0x504F4358</code> (ASCII "POCX")

The preimage and the swap identifier are deterministic, so both survive loss of the state database (`keys.rs:83-89`, `keys.rs:137-142`):

```

s      = TaggedHash("pact/htlc/preimage/v1", privkey at m/7228'/2'/i')  (32 bytes)
H      = SHA256(s)                                                         (the hashlock)
swap_id = hex( TaggedHash("pact/swapid/v1", H)[0..8] )                    (16 hex chars)

```

Only Alice can derive *s* (she owns the seed and the index *i*); Bob only ever learns H until Alice reveals *s* on chain.

Note — The swap key is ECDSA in v1. The *same* derivation path is reused in v2 as a BIP340 x-only MuSig2 signer — same private key, different public encoding. See

the chapter “v2 Taproot/MuSig2 Adaptor Swaps”.

15.2 The HTLC witness script

Each leg is a P2WSH output committing to this witness script (`htlc.rs:62-87`). The script is identical on both chains; only the keys and the locktime `T` differ per leg.

```
OP_IF
  OP_SIZE 32 OP_EQUALVERIFY
  OP_SHA256 <H> OP_EQUALVERIFY
  OP_DUP OP_HASH160 <hash160(redeem_pubkey)>
OP_ELSE
  <T> OP_CHECKLOCKTIMEVERIFY OP_DROP
  OP_DUP OP_HASH160 <hash160(refund_pubkey)>
OP_ENDIF
OP_EQUALVERIFY
OP_CHECKSIG
```

- The **OP_IF (hash) branch** is the redeem path: present a 32-byte item whose SHA256 equals `H`, plus a signature from `redeem_pubkey`. The `OP_SIZE 32 OP_EQUALVERIFY` guard forbids the empty-push trick that would otherwise let `OP_SHA256` run on a zero-length item.
- The **OP_ELSE (timeout) branch** is the refund path: after the absolute locktime `T` matures (enforced by `OP_CHECKLOCKTIMEVERIFY` against BIP113 median-time-past), a signature from `refund_pubkey` spends it.

The scriptPubKey is the standard P2WSH wrapper (`htlc.rs:89-92`):

```
OP_0 <SHA256(witness_script)>
```

15.2.1 Per-leg keys and locktimes

The two legs assign the redeem/refund roles in mirror image (`swap.rs:106-124`):

Leg	redeem_pubkey	refund_pubkey	T
Chain A (Alice funds)	<code>bob_redeem_a</code>	<code>alice_refund_a</code>	<code>t1</code>
Chain B (Bob funds)	<code>alice_redeem_b</code>	<code>bob_refund_b</code>	<code>t2</code>

On leg A, Bob redeems (with `s`) and Alice refunds at `T1`. On leg B, Alice redeems (with `s`) and Bob refunds at `T2`. Both legs share the same hashlock `H`.

Warning — The locktime `T` MUST be a **Unix timestamp** ≥ 500000000 ; block-height locktimes are rejected at construction (`htlc.rs:28-29`, `htlc.rs:48-52`). HTLCs are time-locked, not height-locked, so a deadline means the same wall-clock instant on both chains regardless of their block cadence.

Warning — A party **MUST** reconstruct the counterparty’s witness script locally from the agreed SwapParams and **byte-compare** it before funding. Never trust an address or script handed to you over the wire — derive it.

15.3 Funding

Funding is an ordinary core-wallet send to the leg’s exact-amount P2WSH address (spec §6.1). The funder’s node performs coin selection, change, and fee estimation as for any payment; Pact only supplies the destination and value. The funder then sends a funded(txid, vout) message, and the receiver **verifies the output on chain** (correct script, correct amount, sufficient confirmations) rather than trusting the message. The engine can also discover funding by watching the chain directly if the message is withheld.

The funding vsize used for fee previews is FUND_TX_VSIZE = 160 (swap.rs:34) — a 1-input / 2-output segwit send is a sensible mid-point; a real wallet selecting more inputs may differ.

15.4 Redeem

The redeem transaction spends the HTLC via the hash branch (swap.rs:178-205). It is a version-2, 1-in / 1-out transaction with nLockTime = 0 and an RBF-signalling input sequence. The witness stack is:

```
[ sig||0x01, redeem_pubkey (33), s (32), 0x01, witness_script ]
```

Item	Bytes	Purpose
sig 0x01	DER + SIGHASH_ALL byte	BIP143 signature over the spend
redeem_pubkey	33	the compressed redeem key, hashed in the script
s	32	the preimage; SHA256 must equal H
0x01	1	the OP_IF selector — a non-empty item takes the hash branch
witness_script	var	the full script, revealed for P2WSH

Key parameters (swap.rs:127-205):

- nSequence = 0xFFFFFFFF (HTLC_SPEND_SEQUENCE, swap.rs:17) — signals RBF so the redeem fee can be bumped (see the chapter “Fees, Fee-Bumping & Auto-Refund”).
- nLockTime = 0 — the hash branch has no timelock.
- Signature is BIP143 SIGHASH_ALL (swap.rs:158-169).
- Worst-case vsize for fee math: REDEEM_TX_VSIZE = 155 (swap.rs:25).

The spend sweeps `htlc_value - fee` to the claimer’s destination; the engine refuses to build a spend whose output would fall below the dust limit (`DUST_LIMIT_SAT = 546`, `swap.rs:19-20`, guarded at `swap.rs:138-141`).

15.5 Refund

The refund transaction spends the HTLC via the timeout branch (`swap.rs:210-235`). The witness stack uses an **empty** item to select the `OP_ELSE` branch:

```
[ sig||0x01, refund_pubkey (33), <empty>, witness_script ]
```

Item	Bytes	Purpose
<code>sig 0x01</code>	DER + SIGHASH_ALL byte	signature from the refund key
<code>refund_pubkey</code>	33	the compressed refund key
<code><empty></code>	0	the <code>OP_ELSE</code> selector — an empty item is false, taking the timeout branch
<code>witness_script</code>	var	the full script

Parameters:

- `nLockTime = T` — set to the leg’s absolute locktime, so the transaction is only valid once the chain’s median-time-past reaches `T`. Broadcasting earlier is rejected by the node (not fatal; the engine retries).
- `nSequence = 0xFFFFFFFF` — RBF-signalling, so the refund is also bumpable.
- Worst-case `vsize`: `REFUND_TX_VSIZE = 146` (`swap.rs:26`).

Note — The v1 refund is **signed and persisted at funding time** (`rec.refund_tx_hex`). The instant a party funds a leg, the fully-signed refund transaction already exists on disk, ready to broadcast the moment its timelock matures — even if the daemon has been restarted since. v2 differs: it re-derives the refund from the seed on each call. See the chapter “Fees, Fee-Bumping & Auto-Refund”.

15.6 Preimage extraction

When Alice redeems leg B, her witness publishes `s` on chain B. Bob’s daemon recovers it by scanning the spend’s witness for a 32-byte item whose SHA256 equals the known `H(htlc.rs:104-114)`:

```
for item in witness_items {
    if item.len() == 32 && SHA256(item) == hash_h {
        return Some(item); // this is s
    }
}
```

The candidate is **verified against H**, never taken positionally on faith — backend data is untrusted, and a malicious node cannot feed Bob a bogus preimage that passes the SHA256 check. Once Bob has *s*, he redeems leg A with it before T1.

15.7 Message flow

The end-to-end handshake (spec §8) is:

1. `init` — Alice proposes the swap (amounts, H, T1/T2, her pubkeys).
2. `accept` — Bob agrees and supplies his pubkeys. Both sides can now reconstruct both HTLCs (`SwapParams`, `swap.rs:74-91`).
3. `funded(A)` — Alice funds chain A and announces it; Bob verifies on chain.
4. `funded(B)` — Bob funds chain B and announces it; Alice verifies on chain.
5. `redeemed` — Alice redeems leg B (revealing *s*); the courtesy message tells Bob, but his daemon would find it by chain-watching anyway.
6. Bob extracts *s* and redeems leg A. The swap is Completed.

From step 3 onward the daemon’s scheduler drives the swap automatically: it funds, redeems, and (if a deadline passes with a leg unspent) refunds on a clock. See the chapters “The Swap Lifecycle” and “Timelocks & Action Deadlines”.

15.8 vsize reference

Constant	Value (vB)	Transaction
FUND_TX_VSIZE	160	funding (preview estimate)
REDEEM_TX_VSIZE	155	hash-branch redeem
REFUND_TX_VSIZE	146	timeout-branch refund

These worst-case vsizes turn a feerate into an absolute fee before the witness exists. The fee floor for any HTLC spend is `MIN_SPEND_FEE_SAT = 1000` (`swap.rs:38`).

Chapter 16

v2 Taproot/MuSig2 Adaptor Swaps

The v2 protocol (`pact-htlc-v2`) settles the same trade as v1 but commits to each leg as a *Taproot* (P2TR) output instead of a hash-locked P2WSH script. The cooperative redeem is a single 64-byte Schnorr signature produced by a 2-of-2 MuSig2 key-path spend — on chain it is indistinguishable from an ordinary single-key payment. There is no preimage and no shared hash linking the two legs; instead a MuSig2 *adaptor signature* under a common point T binds them, and broadcasting one redeem reveals the scalar t that unlocks the other. The refund path is a single-key CLTV tapleaf — no MuSig2, no interactive nonce — so it is safe to run unattended.

Roles are unchanged from v1 (`adaptor_swap.rs:5-8`): Alice funds leg A (refund T_1), Bob funds leg B (refund $T_2 < T_1$), Alice holds the adaptor secret t .

Both chains **MUST** support Taproot. PoCX has Taproot active from genesis; Bitcoin since block 709632.

16.1 Keys and secrets

v2 reuses the v1 paths but adds a refund-key branch and a new adaptor secret (`keys.rs:91-135`, spec v2 §3):

Material	Path	v2 type & use
<i>Swap key</i>	<code>m/7228'/1'/coin(c)'/i'</code>	secp256k1, used as a BIP340 x-only key and MuSig2 signer (was ECDSA in v1)
<i>Refund key</i>	<code>m/7228'/3'/coin(c)'/i'</code>	secp256k1 x-only; sole signer of the single-key CLTV refund tapleaf — a new, separate branch
<i>Adaptor secret source</i>	<code>m/7228'/2'/i'</code>	feeds the adaptor tagged hash (was the v1 preimage source)

The swap key is the *same private key* as v1, encoded x-only instead of compressed (keys.rs:96-101). The refund key being a distinct branch (3') is deliberate: it keeps the refund tapleaf single-signature and independent of the MuSig2 aggregate, so the unattended refund path never needs an interactive ceremony.

The adaptor secret and the swap id are deterministic (keys.rs:118-142, spec v2 §3.1):

```
k = privkey at m/7228'/2'/i' (32 bytes)
t = TaggedHash("pact/adaptor/secret/v2", k) mod n (a valid secp256k1 scalar, ≠ 0)
T = t·G (the adaptor point, shared in in
swap_id = hex( TaggedHash("pact/swapid/v2", T)[0..8] ) (16 hex chars)
```

t is the v2 analog of v1's preimage s: Alice-only, seed-re-derivable, never disclosed off-chain. The point T is public and is what binds the two redeem signatures together.

16.2 The Taproot output

Each leg is one P2TR output (taproot.rs:43-120, spec v2 §4). Its two spend paths are:

- **Key path** = the 2-of-2 MuSig2 aggregate of both parties' swap keys — the cooperative redeem.
- **Script path** = a single tapleaf holding a single-key CLTV refund script.

The internal key P is the MuSig2 aggregate, with a **fixed key order** [funder, counterparty] (adaptor_swap.rs:86-90). The single refund tapleaf is (taproot.rs:72-81):

```
<T> OP_CLTV OP_DROP <funder_refund_xonly> OP_CHECKSIG
```

encoded as a TapScript leaf (leaf version 0xc0). The output key is P tweaked by the leaf's merkle root (taproot.rs:83-98), and the address is bech32m.

The leg assignments mirror v1 (adaptor_swap.rs:85-90, and leg B symmetric):

Leg	MuSig2 internal key		
	order	refund tapleaf key	T
A (Alice funds)	[alice_swap_a, bob_swap_a]	alice_refund_a	t1
B (Bob funds)	[bob_swap_b, alice_swap_b]	bob_refund_b	t2

Note — Because the key path is a 2-of-2 MuSig2 spend, a successful v2 redeem is on-chain indistinguishable from any other single-key Taproot payment. There is no hash, no script, and nothing linking leg A to leg B visible to a chain observer. Privacy is a side effect of the construction, not a bolt-on.

16.3 Cooperative redeem (key path)

The happy-path redeem spends via the key path (`taproot.rs:158-185`). It is a version-2, 1-in / 1-out transaction with `nLockTime = 0` (no timelock on the happy path) and a **non-replaceable sequence** — `0xFFFFFFFF` (`Sequence::ENABLE_LOCKTIME_NO_RBF`). That is honest signalling: the redeem's fee is sealed into the adaptor signature, so nothing can ever RBF it (a stuck redeem is CPFP'd), and the non-signal keeps external wallets from even offering a bump. It sweeps value - fee to the claimer's **fresh core-wallet sweep address** — `alice_sweep_b` / `bob_sweep_a`, exchanged in `init/accept` (empty falls back to a key-derived address). The witness is a **single 64-byte aggregate Schnorr signature** (`SIGHASH_DEFAULT`), attached by `attach_keypath_signature` (`taproot.rs:190-196`):

```
witness = [ 64-byte aggregate Schnorr sig ]
```

Worst-case vsize: `KEYPATH_REDEEM_VSIZE = 111` (`taproot.rs:40`).

Warning — The cooperative key-path redeem is **NOT RBF-bumpable**: its fee is sealed into the pre-signed adaptor sighash and cannot be re-signed unilaterally — which is why it is also broadcast without the BIP125 signal. It commits at the live market rate at `init` (`committed_mult`, default 1) and is dragged through by a CPFP child that spends the redeem's own sweep output if the market rises. See the chapter "Fees, Fee-Bumping & Auto-Refund".

Warning — The redeem's sequence value is **protocol, not policy**: it sits inside the shared MuSig2 sighash, so both parties must build the byte-identical transaction (a unit test pins it as a protocol constant, `taproot.rs:387`). rc10 changed it from the RBF-signalling `0xFFFFFFFF` to the non-replaceable `0xFFFFFFFF`, which makes rc10 a **v2 flag-day**: an rc9 peer and an rc10 peer cannot open v2 swaps with each other. Since rc10 this is gated **up-front by wire epochs**: every offer, take, `init` and `accept` carries its protocol's wire-compatibility epoch (`wire`, `v1 = 1`, `v2 = 2`; absent = 1, the pre-rc10 era), and a mismatch is refused with a clear reason before any key material is exchanged — offers from an incompatible release are already badged un-takeable on the Corkboard. A pre-rc10 peer that never sends wire still fails partial-signature verification at the handshake, a clean pre-funding abort with no funds at risk (deliberately no compat shim). Future amendments bump the epoch (`libswap::wire_epoch`) and gate the same way. In-flight v2 swaps that already exchanged partial signatures keep working on the version that made them; **settle or abort live v2 swaps before upgrading**. v1 HTLC swaps are unaffected.

16.4 Refund (script path)

The refund spends via the script path (`taproot.rs:201-256`): a single-key Schnorr signature over the CLTV tapleaf, with `nLockTime = T` (valid only once `MTP ≥ T`). Unlike the redeem, the refund **keeps the RBF-signalling sequence** (`0xFFFFFFFF`, `HTLC_SPEND_SEQUENCE`): it is single-signer, so the funder can always rebuild it locally at a higher fee — and CLTV needs a non-final

sequence anyway. The witness reveals the leaf and its control block:

```
witness = [ sig, refund_script, control_block ]
```

This path uses **no MuSig2 and no interactive nonce** — it is a plain BIP340 single-signature spend, signed deterministically from the refund key (taproot.rs:241-244). That is what makes the refund **unattended-safe**: a daemon recovering from a crash, with no live nonce state at all, can still refund from the seed alone.

Worst-case vsize: SCRIPTPATH_REFUND_VSIZE = 140 (taproot.rs:41). As in v1, the engine refuses any spend whose output would fall below DUST_LIMIT_SAT = 546 (taproot.rs:130-133).

16.5 The adaptor mechanism

The binding between the two legs is built before either party funds (spec v2 §6, adaptor_engine.rs):

1. The parties run a MuSig2 **adaptor** session over the **leg-B** redeem transaction under the point T , producing adaptor signature σ_B .
2. They run a second adaptor session over the **leg-A** redeem under the **same** T , producing σ_A .
3. **Both adaptor signatures are verified against T before any funding.** This is the Signed state in the lifecycle: reaching it guarantees that the cooperative redeems work and that Alice's broadcast will reveal t .
4. Alice (who knows t) **adapts** σ_B into a valid signature and broadcasts the leg-B redeem. The completed on-chain 64-byte signature now encodes t .
5. Bob **extracts** t by subtracting his adaptor contribution from the broadcast signature, adapts σ_A , and claims leg A before $T1$.

The symmetry to v1 is exact: instead of a preimage s published in a witness, the secret t is published *inside* the aggregate signature. Bob's daemon recovers it from chain B the same way it would scan for a v1 preimage.

Note — The funding transactions themselves are broadcast **without** the BIP125 signal (non-replaceable): each funding txid is committed into the pre-signed redeems, so nothing may ever RBF a v2 funding — replacing it would invalidate the exchanged adaptor signatures. A stuck funding is CPFP'd via its change output instead (v1 fundings and ordinary wallet sends keep signalling). See the chapter "Fees, Fee-Bumping & Auto-Refund".

16.6 Nonce safety

MuSig2 is catastrophically broken by nonce reuse — repeating a secret nonce across two different messages leaks the private key. Pact makes reuse structurally impossible (spec v2 §3.2):

- **Fresh from a CSPRNG.** Secret nonces are generated per BIP327 from the operating system's randomness, **never derived from the seed**. (The long-term keys and t are seed-derived; nonces are not.)
- **Write-ahead persisted.** A nonce is written to disk *before* it is used.
- **A monotonic state machine.** Each (swap, leg) nonce slot advances none $\rightarrow$ committed $\rightarrow$ revealed $\rightarrow$ consumed (the nonce_sessions store). Overwriting an already-used slot is **refused**, so the same nonce can never sign two messages.
- **Lost state falls back to refund.** If nonce state is lost (e.g. a crash before a session completes), the engine does **not** improvise a fresh nonce to finish the cooperative redeem — it lets the leg time out and takes the single-key timelock refund instead. Refund never needs a nonce, so this path is always available.

Warning — The cardinal rule: a lost or ambiguous nonce state always resolves to the timelock refund, never to nonce reuse. Correctness is preserved at the cost of falling back from the cooperative path to the refund path.

16.7 The recovery contract

The two roles reconstruct their long-term keys differently, mirroring how they index them (spec v1 §4.2, inherited by v2 §3): **Alice** (initiator) always reconstructs every long-term key and the adaptor secret t from the seed plus her local swap index i (keys.rs:118-128, spec v2 §9). **Bob** (participant) has no counter to fall back on — his swap and refund keys are instead *anchored* to the adaptor point T , so the seed plus T alone re-derives them, with no session state needed either. Therefore:

- Either party can **always refund** via the single-key tapleaf using only the seed (plus, for Bob, the anchor T he learned at `init`) — no live session state required.
- Alice can **always re-derive t** to complete a redeem, even with an empty state DB.
- A machine rebuilt from the seed alone (no local record at all) still reconstructs both parties' long-term keys correctly once it has the anchor — which is what makes the encrypted relay-snapshot rescue (chapter "Seeds, Wallets & Merchants") possible: the snapshot supplies the anchor and any session-specific data (assembled adaptor signatures) that isn't derivable, and the seed supplies everything else.

The only thing that cannot be reconstructed from the seed is a *consumed* MuSig2 nonce — and that is by design, because reusing one would be the unsafe act. Losing nonce state costs you the cooperative path, not your funds.

16.8 Taproot tweak parity

The MuSig2 key aggregation applies the BIP341 taproot tweak via `KeyAggContext::with_taproot_tweak`, so the signing session produces a signature that verifies under the **tweaked output key**, with the parity handling that BIP340 x-only keys require. The engine's tests confirm a key-path signature attached this way verifies under the output key (taproot.rs:363-404), proving the

tweak plumbing the aggregate signature plugs into is correct.

16.9 vsize reference

Constant	Value (vB)	Transaction
KEYPATH_REDEEM_VSIZE	111	cooperative key-path redeem
SCRIPTPATH_REFUND_VSIZE	140	single-key CLTV script-path refund
CPFP_CHILD_VSIZE	150	the CPFP redeem-bump child (see "Fees, Fee-Bumping & Auto-Refund")

The key-path redeem is the smaller spend precisely because its witness is a single signature with no script or control block to reveal.

Chapter 17

Timelocks & Action Deadlines

Every Pact swap is held together by two absolute Unix-time locktimes, $T1$ and $T2$. They are the only thing standing between an atomic swap and a one-sided loss: if a deadline slips, the chain may let a counterparty redeem a leg the other party expected to refund. This chapter sets out the structural rule that orders the locktimes, the per-network minimums that bound their duration, and the §7.4 *action-deadline margins* that build in safety slack — and explains why those margins exist.

17.1 The structural rule: $T2 < T1$

The participant's refund locktime must always be **earlier** than the initiator's:

$$T2 < T1$$

This is checked on every network for both protocol versions (`swap.rs:97-104` for v1, `adaptor_swap.rs:76-83` for v2). The reasoning, from the lifecycle (see the chapter "The Swap Lifecycle"):

- Alice reveals the secret when she redeems leg B; that publishes it so Bob can redeem leg A.
- Bob must have a window to redeem leg A *after* Alice reveals, but *before* Alice could refund leg A. So Alice's refund ($T1$) must come last.
- Bob's own funds on leg B must be refundable ($T2$) before that, so he is never left exposed if Alice simply walks away after funding.

If $T2 \geq T1$, the safety ordering collapses and the swap is unsafe. The engine refuses to construct or accept such a swap.

17.2 Network-profile minimums

Beyond the ordering, mainnet and testnet enforce duration minimums so the windows are wide enough for real confirmation times and operator slack (`validate_profile`, `engine.rs:237-257`). **Regtest is exempt** so tests can use tight timelocks.

Rule	Mainnet / Testnet	Meaning
$T2 \geq \text{now} + 3h$	yes	Bob's leg must give at least 3h before its refund
$T1 - T2 \geq 4h$	yes	at least a 4h gap between the two refund deadlines
$T1 \leq \text{now} + 48h$	yes	the whole swap must conclude within 48h
$N_A \geq 6$	yes	leg A needs ≥ 6 confirmations
$N_B \geq 1$	yes	leg B needs ≥ 1 confirmation

The 4h gap ($T1 - T2$) is the core safety window: it is the time Bob has to redeem leg A after Alice reveals the secret, before Alice's refund could race him.

17.3 The §7.4 action-deadline margins

The profile minimums shape the *timelocks*. The §7.4 *action margins* shape the *deadlines by which the engine must act* — they pull each action earlier than its raw locktime so that a transaction broadcast late, or a daemon recovering from downtime, still has slack to confirm before the chain turns against it (`action_margins`, `engine.rs:272-278`).

Network	(fund, reveal, redeem_A)
Mainnet	(3h, 2h, 1h)
Testnet	(3h, 2h, 1h)
Regtest	(0, 0, 0)

Each margin protects a specific action:

Margin	Action it bounds	Deadline
<code>fund = 3h</code>	Bob funds leg B	no later than $T2 - 3h$
<code>reveal = 2h</code>	Alice redeems leg B (revealing the secret)	no later than $T2 - 2h$
<code>redeem_A = 1h</code>	Bob redeems leg A	before $T1 - 1h$

The engine decides whether an action is still safe to start with one formula (`engine.rs:272-278`):

`action_safe(clock, margin, deadline) = clock + margin < deadline`

If the current clock plus the margin already reaches the deadline, the action is **not** safe — the engine will not start a funding or reveal it cannot expect to finish in time, and instead steers toward refund.

17.3.1 The deadline clock

What “now” means depends on the network (`engine.rs`, deadline clock):

```
deadline_clock = max(local_time, MTP)    off regtest
deadline_clock = MTP                     on regtest
```

Off regtest, the engine takes the **later** of local wall-clock time and the chain’s median-time-past (MTP) — being conservative, it never assumes the chain clock is ahead of where it actually is. On regtest it uses pure MTP so `setmocktime`-driven tests are deterministic.

17.4 Why the margins exist: the reveal-too-late race

The margins are not padding for slow blocks; they close a specific attack window — the *reveal-too-late race*. Walk it through without margins:

1. Alice waits until the very last moment before T2 to redeem leg B and reveal the secret.
2. Her redeem confirms just barely before T2.
3. Bob now has the secret — but only a sliver of time before T1 to get his leg-A redeem confirmed.
4. If Bob’s redeem does not confirm in that sliver (a fee spike, a slow block, a brief outage), T1 passes and **Alice can refund leg A** — taking back the coin Bob just paid her for. Bob has lost his leg-B coin to Alice’s redeem *and* his claim on leg A. One-sided total loss.

The margins defuse this. By requiring Alice to reveal no later than $T2 - 2h$ and bounding Bob’s redeem to $T1 - 1h$, with $T1 - T2 \geq 4h$, Bob is guaranteed a multi-hour window — between Alice’s enforced reveal deadline and his own enforced redeem deadline — to get leg A confirmed even under adverse conditions. The engine will not let Alice’s daemon reveal so late that Bob is squeezed.

Warning — The margins assume both daemons are *running* through the swap. A daemon down across its reveal or redeem deadline cannot act; the margins give a recovering daemon slack, but they are not a substitute for keeping `packtd` up for the life of a funded swap.

17.5 Offer-offset validation at post time

The same constraints are checked when an offer is **posted**, on the offsets (`t2_secs`, `t1_secs`) the maker advertises, so a malformed offer never reaches the board (`validate_offer_offsets`, `engine.rs:312-330`):

```
T2 < T1
t2_secs ≥ 3h
t1_secs - t2_secs ≥ 4h
```

Regtest is exempt here too. Validating at post time means a taker who accepts a listed offer can trust its timelocks are already sane.

17.6 Recommended values: the UI presets

Satchel's offer form ships three timelock presets, all with $T1 = 2 \times T2$ (`OfferForm.tsx:46-50`). These are the recommended values for real swaps:

Preset	T2	T1	Notes
Short	6h	12h	fastest turnaround; least slack
Medium (default)	12h	24h	the recommended balance
Long	18h	36h	most slack; for slow or congested conditions

All three satisfy the profile minimums ($T2 \geq 3h$, $T1 - T2 \geq 4h$, $T1 \leq 48h$) with comfortable headroom. **Medium** is the default and the right choice unless you have a specific reason to widen or tighten the windows.

Tip — Wider timelocks cost nothing but time-to-refund-if-things-go-wrong. If you expect fee volatility or intermittent connectivity, prefer **Long**. For a quick swap between two responsive, well-funded daemons, **Short** is fine. **Medium** is the safe default.

Chapter 18

Fees, Fee-Bumping & Auto-Refund

Atomic swaps are fee-sensitive in a way ordinary payments are not: a redeem that fails to confirm before a timelock matures can cost a party its funds (see the chapter “Timelocks & Action Deadlines”). Pact therefore treats fee-bumping and auto-refund as first-class, scheduler-driven mechanisms. This chapter covers how v1 and v2 bump stuck spends — including the important v2 asymmetry where one path can be bumped and one cannot — the funding-bump nurse, and how the engine fires the timelock refund automatically and safely.

All the numeric fee parameters in this chapter are a **single configurable policy** (`FeeBumpPolicy`, `crate::fee_policy`) rather than scattered constants; the defaults below reproduce the historical behaviour. See “Configurable fee policy” at the end of this chapter.

18.1 The unified fee-bump strategy (market-tracking)

Every local fee-bump in Pact — the v1 redeem and refund, and the v2 single-key refund — shares **one** strategy (`maybe_bump`, `engine.rs:3360`; `adaptor_bump_refund`, `engine.rs:2339`; unified 2026-06-25). It replaces an earlier market-**blind** escalator that simply added `step_pct` percent per tick until it hit the ceiling. That escalator caused a real incident: a live mainnet redeem ratcheted **159 → 358 sat/vB** into a roughly **1 sat/vB** market — a “redeem fee storm” — because nothing tied the bump to what the market actually was. The unified strategy fixes that by tracking the live estimate and gating on block height and mempool state.

Two value-capped targets — funding vs. claim (`FeeBumpPolicy::target_feerate` and `::claim_feerate`, `fee_policy.rs`). They differ only in the ceiling:

```
funding: target = min(market, value_at_risk × FEE_CAP_PCT / vsize, max_feerate_sat_vb).max(1)
claim:    target = min(market, value_at_risk × FEE_CAP_PCT / vsize).max(1)
```

- `market` is the backend’s live feerate estimate, so the bump **never bids above what the network is charging** — the storm is impossible by construction.
- `value_at_risk × FEE_CAP_PCT / vsize` is the **value-at-risk cap**: with `FEE_CAP_PCT = 100` (a hardcoded constant, `fee_policy.rs`, *not* a knob) the absolute fee can never exceed

the amount being claimed. This is Eclair's rule — "it wouldn't make sense to pay more in fees than the amount we're trying to claim on-chain."

- **The `max_feerate_sat_vb` ceiling (default 500) applies to FUNDING only.** A redeem or refund that fails to confirm before its timelock loses the whole leg, so near a deadline paying the real going rate — even above 500 sat/vB during a genuine mainnet spike — is value-justified insurance, bounded only by the value cap (`claim_feerate`). Funding is a pre-commitment spend with no counterparty- refund race, so it keeps the flat ceiling. The estimator's own sanity clamp was correspondingly raised to 10 000 sat/vB (an overflow guard, **not** the fee cap) so a real spike shows through to the claim path.
- The result is floored at 1 sat/vB.

Around that target, the loop applies three gates:

- **Block-driven cadence** (`last_action_height`, `store.rs:87`). The loop acts **at most once per block**; if the tip has not advanced since the last action it does nothing. This replaces the old 30-second scheduler tick that produced the per-tick churn behind the storm — a back-off when the chain is quiet.
- **BIP125 Rule-4 gate.** A replacement is broadcast only when target clears the current feerate by the node's **incremental-relay fee** (`getmempoolinfo` → `incremental_relay_feerate`, `chain.rs:389`). If the market has not risen — or we are already paying enough — there is nothing relayable to do, so the loop emits no replacement. When a bump is warranted, the replacement's **absolute** fee is also floored at `old_fee + incremental_relay_feerate × vsize`: because `old_feerate` is a truncated `old_fee / vsize`, a target built purely from the feerate can otherwise land just below the node's absolute Rule-4 minimum and be rejected (-26, "insufficient fee, rejecting replacement"). Flooring the absolute fee guarantees that a bump the engine decides to make is one the node will accept.
- **Evicted-only rebroadcast** (`reanchor_if_evicted`, `engine.rs:3457`). When no bump is warranted, the engine re-broadcasts the *same* transaction (same txid — invisible to the wallet) **only if it actually fell out of the mempool**. In steady state the tx is present, so this is a silent no-op — the per-tick rebroadcast the old escalator emitted is gone.

A dust guard still applies: a target whose `new_fee` would push the swept output below `DUST_LIMIT_SAT = 546` (`swap.rs:20`) is treated as "no bump" and falls through to the evicted-only rebroadcast path rather than dusting the output.

Deadline-aware redeem escalation. The market-tracking baseline keeps fees cheap when there is time, but a pure market follower never reacts to the *clock*: in a flat market a marginal redeem can sit unconfirmed into its timelock and lose the leg. So the **redeem** nurse (v1 RBF and v2 CPFP alike) is deadline-aware — as the redeem's confirm-by deadline nears it escalates the estimator's confirmation target, `conf_target` 6 → 3 → 2 → 1 (with Core's CONSERVATIVE estimate mode in the tighter bands), keyed off absolute time-to-deadline thresholds (`redeem_conf_target`, `engine.rs`). Deadlines: the initiator's leg-B redeem → T2; the participant's leg-A redeem → T1 - redeem-a margin. This only raises the *market* term — the value-at-risk cap still bounds the absolute fee, so it is insurance, never overpay. Refunds keep the cheap

baseline (we are the only spender — no counterparty race). The baseline itself is unchanged (`estimatesmartfee(6)`); regtest is exempt (test-feerate override).

18.2 v1 fee-bumping (RBF)

In v1 **both** the redeem and the refund are RBF-bumpable. Both spends signal RBF (`nSequence = 0xFFFFFFFF`), and because the v1 keys sign deterministically (ECDSA), the engine can re-sign a higher-fee replacement unilaterally (`maybe_bump`). Both follow the **unified market-tracking strategy** above: `target_feerate` toward the live market, at most once per block, only when the target clears the BIP125 Rule-4 floor, otherwise a silent evicted-only rebroadcast (`max_feerate_sat_vb = 500` by default).

18.3 v2 fee-bumping: a split design

v2 is asymmetric, and the asymmetry is load-bearing (spec v2 §8):

v2 spend	Bumpable?	Why
Single-key CLTV refund	Yes , RBF	single-key, deterministic re-sign
Cooperative MuSig2 key-path redeem	No	fee sealed into the pre-signed adaptor sighash

18.3.1 The refund is RBF-bumpable

The v2 single-key refund (`adaptor_bump_refund`, `engine.rs:2339`) bumps exactly like v1: the same **unified market-tracking strategy** (block-driven cadence, value-capped `target_feerate`, BIP125 Rule-4 gate, evicted-only rebroadcast), with a deterministic single-key Schnorr re-sign and an RBF sequence. No interactive ceremony is needed because the refund tapleaf is single-signature (see the chapter “v2 Taproot/MuSig2 Adaptor Swaps”).

18.3.2 The cooperative redeem is NOT bumpable

The cooperative key-path redeem’s fee is fixed at signing time: the fee is part of the sighash the MuSig2 adaptor session signed, and re-signing would require re-running the interactive ceremony. The engine cannot raise it after the fact. Two mitigations make this safe in practice.

(a) Commit the fee at init. The adaptor redeem feerate is fixed *before* signing (`engine.rs`, `adaptor_redeem_feerate`):

```
adaptor_redeem_feerate = live_market_estimate × redeem.committed_mult(1)
                        clamped to [MIN_REDEEM_FEERATE = 1, MAX_REDEEM_FEERATE = 500] sat/vb
                        fallback 20 sat/vB if no backend is reachable
```

With the default `committed_mult = 1` the redeem commits at the **live market rate, with no over-provision** — the CPFP child below is what lifts it if the market climbs while it’s pending,

so padding the committed fee up front is no longer needed (the multiplier was $2\times$ before this, and $3\times$ before CPFP existed). Raise `committed_mult` above 1 to deliberately pre-pay a cushion. There is no longer a regtest special-case: regtest has no fee history, so the estimate lands on its ≈ 1 sat/vB floor (`MIN_REDEEM_FEERATE`) like any other quiet chain. Note the clamp here is the **protocol** bound `MAX_REDEEM_FEERATE` (the value is negotiated into the init message and validated by the counterparty), *not* the local `max_feerate_sat_vb` bump ceiling.

(b) The CPFP redeem-bump child (v2+). Because the committed redeem is priced at market (not padded), the CPFP child is the **primary** accelerator, not just a fallback: if the market rises while the redeem is pending, the claimer drags it through with a child-pays-for-parent transaction (`adaptor_cpfp_bump`, `engine.rs:1942-1985`):

- The child spends the redeem's **own vout 0** — the claimer's wallet-owned sweep output — so it is self-funded and needs no extra inputs.
- The child signals RBF, so the *child* itself can be bumped further.
- Child vsize is `CPFP_CHILD_VSIZE = 150` (`engine.rs`).
- It emits `adaptor-cpfp` / `adaptor-rebroadcast` events.
- The child's package target is **deadline-aware** (see "Deadline-aware redeem escalation" above): as `T2` (initiator) or `T1 - margin` (participant) nears it escalates `conf_target`, and it is bounded by the sweep output's value — if that output is too small to fund the child to the target, the shortfall is logged rather than silently under-bumped.

This is a **plain CPFP** (no `submitpackage` / `package relay`): the parent redeem stays relayable on its own, so a normal CPFP child suffices to drag it through. Proven by `test_adaptor_redeem_cpfp` (and `..._1tc`, the first v2 swap on litecoind).

Note — The cooperative redeem is not RBF-bumpable, so it commits at the market rate up front and relies on a CPFP child to drag the parent through if conditions tighten before the deadline. The single-key refund path is always bumpable, so the *funder* is never stuck. See the chapter "Network Support, Reorgs & Safety".

18.4 The funding-bump nurse

The redeem and refund spend an existing HTLC output, so their fees come *out of that output*. The **funding/lock** is different: it is the only wallet-funded action in a swap, and it had no bump at all — if the market rose above the rate the lock went out at, it could sit unconfirmed. The funding nurse closes that gap. It runs each scheduler tick while our own funding is **unconfirmed** and we are still before that leg's fund-margin deadline, and chases the current market:

`target = min(market, max_feerate_sat_vb, funding.reservation_mult × old_feerate)`

where `old_feerate = fee / vsize` of the broadcast funding (recomputed live, not persisted). The `reservation_mult × old_feerate` bound keeps the bump within the balance the pre-flight funds gate (`ensure_can_fund`) set aside as headroom.

The market estimate is a per-coin ~30-min target, not a blind 6 blocks. Everywhere funding picks a feerate — the initial broadcast, this nurse, and the funds-gate headroom — it uses

`estimatesmartfee(conf_target)` where `conf_target` is derived from the coin's block spacing (`funding_conf_target, chain.rs`):

```
funding_conf_target = clamp(1800 / target_spacing_secs, 1, 6)
```

so a lock targets confirmation within roughly half an hour on any chain rather than a fixed 6-block target, which is a full **hour** on Bitcoin's 10-min blocks. Bitcoin (600 s) → **3**; Litecoin (150 s) → 12 clamped back to the standard **6** (6 blocks ≈ 15 min is already inside the budget); BTCX (120 s) → **6**. The 30-min cap only ever pulls the target *faster* on slow chains — it never prices a fast chain below the historical `estimatesmartfee(6)` baseline, and it needs no per-coin config (block spacing is already in `coins.toml`). Redeem and refund keep their own targets (deadline-aware for redeem, flat 6 for refund); this applies to funding only.

This is **liveness, not safety**: a funding that can't keep up simply stalls and the timelock refund returns the coin — never a loss. The two protocols bump differently, and the asymmetry mirrors the redeem/refund split:

	v1 funding	v2 funding
Mechanism	RBF (bumpfee)	CPFP-via-change
Broadcast	explicitly BIP125-replaceable (the nurse's RBF is accepted)	non-replaceable — no BIP125 signal (Core <code>{"replaceable": false}</code> , bdk <code>ENABLE_LOCKTIME_NO_RBF</code>), so an external wallet won't even offer a bump
Why	the only outpoint-dependent downstream tx is the single-key refund — re-sign it locally against the new outpoint	the outpoint feeds the 2-of-2 MuSig2 adaptor sigs already exchanged; RBF would invalidate them, so spend the change output instead and keep the outpoint fixed

- **v1 (maybe_bump_funding_v1)**. RBF via the wallet, then re-locate the HTLC output, rebuild + re-sign the single-key CLTV refund against the new outpoint, and persist atomically. Safe for the counterparty: they detect the lock by **scriptPubKey, not txid** (`find_funding → scantxoutset`), so an RBF that keeps the HTLC output identical is invisible to them — and the nurse runs only while the funding is unconfirmed, before they have waited out the confirmations. The funding is broadcast explicitly BIP125-replaceable so bumpfee is accepted. A crash mid-bump self-heals: `find_funding` re-discovers the live outpoint on restart.
- **v2 (maybe_bump_funding_v2)**. A CPFP child spends the funding's wallet-owned **change** output, leaving the funding outpoint — and therefore the exchanged adaptor sigs and the refund — untouched. The v2 funding is broadcast **without** the BIP125 signal: nothing

may ever RBF it (its txid is committed into the pre-signed MuSig2 redeems), and the non-signal keeps external wallets from even offering a replacement. A funding with no change output (exact-UTXO) cannot be CFPF'd → it stalls → refund (acceptable).

A recoverable bumpfee/sign failure (e.g. insufficient funds — the funds gate is a soft pre-flight, not a lock — or a not-replaceable tx) is a graceful skip event for that tick, never a crash: the funding stalls and refunds.

Note — The nurse is the **only** legitimate bumper of a live swap's funding. The wallet-level bumpfee RPC (chapter "API: Node, Seed, Merchants, Coins") refuses a txid that funds a live swap — v1 HTLC and v2 funding txids alike — with `<txid> funds live swap <id>` — the swap engine manages its fee (see `get/setfeepolicy`), bumpfee must not replace it. A hand-RBF of a v2 funding would change its txid and invalidate the pre-signed MuSig2 redeems; v1 fundings are re-bumped under the swap's own `FeeBumpPolicy`. Satchel likewise hides its Bump action on those rows.

18.5 Auto-refund

The refund is the safety net: if a counterparty disappears after a leg is funded, the funder gets its coin back once the leg's timelock matures. It is scheduler-driven and clock-based — the operator does nothing.

18.5.1 v1 auto-refund

The v1 refund is **signed and persisted at funding time** (`engine.rs:2250-2266`). The fully-signed refund transaction exists on disk the instant a leg is funded, ready to broadcast even across a daemon restart.

It fires from `try_refund_due` (`engine.rs:2877-2908`), which broadcasts only when **both** conditions hold:

`tip_median_time_min() ≥ locktime` (the least-advanced backend's MTP has reached T)
AND the HTLC output is still unspent

Using the **least-advanced** backend's MTP (`tip_median_time_min`) is conservative: the engine waits until *every* watched chain agrees the timelock has matured before refunding.

Several safety details:

- **M7 guard.** `refund()` refuses to broadcast a refund that would *race a counterparty redeem*. If the counterparty has already redeemed (or could), the engine does not fire a refund that the chain would reject or that could double-spend the wrong way.
- **-27 is success.** A node returning -27 ("transaction already in block chain" / already known) is treated as success, not an error — the refund (or an equivalent) is already on chain.
- **Armed until N-deep.** The refund stays armed until the redeem is confirmed N blocks deep (spec §9.5), so a shallow reorg that un-confirms a redeem re-arms the refund.

18.5.2 v2 auto-refund

The v2 refund is **not** pre-signed at funding. It is re-derived from the seed on each call (`adaptor_refund`, `engine.rs:1599-1660`): a single-key, deterministic, unattended-safe Schnorr spend over the CLTV tapleaf. Readiness is the same MTP test:

```
tip_median_time_min() ≥ leg.locktime
```

Re-deriving from the seed (rather than persisting a signed tx) is possible because the refund key is a deterministic seed branch and the refund path needs no MuSig2 nonce — so even a daemon with an empty state DB can refund. This is the design asymmetry to remember: **v1 pre-signs the refund; v2 re-derives it.**

Note — Both versions refund off the chain clock (MTP), never off local wall-clock alone. A timelock is “mature” only when the watched chains’ median time past actually reaches it.

18.6 Confirmation depth as the reorg-finality knob

How many confirmations the engine waits for before treating a leg as final is the per-coin reorg-finality knob (`default_confirmations`, `engine.rs:388-394`):

Chain class	Default confirmations
Regtest	1
Fast chain (< 5-min block spacing; BTCX ≈ 120s)	10
Slow chain (≥ 5-min spacing; BTC ≈ 600s)	6

Override per coin via `Engine.coin_confirmations` (`satchel.json` → `--coin-confs`), floored at ≥ 1 . Deeper confirmations mean stronger reorg protection at the cost of a slower swap; this is the dial an operator turns to trade finality against speed. See the chapter “Network Gating, Reorgs & Safety”.

18.7 Configurable fee policy

The numeric fee parameters are one struct, `FeeBumpPolicy` (`crate::fee_policy`), owned per-merchant by `pactd`’s store and surfaced as typed RPC:

- `getfeepolicy` / `setfeepolicy` — read and update over JSON-RPC (each field a plain typed param, no JSON blob), callable from the CLI like any other method. A change is validated, applied to the live engine, and persisted, so it survives a restart with no Satchel involved.
- **Satchel** → **Settings** → **Fees** edits the **active merchant’s** policy over the same RPC (applied live, no relaunch). It exposes three knobs (`max_feerate_sat_vb`, `reservation_mult`, `committed_mult`); every fee is market-derived, so there is no minimum-fee floor to show. The retired `step_pct` knob was removed from this page when the bump strategy was unified.

Changes take effect on the **next** bump; swaps already funded keep the `committed_mult` and gate reservation they were funded under (both fixed at funding time).

Field	Default	Meaning
<code>max_feerate_sat_vb</code>	500 sat/vB	ceiling for funding bumps only (redeem/refund are bounded by the value-at-risk cap instead, so they can exceed it near a deadline); settable 1..=500. The estimator's own sanity clamp is a separate 10 000 sat/vB overflow guard.
<code>funding.reservation_mult</code>	3×	funds-gate headroom + funding-nurse bound (× <code>old_feerate</code>)
<code>redeem.committed_mult</code>	1×	v2 committed-redeem multiplier over live market (1 = commit at market, no padding; CPFP lifts it if the market climbs)

Note — `min_fee_sat` (the old 1000-sat floor) and `redeem.step_pct` / `refund.step_pct` (default 50) are **retired**. The floor and the per-tick percentage step both drove the old market-blind escalator that the unified market-tracking strategy replaced. They survive in the `FeeBumpPolicy` struct only for `serde` back-compat (an old stored policy that still sets them deserializes cleanly) and are **not** exposed by `getfeepolicy` / `setfeepolicy` or the Satchel Fees page. Setting them has no effect on bump behaviour.

Other fee-related constants remain fixed (not policy):

Constant	Value	Meaning
<code>FEE_CAP_PCT</code>	100%	value-at-risk cap in <code>target_feerate</code> : a bump's fee never exceeds the amount being claimed (Eclair's rule). Hardcoded (<code>fee_policy.rs:28</code>), not a knob
<code>DUST_LIMIT_SAT</code>	546 sat	swept output must stay above this

Constant	Value	Meaning
MAX_REDEEM_FEERATE	500 sat/vB	protocol bound on the negotiated redeem feerate (distinct from max_feerate_sat_vb)
CPFP_CHILD_VSIZE	150 vB	the CPFP redeem/funding-bump child
FUNDING_VSIZE_EST	250 vB	sizing estimate for the funds-gate reservation

The unified RBF strategy (v1 redeem/refund, v2 refund) and the v2 commit-at-market-plus-CPFP strategy are two answers to the same question — *how do I make sure a time-critical spend confirms?* — chosen because an RBF-able spend can re-sign freely toward the market, while v2's cooperative redeem cannot and must commit its fee up front.

Chapter 19

Network Support, Reorgs & Safety

This chapter states, precisely, where Pact’s swap protocols stand on safety: which versions run on which networks, how the engine picks a protocol, how confirmation depth defends against reorgs, and the safety properties that hold in every swap. Both swap protocols — v1 (HTLC) and v2 (Taproot/MuSig2 adaptor) — are reviewed and live on mainnet. The point of this chapter is to make the safety posture legible.

19.1 Network support

Both protocols run on every network — regtest, testnet, and mainnet:

Protocol	State
v1 (pact-htlc-v1)	Live on every network, including mainnet
v2 (pact-htlc-v2)	Live on every network, including mainnet

In the code, `ADAPTOR_BUILT = true (registry.rs:53)` and `ADAPTOR_MAINNET_ENABLED = true (registry.rs:59)` together make `adaptor_allowed` return true on every network; the test suite asserts mainnet is allowed. v1 has no network restriction.

Note — Some older design docs and a few stale source comments still describe v2 as “refused on mainnet” or “not built”. Those are out of date. The shipped code runs v2 on every network. Treat the registry constants (`ADAPTOR_BUILT`, `ADAPTOR_MAINNET_ENABLED`) as the source of truth.

19.2 Network isolation: the foreign-pactd guard

Each network runs `pactd` on its own JSON-RPC port — **regtest 9739, testnet 9738, mainnet 9737** — so a mainnet and a regtest install never share a daemon. Before the launcher adopts a `pactd` already listening on the configured `listen` port, it confirms the daemon is genuinely *ours*: it reads the `data-dir` `.cookie`, calls `getinfo`, and checks both that the cookie

authenticates **and** that `getinfo.network` matches the active network (`probe_adoptable`, `satchel/src/main.rs:472`).

If something is healthy on the port but is **not** this install's engine — a different network's daemon, or one whose cookie is unreadable — the launcher **refuses to start** rather than silently latching onto the wrong daemon (which would point every RPC at another network's chains) or starting with empty auth. It fails loud with an actionable error (`satchel/src/main.rs:1251`):

```
configured listen <listen> is already serving a different engine (not this
<network> install) – another network's Satchel/pactd is using that port. Set a
distinct `listen` in satchel.json (regtest uses 9739, testnet 9738, mainnet
9737), or stop the other instance, then relaunch.
```

The usual trigger is a stale or copied `satchel.json` whose `listen` still points at another network's port. The remedy is to give each network a distinct `listen` (the defaults above already are) or stop the colliding instance.

19.3 Protocol selection prefers HTLC

When a pair could run either version, the engine **prefers v1 HTLC** (`select_protocol`). It chooses v2 adaptor only for Taproot pairs that lack an HTLC option:

- If both legs support CLTV + segwit (the v1 requirement), the engine selects **v1 HTLC**.
- v2 adaptor is selected only for taproot-capable pairs where HTLC is not available.

The shipped **BTCX** ↔ **BTC** pair therefore defaults to **v1 HTLC**. v2 is the more advanced construction (better privacy, single-sig refund); the engine uses the older, more-exercised path whenever it is available and v2 where it is the better or only fit.

19.4 Confirmation depth and reorg protection

A blockchain reorganization can un-confirm a transaction the engine already acted on. Pact's defense is to withhold finality until a leg is buried deep enough that a reorg is implausible (`default_confirmations`, `engine.rs:388-394`):

Chain class	Default N
Regtest	1
Fast chain (< 5-min spacing; BTCX ≈ 120s)	10
Slow chain (≥ 5-min spacing; BTC ≈ 600s)	6

Two behaviors flow from this:

- **Auto-redeem / completion is withheld until N-deep.** The engine does not treat a swap as done — and keeps the refund armed — until the relevant spend is confirmed N blocks deep. A shallow reorg that un-confirms a redeem re-arms the refund rather than leaving a party exposed.

- **Reorg alert.** If a watched HTLC or leg output *vanishes* from the chain (a reorg dropped its funding or spend), the engine raises a reorg alert so the operator and the state machine can react rather than silently proceeding on a stale view.

Override N per coin via `Engine.coin_confirmations(satchel.json → --coin-confs)`, floored at ≥ 1 . Deeper is safer but slower.

Note — Spec §7.3’s suggested-minimums table lists $N_B = 3$ for BTC, but the engine’s heuristic `default_confirmations` returns **6** for any chain with ≥ 5 -minute block spacing, including BTC. The engine default is more conservative than the spec’s floor; the spec value is a *minimum*, not the shipped default.

19.5 The wallet-lock funding gate

Before committing a trade, the engine verifies that the funding coin’s node wallet is actually **unlocked** — a pre-flight check on the side that is about to spend. If the wallet is **encrypted and locked**, the engine **refuses up front** rather than accepting a trade it cannot fund: the taker’s get-leg is refused at **take**, and the maker’s give-leg is refused at **post**, each with a message telling the user to unlock the wallet (`walletpassphrase`) and keep it unlocked until the swap completes.

The reason the balance check alone is not enough: a locked wallet still **reads** its balance, so the funds gate passes, but it **cannot sign** the funding transaction — `signrawtransactionwithwallet` returns RPC -13 (“wallet must be unlocked”). Without this gate that failure surfaced only at funding time, after the handshake, stranding the swap. The gate **fails open** on a transient `get-walletinfo` error (node reachability is already covered by the balance read), so a momentary RPC hiccup never blocks an otherwise-fundable trade.

Note — **Companion funding self-retry.** A v1 fund that fails to broadcast *after the state has already advanced* — e.g. a wallet locked mid-swap — is re-attempted on each scheduler tick. The retry is idempotent (it locates the funding on-chain first, so it never double-funds) and self-heals the moment the wallet is unlocked again; it composes with the pre-funding timeout-abort. v2 adaptor funding does **not** yet auto-retry on a tick: it fails closed into a recoverable `Accepted` state, resumable via relay re-drive or a manual `adaptor_fund`.

Note — **Pre-funding timeout-abort now covers v2 too.** A v1 handshake stuck pre-funding has long self-aborted after `PRE_FUNDING_TIMEOUT_SECS` (15 minutes). A v2 adaptor handshake stalled in `created`, `accepted`, or `nonces_exchanged` — before either leg is funded — now does the same, silently and independently on each side’s own clock (`signed` is excluded, as funding may already be in flight by then). Previously a stalled pre-signed v2 handshake was inert to the ticker: neither the abort RPC (which only read the v1 table) nor a timeout could clear it, so a maker gone unreachable during the handshake left the taker with a record that could neither be cancelled nor time out. See the chapters “API: v1 HTLC Swaps” and “API: v2 Adaptor Swaps”.

19.6 Safety properties

Pact's crypto core — the v1 witness construction, the v2 Taproot output and tapleaf, the key derivation paths, and the adaptor reveal — matches the specs and the pinned test vectors. Two operational details are worth understanding:

- **v2's cooperative redeem is not RBF-bumpable.** Its fee is sealed into the pre-signed adaptor sighash, so it cannot be fee-bumped after the fact the way an ordinary RBF transaction can. The engine handles this two ways: the redeem commits at the live market rate at init (live estimate $\times$ committed_mult, default **1**, clamped) plus a CPFP redeem-bump child that spends the redeem's own sweep output to lift the package feerate if the network gets busy. The single-key refund path is always bumpable, so the funder is never the stuck party. See the chapter "Fees, Fee-Bumping & Auto-Refund".
- **Relay trust is liveness-only.** Pact never trusts a relay or a chain backend for *safety* — timelocks and on-chain enforcement protect funds regardless of what any relay says. A misbehaving or offline relay can only delay a swap (a liveness failure), never steal from it. The remedy for a dead relay is the timelock refund.

Tip — The atomicity guarantees are real and chain-enforced. As with any self-custody software, keep pactd running for the life of any funded swap, and prefer the **Medium** or **Long** timelock presets for larger value (see the chapter "Timelocks & Action Deadlines").

19.7 Summary

The safety model is: the chain enforces the deal, timelocks bound the worst case, confirmation depth defends against reorgs, and relays are trusted only for liveness. v2's cooperative redeem is non-bumpable by construction; it commits at the market rate and is dragged through by a CPFP child. Both protocols are reviewed and live on every network.

Part V

Transports

Chapter 20

The Noticeboard Abstraction

A *noticeboard* is where offers are advertised and coordination messages are relayed. Pact treats the noticeboard as a pluggable transport behind a single trait, so the swap engine does not care whether an offer travelled over an HTTP Corkboard or a set of Nostr relays. Everything crossing the boundary is a transport-agnostic, signed Envelope (see the chapter “Wire Format (pact-proto)”); the relay carries only opaque, end-to-end-sealed blobs.

This chapter covers the trait itself, its two shipped implementations, and the fan-out/browse model that lets two parties trade as long as they share *one* board.

20.1 The Noticeboard trait

The trait lives in libswap at `pact/libswap/src/board.rs`. It has exactly five methods — there is no reputation, scoring, or accounts surface anywhere on it:

```
pub trait Noticeboard {  
    fn post_offer(&self, offer: &Envelope) -> Result<String>;  
    fn offers(&self) -> Result<Vec<Envelope>>;  
    fn revoke(&self, revocation: &Envelope) -> Result<()>;  
    fn relay_send_blob(&self, to: &str, blob: &str) -> Result<()>;  
    fn relay_poll(&self, poll: &Envelope) -> Result<Vec<(i64, String)>>;  
}
```

Method	Purpose
<code>post_offer</code>	Publish a signed offer envelope. Returns the offer id (the envelope <code>swap_id</code>).
<code>offers</code>	List the board’s currently active offers (signed Envelopes).
<code>revoke</code>	Withdraw one of our own offers via a signed revoke envelope.

Method	Purpose
relay_send_blob	Store-and-forward a <i>pre-sealed</i> blob to recipient to (an x-only identity pubkey, hex). The blob is opaque to the board.
relay_poll	Fetch our coordination mail. Takes a signed relay_poll envelope (proving we own the recipient identity); returns (cursor, blob) pairs.

Note — The trait is deliberately **not** Send + Sync. Boards are built on demand and used synchronously inside a single engine call — no await, no thread hop. Nostr-Board also borrows the engine’s SQLite Store, whose connection is !Sync.

20.1.1 Sealed relays, signed offers

The split between offers and relay is intentional:

- **Offers are public and signed.** Anyone may read them; the BIP340 signature proves the maker authored the terms, and listings cannot be forged.
- **Relay messages are sealed.** relay_send_blob only ever carries a PACTSEALED1: blob produced client-side. A board operator sees the recipient pubkey and ciphertext — never the sender, message type, or contents. There is no plaintext relay path. (Sealing is covered in “Wire Format (pact-proto)”.)

20.2 The two implementations

20.2.1 BoardClient — HTTP Corkboard

BoardClient (board.rs) speaks the Corkboard HTTP API over a base URL. Each trait method maps to one HTTP call:

Method	HTTP call
post_offer	POST <base>/v1/offers → { "offer_id": ... }
offers	GET <base>/v1/offers → { "offers": [...] }
revoke	POST <base>/v1/offers/revoke
relay_send_blob	POST <base>/v1/relay with { "to", "blob" }
relay_poll	POST <base>/v1/relay/poll → { "messages": [...] }

Here cursor is the Corkboard’s own autoincrement row id. The full wire surface is documented in the chapter “Corkboard (self-hostable board)”.

20.2.2 NostrBoard — local buffer over Nostr

NostrBoard (pact/libswap/src/nostr_board.rs) implements the same trait but performs **no network I/O at all**. Every method reads or writes the engine's nostr_* SQLite buffers:

- post_offer / revoke push a row onto nostr_outbox.
- offers reads the active rows of nostr_offer_cache.
- relay_send_blob queues a gift-wrap for the recipient on nostr_outbox.
- relay_poll reads nostr_inbox past the cursor.

A separate background service (pactd/src/nostr_service.rs) drains the outbox to relays and fills the inbox/cache from subscriptions. Because the inbox uses a local autoincrement id, relay_poll returns the *exact* (i64, blob) contract the HTTP board does, so the engine's cursor/dispatch loop is shared unchanged. The relay-to-event mapping is covered in "Nostr Transport (pact-nostr)".

20.3 Fan-out vs browse

The engine wires every configured board into one list, via boards() (engine.rs): one entry per comma-separated HTTP --board-url, plus a single aggregate entry named "nostr" if any Nostr relay is configured. From there, the two directions are asymmetric.

20.3.1 POST fans out to all boards

When you post or relay, the engine seals once and loops over *every* configured board. relay_send_all (engine.rs) is the canonical example:

```
fn relay_send_all(&self, to: &str, envelope: &Envelope) -> Result<()> {
    let blob = crate::board::seal_envelope(to, envelope)?;
    let mut sent = false;
    for (_, board) in self.boards()? {
        if board.relay_send_blob(to, &blob).is_ok() {
            sent = true;
        }
    }
    // ... succeeds if ANY board accepted
}
```

The same one sealed blob goes to all boards; the call **succeeds if any single board accepts it**. Offer posting fans out the same way. This is best-effort redundancy: a flaky or hostile board cannot block you as long as one path works.

20.3.2 Browse selects one board

Browsing is per-board selection. list_board_offers(sel) (engine.rs, the method behind the boardlistoffers RPC) picks the **one** board named sel (an HTTP URL, or the literal "nostr"),

defaulting to the first configured board if `se1` is absent. It then filters out any offer you have locally revoked. The UI browses one board at a time; it does not merge listings across boards.

Note — All Nostr relays collapse into a single logical board, keyed in the engine as `relay_cursor:nostr`. Whether you configure one relay or six, the browse view and the cursor treat them as one board. Adding relays adds redundancy, not separate listings.

20.4 The key property: one board in common

Because posting fans out to every board while browsing reads one, two parties need only **one board in common** to trade. The maker may post to a Corkboard *and* a half-dozen Nostr relays; the taker only has to be watching one of them. There is no global, canonical order book — just overlapping noticeboards, any one of which is enough.

20.5 The `relay_poll` cursor model

Coordination mail is pulled, not pushed. Each board exposes a **per-board monotonic cursor**:

- The engine persists the highest cursor it has processed per board (`relay_cursor:<board>`), so a poll only ever returns *new* messages.
- For the HTTP Corkboard the cursor is the `relay` table's `AUTOINCREMENT` id; for Nostr it is the local `nostr_inbox` autoincrement id. Either way, `relay_poll` returns `Vec<(i64, String)>` of `(cursor, blob)`.
- The poll envelope is **signed** (type `relay_poll`, with `body.since_id`), proving the caller controls the recipient identity, so no one can read another party's mailbox.

Tip — A board reset (the SQLite file replaced) would leave a stale cursor *ahead* of every message on the fresh board. The Corkboard guards against this by serving from `0` whenever the polled cursor exceeds the maximum id it holds. See “Corkboard (self-hostable board)”.

Chapter 21

Wire Format (pact-proto)

Every message that crosses a noticeboard — offers, takes, and the swap handshake — is a single, signed JSON object called an *Envelope*. The pact-proto crate (pact-proto/src/) defines the envelope, its canonical encoding, the BIP340 signing rule, the sealed-blob format used by the blind relay, and the private-offer slip codec. The crate is deliberately chain-agnostic: it knows nothing about HTLCs or Taproot. Typed message *bodies* live in the engine (libswap), keeping pact-proto a pure wire layer.

21.1 The Envelope struct

Defined in pact-proto/src/envelope.rs:

```
pub struct Envelope {
    pub v: u32,
    #[serde(rename = "type")]
    pub msg_type: String,
    pub swap_id: String,
    /// 32-byte x-only identity pubkey, hex.
    pub from: String,
    pub body: Value,
    /// 64-byte BIP340 signature, hex. Empty until signed.
    #[serde(default, skip_serializing_if = "String::is_empty")]
    pub sig: String,
}
```

Field	Wire name	Meaning
v	v	Envelope version (u32).
msg_type	type	Message type (serde-renamed to type on the wire).

Field	Wire name	Meaning
swap_id	swap_id	The swap this message belongs to.
from	from	Sender identity: a 32-byte BIP340 x-only pubkey, hex.
body	body	Opaque JSON payload; shape depends on type.
sig	sig	64-byte BIP340 Schnorr signature, hex. Omitted entirely when empty.

Note — On the wire the field is `type`, not `msg_type`. The Rust field is renamed because `type` is a reserved word. When you craft envelopes by hand, use `"type"`.

21.2 Message types

The `type` value is one of:

`offer`, `take`, `revoke`, `relay_poll`, `init`, `accept`, `funded`, `redeemed`, `abort`

The first four are board/coordination messages; `init/accept/funded/redeemed/abort` drive the swap handshake itself. The *bodies* for these live in the engine, not in `pact-proto`.

21.2.1 OfferBody

The body of an offer envelope (`libswap board.rs`):

Field	Type	Meaning
protocol	string	<code>pact-htlc-v1</code> or <code>pact-htlc-v2</code> .
network	string	<code>regtest</code> / <code>testnet</code> / <code>mainnet</code> .
give_asset	string	Coin id the maker gives.
give_amount	u64	Amount, smallest units.
get_asset	string	Coin id the maker wants.
get_amount	u64	Amount, smallest units.
t1_secs	u32	T1 timelock as a duration , not an absolute time.
t2_secs	u32	T2 timelock duration (<code>t2_secs < t1_secs</code>).
ttl_secs	u64?	Offer lifetime; defaults to 24h when omitted.

Field	Type	Meaning
created	u64	Unix creation time, inside the signed body, so expiry is verifiable from the envelope alone.

A take body simply echoes the maker's full signed offer back: { "offer": <full signed offer envelope> }. The maker can therefore rebuild the terms statelessly and cannot be tricked into different terms — the embedded offer's signature is re-verified and must carry the maker's own identity.

21.3 Canonical JSON

Signatures commit to a *canonical* JSON encoding (envelope.rs, spec §8.1) so that two implementations always hash the same bytes:

- Object keys are sorted **bytewise**.
- **No whitespace** between tokens.
- Integers are rendered in **decimal**.
- **Floats are rejected** — a number that is not an i64/u64 is an error, not a rounding hazard.

The canonical writer is implemented explicitly (not via serde map ordering) so that a `preserve_order` feature enabled by some unrelated dependency can never silently change a signature.

21.4 BIP340 signing

Signing (envelope.rs) is BIP340 Schnorr by the sender's identity key:

1. Clear `sig` (sign over the unsigned envelope).
2. Serialize to canonical JSON.
3. Compute the digest: `tagged_hash("pact/msg/v1", canonical_bytes)`.
4. Sign with deterministic `sign_schnorr_no_aux_rand`; hex-encode into `sig`.

Verification decodes `from` (32 bytes) and `sig` (64 bytes), recomputes the same digest, and checks the Schnorr signature. **The caller must additionally check that `from` matches the identity pinned for this swap** — `verify` only proves the signature is internally consistent, not that it came from the expected party.

The `tagged-hash` construction itself (pact-proto/src/crypto.rs) is the BIP340 form: `tagged_hash(tag, msg) = SHA256(SHA256(tag) || SHA256(tag) || msg)`.

21.5 swap_id derivation

swap_id is 16 hex characters (8 bytes), derived via a tagged hash (crypto.rs):

- v1 HTLC: swap_id = hex(tagged_hash("pact/swapid/v1", H)[..8]), where H = SHA256(preimage).
- v2 adaptor: the same shape with tag pact/swapid/v2 over the adaptor point T.

Offer swap_ids (before a concrete swap exists) are random 8-byte nonces.

21.6 The PACTSEALED1 sealed-blob format

The blind relay never sees plaintext. A coordination envelope addressed to a recipient is sealed client-side (pact-proto/src/seal.rs, spec §10) into:

PACTSEALED1:<hex(ephemeral_pubkey 33B)>:<hex(nonce 12B)>:<hex(ciphertext)>

Construction:

1. Lift the recipient's x-only identity pubkey to its **even-Y** point.
2. Generate a fresh ephemeral secret key; ECDH against the recipient point.
3. Derive the symmetric key: tagged_hash("pact/relay/ecdh/v1", shared).
4. Pick a random 12-byte nonce; encrypt the envelope JSON with **ChaCha20-Poly1305**.

Opening reverses this with the recipient's identity key (negating the secret if its parity is odd, to match the even-Y lift). The operator sees only the ephemeral pubkey, the nonce, and the ciphertext — sender, type, and contents are all hidden.

Warning — There is **no plaintext downgrade**. open_envelope rejects any blob that does not start with PACTSEALED1. A board cannot inject an unencrypted envelope and have the engine parse it.

21.7 The private-offer slip codec

A *slip* is a private (off-market) offer serialized for an out-of-band channel such as a chat message (pact-proto/src/slip.rs). It is the **same** signed offer envelope, with no new fields and no protocol bump:

pactoffer1:<base64url(canonical_json(offer_envelope))>

The base64url is the RFC 4648 URL-safe alphabet, **unpadded** (no =).

decode_slip is the only trust gate. Before returning anything to the caller it rejects, in order:

1. an unknown or missing pactoffer1: prefix,
2. malformed base64url,
3. bytes that are not a valid Envelope,
4. an envelope whose type is not "offer",
5. a bad BIP340 signature over the canonical JSON (verified against from).

Because a slip is *bearer* — anyone holding it can act on it — this validation is what guarantees a pasted slip is the maker's genuine, untampered offer before a card is ever shown. Private offers are covered end-to-end in the chapter "Private (Off-Market) Offers".

Chapter 22

Nostr Transport (pact-nostr)

Nostr is Pact’s default offer/relay transport. The `pact-nostr` crate (`pact-nostr/src/lib.rs`) is **pure mapping**: it converts Pact Envelopes to and from Nostr events and back. It opens no relay connections, runs no async code, and touches no engine state. The relay pool, buffering, and polling live in the background service (`pactd/src/nostr_service.rs`); the Noticeboard facade (`NostrBoard`) lives in `libswap`. This separation is what lets the engine’s existing `messages::verify` and `seal::open_envelope` still apply unchanged on the far side of a relay.

22.1 Event kinds and constants

```
pub const OFFER_KIND: u16 = 31510; // addressable advert (NIP-33)
pub const GIFTWRAP_KIND: u16 = 1059; // sealed relay message (NIP-59)
pub const RELAY_TTL_SECS: u64 = 30 * 60; // rolling relay TTL (1800s)
const DEFAULT_TTL_SECS: u64 = 24 * 3600; // offer-body TTL fallback
```

22.1.1 Offers — kind 31510

An offer maps to an **addressable** (NIP-33) event of kind 31510:

- The `d` tag is the `swap_id`, so each (`maker pubkey`, `swap_id`) has exactly one current event — a re-publish replaces it.
- The content is the **unchanged signed offer envelope JSON**. The inner Pact signature authenticates the terms; the outer Nostr signature lets relays and clients accept the event. Both are signed by the maker’s identity key.
- Plain descriptive tags carry `network`, `give`, and `get` (the `give/get` coin ids). Discovery subscribes by kind and filters the `pair/network` client-side, mirroring the HTTP board.
- The custom kind keeps non-spendable swap offers out of generic Nostr marketplace clients.

The reader (`offer_from_event`) verifies the Nostr event signature, confirms its author matches the inner `offer.from`, and returns the envelope; the engine still verifies the inner Pact signature and freshness before trusting terms.

Note — An addressable offer re-publishes with a **new event_id but the same d tag**, and a relay pool can serve a **stale** copy out of order. The `nostr_offer_cache` therefore keeps **exactly one row per d tag**: an incoming event older than the cached one is ignored, otherwise it replaces every row for that d tag, and reads collapse to the freshest event per d tag. So one offer can never render as two cards on the board — the previous out-of-order arrival could otherwise double a listing.

22.1.2 Gift wraps — kind 1059

A relay message maps to a NIP-59-style gift wrap of kind 1059:

- The content is the existing `PACTSEALED1: blob` (already sealed by `pact_proto::seal`).
- A `["p", recipient]` tag addresses the recipient by x-only pubkey.
- The event is signed by a **fresh, one-time ephemeral key** (`Keys::generate()`), which hides the sender from relays.

So the seal hides the *contents* from everyone, and the ephemeral author hides the *sender* from the relay. The recipient unwraps the event and opens the seal with their identity key, which also proves the message was addressed to them.

Note — One-time gift-wrap keys are never reused. A relay learns only that “pubkey X has a mailbox”; it cannot link a wrapped message back to its sender.

22.1.3 Rescue snapshots — kind 31512

A distinct addressable (NIP-33) kind carries the seed-only swap-rescue snapshots described in the Pact handbook’s “Seeds, Wallets & Merchants” chapter (`SNAPSHOT_KIND: u16 = 31512`, `pact-nostr/src/lib.rs`):

- **Author and reader are the same party.** Unlike an offer (public) or a gift-wrap (addressed to a counterparty), a snapshot event is signed by, and its content sealed to, **our own identity key** — nobody else can decrypt it, and nobody but us has any reason to fetch it.
- **The d tag is opaque**, not the plain `swap_id` the offer’s d tag uses: it’s `tagged_hash("pact/rescue/dtag/v swap_id)`. This is deliberate — a relay operator can see that some pubkey published both an offer (kind 31510, `d = swap_id`) and a snapshot (kind 31512, `d = opaque`), but cannot correlate *which* snapshot belongs to *which* public offer. Rescue never needs to reverse the tag: it fetches every snapshot by `{ kind: 31512, author: me }` and reads the `swap_id` from inside the decrypted payload.
- **No NIP-40 expiration.** A live swap’s snapshot must persist until it is explicitly tombstoned on completion — unlike an offer, it is not refreshed on a timer, since it is only published at accepted (and, for v2, again at signed).
- **Explicit created_at, not “now”.** Addressable-event replacement ties on equal `created_at` by keeping the *lowest* event id, and the accepted and signed snapshots of a fast v2 handshake can land within the same second. So the caller stamps `created_at = now + state_rank` (0 for the pre-signed states, 1 once signed), guaranteeing the later snapshot always strictly replaces the earlier one on the relay — otherwise a rescued maker

could be stranded on the stale accepted copy, which lacks the unrecoverable assembled adaptor signatures.

- **NIP-09 tombstone on terminal.** `snapshot_tombstone_event` deletes the coordinate `31512:<our pubkey>:<dtag>` once a swap reaches completed, refunded, or aborted, stamped past the last snapshot's `created_at` (NIP-09 only covers events up to the deletion's own timestamp) — so a machine restored later never resurrects a finished swap.

See the Pact handbook chapter “Seeds, Wallets & Merchants” for the engine-side rescue flow (`rescue_from_blobs/rescue_preview`, the gated `restorefromrelay` RPC, and the anchored participant-key derivation that makes a restored machine's keys re-derivable in the first place).

22.2 NIP-40 rolling expiration

Offer events carry a NIP-40 expiration tag, and it is **rolling**, not a fixed `ttl_secs`. For a publish at time now:

```
expiration = min(now + RELAY_TTL_SECS, created + ttl_secs)
```

That is, a published listing drops from relays a short while (`RELAY_TTL_SECS` = 1800s) after each publish *unless the maker re-publishes*, but never later than the offer's own final expiry (`created + ttl_secs`). The engine refreshes live offers on a shorter cadence (`Engine::REFRESH_SECS`, 10 minutes) so a genuinely live offer never lapses, while an abandoned one falls off relays quickly — a listing therefore reflects a maker who is actually online.

Warning — Earlier docs described the offer expiration as `= ttl_secs`. The shipped behaviour is the rolling `min(now + 1800, created + ttl_secs)` above.

22.3 Revocation — NIP-09

Revoking an offer (when it is taken or withdrawn) publishes a NIP-09 `EventDeletion` referencing the addressable coordinate `31510:<maker pubkey>:<swap_id>` via an `["a", ...]` tag, telling relays to drop the listing.

Because relays may not honour NIP-09, viewers also **enforce revocations client-side**. The sync loop subscribes to deletions (`{ kinds: [5] }`, its own cursor) alongside offers, and for each deletion it **verifies the event signature and that the deleting author matches the maker pubkey in the coordinate** — so a maker can only revoke offers it signed, never someone else's. A verified deletion writes a persistent `nostr_revoked:<swap_id>` tombstone and evicts the offer from the cache; the tombstone is applied *before* upserts each round, so an offer a relay keeps serving (NIP-09 ignored) never reappears. The effect: a taken or withdrawn offer leaves **every** board immediately, instead of lingering on other viewers' boards until its NIP-40 expiration lapses.

When a deletion arrives for one of the engine's **own** offers, the engine also reconciles its `my_offers` view, so a self-revoked offer is cleared from the maker's own listing instead of lingering there in a “posting...” state.

22.4 Subscription filters

- **Offers:** { kinds: [31510] } — subscribe by kind; pair/network filtering is client-side.
- **Deletions:** { kinds: [5] } — NIP-09 deletions, so revocations are enforced client-side (see “Revocation” above).
- **Mailbox:** { kinds: [1059], #p: [me] } — kind-1059 events tagged to our identity.

22.5 Identity equals npub

A Pact identity *is* a Nostr npub. Both are BIP340 x-only keys over the same 32-byte secp256k1 secret, so `keys_from_secret_hex` yields a Nostr key whose public key equals the Pact identity pubkey exactly. There is no separate Nostr account.

22.6 The sync / inbox model

NostrBoard does no I/O; the background service polls. Each scheduler tick runs **three steps**, so the engine lock is held only for fast SQLite work and never across a relay round-trip:

1. **prep** (*under the lock*) — read the active merchant’s identity, pending `nostr_outbox` rows, and the offer/mailbox fetch cursors.
2. **round** (*lock-free*) — **publish the outbox first**, then fetch offers, mail, and deletions, with `FETCH_TIMEOUT = 10s`. Publishing first reclaims latency: outgoing swap-handshake gift-wraps don’t wait on the fetch round, and an own offer’s “posting...” → “live” confirmation can land in the *same* round (~30s sooner) instead of the next. The publish step is honest about delivery — with **no relay connected**, outbox rows stay **queued** (not falsely marked sent) and retry next round; a row is marked sent **only if the event reached at least one relay**.
3. **apply** (*under the lock*) — mark the delivered outbox rows sent, insert inbox rows, upsert the offer cache, and advance the cursors.

Note — An earlier build ran this round **fetch-first**, as a mitigation for a *theorized* cursor-stranding race (publishing first could in principle let a high-water since cursor skip past our own just-published offer, leaving it stuck on a “posting...” badge). That race was never observed in practice; the real “posting... forever” symptom traced to a stale revocation tombstone (fixed separately — see “Revocation”), so the ordering was reverted to publish-first. If cursor-stranding is ever seen for real, the intended robust fix is a windowed / author-scoped offer re-fetch, not reordering the round.

Polling (fetch-per-tick) rather than long-lived subscriptions keeps this aligned with how the engine already drives the HTTP relay each tick and sidesteps subscription/reconnect lifecycle. Cross-relay duplicates are absorbed by `event_id` uniqueness in the buffer tables.

The buffer tables:

Table	Role
<code>nostr_outbox</code>	Offers/revocations/gift-wraps awaiting publication.
<code>nostr_inbox</code>	Received gift-wrap blobs; autoincrement id = the <code>relay_poll</code> cursor; deduped by <code>event_id</code> .
<code>nostr_offer_cache</code>	Offers discovered from relays, with expiry.

22.7 Default relay list

The default relay list lives in **Satchel**, not the engine. A fresh Satchel install is prewired with `RECOMMENDED_NOSTR_RELAYS` (`satchel/src/main.rs`), six relays:

```
wss://relay.damus.io
wss://nos.lol
wss://relay.primal.net
wss://nostr.mom
wss://nostr-pub.wellorder.net
wss://offchain.pub
```

`pactd`'s `--nostr-relay` flag defaults to **empty** — the engine itself ships no relays, so the transport is opt-in. Satchel passes its configured relays to `pactd` at launch. An explicit empty list a user saves is respected (transport off).

Tip — The Nostr transport runs *alongside* `--board-url`, not instead of it. Configure both for maximum redundancy; recall that two parties need only one board in common (see “The Noticeboard Abstraction”).

Chapter 23

Corkboard (self-hostable board)

Corkboard is the HTTP noticeboard: a single axum + SQLite binary that anyone can run. It is deliberately dumb. It stores and serves signed offer envelopes and blind relay blobs, and does nothing else — it never matches orders, never executes a trade, never holds keys or funds, charges no fees, and has no accounts. Humans pick offers; the swap happens entirely between the two *pactd* instances. Running multiple independent operators is the goal (a Bisq-style model), which is why two parties need only one board in common (see “The Noticeboard Abstraction”).

The source is `corkboard/src/main.rs`.

23.1 Running it

```
corkboard --listen 127.0.0.1:9780 --db corkboard.sqlite
```

Flag	Default	Meaning
<code>--listen</code>	<code>127.0.0.1:9780</code>	Bind address.
<code>--db</code>	<code>corkboard.sqlite</code>	SQLite database path (created if absent).

The database is created on first run. The process serves until interrupted (graceful shutdown on Ctrl-C).

23.2 HTTP API

All write endpoints require a valid BIP340 envelope signature (`pact_proto::envelope::verify`), so listings cannot be forged. The board never talks to any chain — proof-of-funds verification is the *client's* job.

Method	Path	Request	Response
GET	/health	—	"ok"
POST	/v1/offers	Envelope (type=offer)	{ "offer_id": <swap_id> }
GET	/v1/offers	query give, get, network (all optional)	{ "offers": [Envelope...] }
POST	/v1/offers/revoke	Envelope (type=revoke)	{ "revoked": <bool> }
POST	/v1/relay	{ "to": <x-only hex 32B>, "blob": <string> }	{ "id": <i64> }
POST	/v1/relay/poll	Envelope (type=relay_poll, body.since_id)	{ "messages": [{ "id": <i64>, "blob": <string> }...] }

Notes on each:

- **GET /v1/offers** returns only unrevoked, unexpired offers, newest first, LIMIT 500. The give/get/network query params filter on the offer body's give_asset/get_asset/network.
- **POST /v1/offers/revoke** flips revoked only on the signer's *own* offer (matched on offer_id **and** identity); revoked reports whether a row changed.
- **POST /v1/relay** is the **blind deposit**. It is **not** signed — by design (see Auth, below). It stores { to, blob } and returns the new row id. The board never inspects the blob.
- **POST /v1/relay/poll** is signed; it returns only the caller's own mail with id > since_id, LIMIT 100.

23.3 Storage model

Two tables (open_db):

```
CREATE TABLE offers (
  offer_id TEXT PRIMARY KEY,
  identity TEXT NOT NULL,
  envelope TEXT NOT NULL,
  created INTEGER NOT NULL,
  expires INTEGER NOT NULL,
  revoked INTEGER NOT NULL DEFAULT 0
);

CREATE TABLE relay (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  recipient TEXT NOT NULL,
  blob TEXT NOT NULL,
```

```
    created    INTEGER NOT NULL
);
```

The `relay.id` autoincrement is the cursor returned by `relay_poll`; the engine persists the highest id it has seen.

23.4 Limits and errors

Constant	Value	Effect
MAX_BLOB_BYTES	64 KiB	Relay blobs larger than this are rejected. The relay is for coordination envelopes, not file transfer.
DEFAULT_OFFER_TTL_SECS	24h	Offer TTL when the body omits <code>ttl_secs</code> .
Offer TTL cap	7 days	<code>ttl_secs</code> is clamped to a week — offers are not archives.

All errors are returned as **HTTP 400** with a body of { "error": <message> }. Examples: a to that is not a 32-byte x-only pubkey, a blob over 64 KiB, or an envelope whose type does not match the endpoint.

23.5 Blind relay

The relay is content-blind. A blob arrives pre-sealed (PACTSEALED1: — ephemeral ECDH + ChaCha20-Poly1305, sealed in the client) and the board stores and forwards the ciphertext verbatim. The operator can see *that* recipient X has mail and how big it is, never *what* it says or *who* sent it. Sealing is covered in “Wire Format (pact-proto)”.

23.6 Reset hygiene

If a board’s SQLite file is wiped or replaced, a returning client’s saved cursor could be *ahead* of every message on the fresh board, silently swallowing all new mail. `relay_poll` guards against this: if the caller’s `since_id` exceeds the maximum id the board still holds for that recipient, it serves from 0. Ids are AUTOINCREMENT (never reused), so a cursor greater than the max only happens on a real reset, not ordinary pruning. The poll is signed, so this only ever re-serves the caller’s own mail.

23.7 Auth model

- **Writes to offers** (`/v1/offers`, `/v1/offers/revoke`) require a valid BIP340 envelope signature. Revocation additionally checks the offer's identity matches the signer.
- `/v1/relay deposit` is **unauthenticated by design**. Anyone may drop a sealed blob addressed to a pubkey — this is how a counterparty who has never met you reaches your mailbox. Confidentiality comes from the seal, not from access control.
- `/v1/relay/poll` is **signed**, proving the caller owns the recipient identity, so strangers cannot drain someone else's mailbox.
- `GET /health` and `GET /v1/offers` are unauthenticated reads.

23.8 Self-hosting walkthrough

1. **Build and run** the board:

```
cargo run -p corkboard -- --listen 127.0.0.1:9780 --db corkboard.sqlite
```

2. **Verify** it is up:

```
curl -s http://127.0.0.1:9780/health # -> ok
```

3. **Point pactd at it** with `--board-url` (comma-separate to use several):

```
pactd --data-dir ~/.pact --board-url http://127.0.0.1:9780 \
      --coin btcx=... --coin btc=...
```

In Satchel, set the board URL in the noticeboard settings instead.

4. From there, posting an offer fans out to this board (and any others configured), and the two pactds coordinate the swap through its blind relay.

Tip — To expose a board beyond loopback, bind it to a routable address with `--listen` and front it with your own TLS terminator. The board itself is plain HTTP and does no transport encryption; the *payloads* it relays are already end-to-end sealed.

Chapter 24

Private (Off-Market) Offers

A *private offer* lets you trade with a specific counterparty without ever posting to a public board — “trade with a friend”. The artifact is a *slip*: the exact same signed offer envelope that would otherwise go to the Corkboard, serialized for an out-of-band channel like a chat message. No new wire fields, no protocol bump — just a different distribution path.

24.1 The slip

A slip is the signed offer envelope, base64url-encoded (unpadded) with a version prefix:

```
pactoffer1:<base64url(canonical_json(offer_envelope))>
```

The codec lives in `pact-proto/src/slip.rs` and is described byte-for-byte in “Wire Format (pact-proto)”. The decode side, `decode_slip`, is the **only trust gate**: before handing back anything it rejects a bad prefix, malformed base64url, a non-Envelope, an envelope whose type is not “offer”, and a bad BIP340 signature. A slip is therefore self-authenticating — a pasted slip is either the maker’s genuine, untampered offer or it is refused.

24.2 Engine surface

The engine (`libswap engine.rs`) exposes four operations:

Operation	Behaviour
<code>make_private_offer(network, give, get, t1_secs, t2_secs, ttl_secs, protocol)</code>	Builds and signs the offer, stores it locally under <code>private_offer:<swap_id></code> , and returns the slip. It does not post to any board.
<code>take_offer_slip(slip)</code>	Decodes and verifies the slip, runs the same gating as a board take, records a pending take, and <code>relay_send_all</code> s the sealed take to every configured board.
<code>list_private_offers()</code>	Lists the maker’s outstanding private offers.

Operation	Behaviour
<code>cancel_private_offer(id)</code>	Marks the offer revoked and deletes its row.

Because `take_offer_slip` seals the take and fans it out over the blind relay, the taker reaches the maker even though the offer was never public — the relay addressing is by the maker's identity pubkey, which is inside the slip.

24.3 RPCs

Method	Params	Returns
<code>makeprivateoffer</code>	<code>give, get, t1_secs, t2_secs, protocol?, ttl_secs?</code>	<code>{ slip }</code>
<code>takeoffer</code>	<code>slip</code>	<code>{ taken: true }</code>
<code>listprivateoffers</code>	—	<code>{ offers }</code>
<code>cancelprivateoffer</code>	<code>offer_id</code>	<code>{ cancelled: true }</code>

Warning — On `makeprivateoffer` the optional `protocol` is parameter **4** and `ttl_secs` is parameter **5** (positional). Earlier docs listed them in the opposite order. Use named params to avoid the ambiguity: `{ "give": "btcx:1.0", "get": "btc:0.5", "t1_secs": 86400, "t2_secs": 43200, "protocol": "pact-htlc-v1", "ttl_secs": 3600 }`.

24.4 The relay-assisted flow

1. **Maker** calls `makeprivateoffer` and gets a slip.
2. **Maker** pastes the slip into a chat (or any out-of-band channel) to the friend.
3. **Friend** calls `takeoffer` with the slip. The engine decodes/verifies it, gates the terms, and sends a sealed take through the blind relay of every configured board.
4. **Maker** polls their mailbox (the scheduler's tick), finds the take, and the swap proceeds exactly like a board-driven swap from there.

Note — The maker **must be online and polling** to receive the take and drive the swap. A slip handed to a friend who acts on it while the maker is offline simply waits in the relay mailbox until the maker next polls (subject to the offer's TTL).

24.5 Bearer-slip security

A slip is a **bearer** instrument: whoever holds it can take the offer. There is no taker allow-list. Safety comes from the swap's own properties, not from secrecy of the slip:

- **Fixed terms** — the amounts, assets, and timelock durations are signed into the offer; a slip cannot be edited without breaking its signature.

- **Atomic settlement** — the swap is an atomic cross-chain swap; either both legs settle or both refund. Taking a slip cannot make the maker lose funds one-sidedly.
- **Maker-funds-first** ordering and **TTL auto-expiry** — an unused slip lapses at `created + ttl_secs`.

So a leaked slip is, at worst, an offer someone else takes on the stated terms — not a theft vector.

24.6 Fully-manual CLI fallback

Everything above is automated, but the slip is just text, so a fully manual flow works too:

Maker: produce a slip and copy it out of band.

```
pact-cli --data-dir ~/.pact call makeprivateoffer \
  '{"give":"btcx:1.0","get":"btc:0.5","t1_secs":86400,"t2_secs":43200}'
```

Friend: take it.

```
pact-cli --data-dir ~/.pact call takeoffer "pactoffer1:..."
```

Maker: poll for the take and drive the swap.

```
pact-cli --data-dir ~/.pact call tick
```

For the swap handshake that follows the take, see the chapters on the v1 and v2 APIs and “The Swap Lifecycle”.

Tip — A slip carries **only** an offer. Other handshake envelopes (take, init, accept, ...) are not slips; when moved manually they travel as raw JSON or as PACTSEALED1: blobs through the relay, not behind the pactoffer1: prefix.

Part VI

Building Against Pact

Chapter 25

Building Your Own Front-End

Satchel is the reference desktop client, but it has no privileged access: it is just one consumer of `pactd`'s JSON-RPC API. Anything Satchel does, your own front-end — a CLI, a TUI, a web dashboard, a bot — can do, by speaking the same JSON-RPC over loopback and authenticating with the cookie. This chapter is the orientation for that; per-method detail lives in the API chapters ("API: Node, Seed, Merchants, Coins", "API: v1 HTLC Swaps", "API: v2 Adaptor Swaps", "API: Board, Private Offers & Fees").

25.1 The integration contract

`pactd` is to the swap engine what `bitcoind` is to Bitcoin: a long-running daemon you drive entirely over RPC. To integrate:

1. **Speak JSON-RPC 2.0 over loopback.** `POST /` with `{ "jsonrpc": "2.0", "id", "method", "params" }`. The listen address is `127.0.0.1:9737` by default and is enforced to be loopback. Params accept a positional array or a named object.
2. **Authenticate with the cookie.** `pactd` writes `<datadir>/.cookie` holding `__cookie__:<hex>` each run; pass it as HTTP Basic credentials. A `pact.conf` with `rpcuser/rpcpassword` is an alternative. `GET /health` is the only unauthenticated route. See "JSON-RPC Conventions" for the full handshake.
3. **Drive the same methods** Satchel does. There is no second, private surface.

Warning — Never expose the `pactd` port directly to a network. It binds loopback only; if you need remote access, front it with your own authenticated, encrypted tunnel.

25.2 A recommended app flow

A front-end typically walks this sequence. Each step maps to one or more RPC methods; consult the API chapters for params and return shapes.

1. **Probe the daemon.** `getinfo` and `walletstatus` — is there a seed, is it encrypted, is it locked, what coins are configured, what network.
2. **Bring up the seed.** If no seed: `createseed` (or `generateseed` → `importseed` for a preview-then-confirm flow). If encrypted and locked: `unlock` with the passphrase (or set `PACT_PASSPHRASE` at boot).
3. **Select a merchant** if running nested mode (`--merchants`): `listmerchants`, then `create-merchant` / `loadmerchant`. All swap/board calls target the active merchant.
4. **Configure and check coins.** `listcoins` (live status, tip height, genesis, capabilities), `validatecoin` before saving a backend, `listpairs` to learn which pairs and protocols are available.
5. **Browse a board.** `boardlistoffers` (a single board per call — an HTTP URL or "nostr"), or build your own ladder from the returned offer envelopes.
6. **Post or take.** `boardpostoffer` to advertise; `boardtake` to take an offer; or `makeprivate-offer` / `takeoffer` for off-market slips.
7. **Poll and render.** Call `tick` on an interval to advance the scheduler and drain coordination mail, then `listswaps` / `listadaptorswaps` / `listpendingtakes` / `listmyoffers` to render state.

25.3 What the front-end owns vs what pactd owns

The division of responsibility is strict and worth internalizing:

The front-end owns	pactd owns
Presentation: order-book ladder, swap cards, balances, forms.	The seed and all keys; they never leave the daemon.
Input validation for UX (amounts, pair selection).	Funding, redeeming, and refunds — the on-chain transactions.
When to call <code>tick</code> / how often to poll.	The scheduler that auto-funds, auto-redeems, and auto-refunds on the clock.
Choosing which board to browse.	Sealing, signing, and the noticeboard fan-out.
Storing UI preferences.	Swap state, timelock enforcement, fee bumping.

In short: your front-end is a *view and a controller*. It never holds key material and never has to implement swap safety — the engine enforces timelocks and refunds whether or not any UI is attached.

25.4 Threading and polling

- **pactd self-ticks.** With `--tick-secs` (default 30) the daemon runs its own scheduler loop; safety does not depend on your UI polling. A swap will refund on time even if your front-end is closed, as long as the daemon is running.

- **Poll tick for responsiveness.** Calling `tick` from your UI on a short interval makes state advance promptly (e.g. picking up a take, redeeming a newly confirmed leg) rather than waiting for the next self-tick. `tick` returns `{ events: [ ... ] }` you can surface as activity.
- **One active merchant at a time.** RPC calls operate on the active merchant; do not assume concurrency across merchants within one process.

25.5 Error handling

Errors come back as a JSON-RPC error object, never an HTTP error status for business failures:

```
{ "jsonrpc": "2.0", "id": 1,
  "error": { "code": -1, "message": "no active merchant – create or load one first" } }
```

code is -32601 for an unknown method (with a *did-you-mean* suggestion) and -1 for everything else; the human-readable cause is in message. Common ones to handle: unknown method '`<m>`' – did you mean '`<nearest>`'? (see '`help`'), missing param '`<name>`', and no active merchant – create or load one first (prompt the user to create or load a merchant). Treat the presence of an error key — not the HTTP status — as the failure signal.

Tip — For the exhaustive list of methods, their parameters, and return shapes, work from the API chapters. This chapter only sketches the flow; those are the contract.

Chapter 26

Testing & the Regtest Harness

Pact is tested at two levels: fast in-process Rust tests (unit logic plus spec-vector conformance), and a Python regtest harness that drives real `pactd` daemons and real regtest nodes through complete swaps. This chapter inventories both and shows how to run them.

26.1 The cargo test suite

`cargo test` over the workspace runs the unit tests embedded in each crate (canonical JSON, sign/verify, sealing, slip codec, the Noticeboard buffers, and the swap state machines) plus the **protocol vector tests**. The vector tests pin the wire/script construction against frozen fixtures so an accidental change to a script, key path, or hash is caught immediately:

Test file	Pins
<code>pact/libswap/tests/vectors.rs</code>	<code>spec/vectors/htlc_v1.json</code> (v1 HTLC).
<code>pact/libswap/tests/vectors_v2.rs</code>	<code>spec/vectors/htlc_v2.json</code> (v2 adaptor).

Regenerate the vectors with `cargo run -p libswap --example gen-vectors` and `cargo run -p libswap --example gen-vectors-v2` (in `pact/`); the tests then re-pin them.

```
cargo test --workspace      # unit + vector tests
cargo clippy --workspace    # lints
```

26.2 The Python harness

The harness lives in `pact/harness/`, is Python 3.10+ stdlib-only, and is built around `regtest_harness.py`:

- It brings up a **Bitcoin PoCX (btcx)** regtest node and a **Bitcoin (btc)** regtest node — always; an optional **Litecoin (ltc)** node starts only with `Harness(with_ltc=True)`.
- It funds one wallet per party per chain (e.g. `alice_pocx` funded / `bob_pocx` empty; `bob_btc` funded / `alice_btc` empty).

- It keeps both chains on a **shared mock clock** (setmocktime) and exposes advance_time() to push median-time-past (MTP) forward — this is how CLTV/timelock and refund behaviour is driven deterministically.
- Genesis hashes are asserted at startup, so a mis-pointed daemon binary fails loudly instead of silently testing the wrong chain.

The two end-to-end suites build the cargo workspace (pactd, pact-cli) and the Corkboard first, then run every scenario against a single shared Harness.

26.2.1 v1 HTLC scenarios — test_swap_e2e.py

Scenario	What it proves
test_complete_swap	Happy-path manual swap: two pactds driven by pact-cli through the file handshake; Bob extracts the preimage from the chain (no courtesy message). HTLC outpoints spent on-chain, both parties received the other asset.
test_refund	Both fund, Alice goes silent, clocks pass T1, both refund. Negative checks: premature refunds and a late redeem past T2 are rejected (§7.4).
test_daemon_autopilot_swap	Alice drives everything over pactd JSON-RPC (cookie auth; an unauthenticated call is rejected 401); duplicated backends (multi-backend path); the scheduler auto-redeems and RBF-bumps Alice's unconfirmed redeem; books completed on confirmation.
test_daemon_autopilot_refund	Both parties walk away after funding; the tick scheduler alone reclaims both legs after the timelocks — and does nothing before them.
test_chain_watched_funding	funded messages are withheld; the swap still completes via on-chain funding discovery.
test_funding_fee_bump_v1	The v1 RBF funding nurse: funding is pinned under the fee fallback, then ticks bump it — asserts a funding-fee-bump event and that the funding txid is replaced; completes via chain-watched detection (the RBF replacement is invisible to the counterparty).
test_balance_validation	Make-offer / take balance checks reject under-funded actions.

Scenario	What it proves
test_create_import_then_swap	Seedless start: Alice createseed, Bob importseed (encrypted) via the wizard's seed-lifecycle RPCs, then a normal swap.
test_coin_setup	listcoins / listpairs / validatecoin: configured + connected + genesis state, capability-derived pair availability, and the genesis-hash gate (accepts the right node, rejects a cross-wired one).
test_corkboard_swap	Maker posts a signed offer, taker takes it, the whole handshake travels through the blind relay to completion. Covers offer withdraw, a competing second take (served-once + reject + auto-delist), and asserts every relay blob is sealed PACTSEALED1: ciphertext, never plaintext.
test_board_reset_recovery	A board reset / stale cursor is recovered (serve-from-0 hygiene).
test_nostr_relay_swap	A swap over a live local Nostr relay , offers and relay mail flowing over Nostr alone.
test_private_offer_swap	Off-market: makeprivateoffer produces a pactoffer1: slip posted to no board; takeoffer <slip> relays the take and the swap auto-completes with no board listing.

26.2.2 v2 adaptor scenarios — test_adaptor_swap.py

Scenario	What it proves
test_adaptor_swap	Happy-path v2 over RPC: the full adaptor lifecycle (adaptorinit → adaptoraccept → adaptorfund → adaptornonces → adaptorsign → adaptorassemble → adaptorredeem), cooperative MuSig2 key-path redeem.
test_adaptor_refund	Single-key CLTV-tapleaf refund (script-path, no MuSig2, unattended-safe).
test_adaptor_refund_feebump	The single-key refund is RBF-bumpable (deterministic re-sign).

Scenario	What it proves
test_adaptor_redeem_cpfp	CPFP child bumps an unbumpable cooperative redeem on BTC (the key-path redeem fee is sealed; a self-funded child spends the redeem's own output).
test_adaptor_redeem_cpfp_ltc	The same CPFP redeem-bump on litecoind — the first v2 swap on LTC.
test_adaptor_funding_cpfp	The v2 CPFP-via-change funding nurse: drives to Signed with leg-A funding unconfirmed, then a tick emits funding-cpfp-bump — asserts the child spends the funding's change while the leg-A swap outputpoint stays unchanged; the package is mined and the swap completes (adaptor sigs over the unchanged outputpoint still redeem).
test_adaptor_depth_gate	The reveal/redeem gate fires at the configured --coin-confs confirmation depth.
test_adaptor_corkboard_swap	A board-driven v2 swap (a PoCX↔BTC pair off-mainnet defaults to pact-htlc-v2, so a plain boardpostoffer posts a v2 offer).

26.2.3 Nodeless wallet scenarios — test_nodeless_e2e.py

The nodeless (bdk + Electrum) wallet path has its own parity suite: real swaps where one side's btcx wallet is the **nodeless bdk wallet over a live electrs** — no Core-RPC URL at all. The stack adds one electrs instance indexing the PoCX node (binary at pact/harness/bin/electrs.exe, gitignored; \$PACT_ELECTRS_BIN overrides).

Scenario	What it proves
v1 HTLC swap, nodeless maker	Alice gives btcx from the bdk wallet: wallet_send funds leg A over Electrum broadcast; completion on both sides.
v2 adaptor swap, nodeless taker	Alice gives btcx as participant: the bdk two-phase leg-B funding (build at accept, broadcast only after leg A is deep); completion on both sides.
v2 cancel pre-broadcast	Leg B is BUILT in the bdk wallet but the maker's leg A never confirms; abort takes the commitment-semantics path and wallet_cancel_funding RELEASES the reserved inputs — balance restored.

Scenario	What it proves
v1 nodeless ↔ nodeless	Both parties' btcx wallets are bdk over the SAME electrs, different seeds.

spike_electrs.py is the companion smoke: the minimal wallet flow (fresh-seed scan, receive, send, activity, restart persistence) against a real electrs, kept for quick iteration on the chain source.

26.3 Running the suites

```
cd pact/harness
```

```
# infrastructure only (nodes start, mine, mocktime works)
```

```
python regtest_harness.py --smoke
```

```
# v1 (HTLC) end-to-end suite
```

```
python test_swap_e2e.py
```

```
# v2 (Taproot/MuSig2 adaptor) end-to-end suite
```

```
python test_adaptor_swap.py
```

```
# nodeless (bdk + Electrum) wallet parity suite
```

```
python test_nodeless_e2e.py
```

The harness resolves node binaries from \$POCX_BITCOIND / \$BTC_BITCOIND (or bin/), so copy your installed daemons into pact/harness/bin/ first. PoCX regtest refuses rapid blocks unless mocktime is active, which is why the harness always sets and advances mocktime on both chains.

26.4 The playground scripts

For a full local stack you can click through (rather than assert against), use the PowerShell playground scripts in tools/:

Script	Brings up
tools/playground-cork.ps1	Regtest PoCX + BTC nodes, a Corkboard , two headless counterparties posting a two-sided book, and Satchel launched as a funded "Alice". Offers on PoCX↔BTC default to v2.

Script	Brings up
<code>tools/playground-nostr.ps1</code>	The same, but relays-only : no Corkboard, a single local ephemeral Nostr relay, Satchel with a relays-only <code>satchel1.json</code> . Proves offers flow over Nostr alone.
<code>tools/playground-nostr-nodeless.ps1</code>	The relays-only stack with Alice nodeless : she runs no btcx/btc nodes — BTCX syncs over a PoCX-patched electrs and BTC over a vanilla electrs, both wallets on her Pact seed, while LTC rides along as her one local-node (core-rpc) coin , so the mixed Electrum+RPC configuration is exercised too. Bob and Carol stay node-backed market makers, with LTC legs so the LTC sub-book is live.

All three block on the Satchel window and tear the whole stack down (PID/port-only) when you close it; `tools/knockdown.ps1` force-tears a stale run. Logs land in `<repo>\.playground\`.

Warning — Teardown is **PID- and port-only**, never by process name. A live mainnet `pocx-bitcoind` must not be killed by a name-based sweep; the playground nodes run on dedicated ports the teardown targets explicitly.

Chapter 27

Reference

A consolidated quick-reference for the whole handbook. Each entry points to the chapter that covers it in full.

27.1 Default ports & paths

Item	Default	Notes
pactd JSON-RPC	127.0.0.1:9737	Loopback-enforced; POST <code>/</code> , GET <code>/health</code> .
Corkboard	127.0.0.1:9780	<code>--listen</code> ; HTTP API.
Nostr relays	none (engine)	Default 6 live in Satchel; <code>pactd --nostr-relay empty</code> .

Per-merchant data directory (flat root, or `merchants/<id>/` nested):

File	Contents
<code>pact.sqlite</code>	Swap/offer/nonce/Nostr state.
<code>seed.mnemonic</code>	BIP39 seed (plaintext, or <code>PACTSEEDv1:<salt>:<nonce>:<ciphertext></code> when encrypted).
<code>.cookie</code>	Per-run RPC cookie <code>__cookie__:<hex></code> (data-dir root only).
<code>pact.conf</code>	Optional <code>rpcuser</code> / <code>rpcpassword</code> .
<code>merchants.json</code>	Merchant manifest (parent data dir, nested mode).
<code>logs/pactd.log.<date></code>	Rolling daily log (data-dir root); secret-free; <code>RUST_LOG</code> , default <code>INFO</code> .

`pact.sqlite` tables: `swaps`, `meta` (`counters` / `relay_cursor` / `private offers`), `pending_takes`,

nonce_sessions, adaptor_swaps, nostr_outbox, nostr_inbox, nostr_offer_cache, my_offers.

Corkboard SQLite tables: offers, relay. See “Corkboard (self-hostable board)”.

27.2 RPC method index

All methods are JSON-RPC over POST / — **64 public methods**, grouped by area; see the named chapter for params and return shapes. The daemon prints the same catalog live: `help` (by category, plain text) and `listmethods` (a name array).

Node / info (“API: Node, Seed, Merchants, Coins”): `getinfo`, `walletstatus`, `setwatchonly`, `help`, `listmethods`, `stop`, `getfeepolicy`, `setfeepolicy`.

Seed-only rescue (same chapter; full mechanics in “Seeds, Wallets & Merchants”): `restorefromrelay` (adopt rescuable relay snapshots), `rescuestatus` (read-only preview + the two-machines warning).

Seed lifecycle (same chapter): `createseed`, `generateseed`, `importseed`, `unlock`.

Merchants (same chapter): `createmerchant`, `listmerchants`, `loadmerchant`, `renamemerchant`, `unloadmerchant`, `getmerchantinfo`.

Coins / pairs (same chapter): `listcoins`, `listpairs`, `validatecoin`.

Wallet helpers (same chapter): `getbalance`, `getnewaddress`, `estimatesendfee`, `sendtoaddress`, `bumpfee`, `listtransactions`.

Swaps — v1 HTLC (“API: v1 HTLC Swaps”): `listswaps`, `getswap`, `listpendingtakes`, `listmyoffers`, `offer`, `acceptoffer`, `recv`, `fund`, `redeem`, `refund`, `abort`, `tick`.

Swaps — v2 adaptor (“API: v2 Adaptor Swaps”): `listadaptorswaps`, `adaptorinit`, `adaptoraccept`, `adaptorrecv`, `adaptorfundingready`, `adaptornonces`, `adaptorsign`, `adaptorassemble`, `adaptorfund`, `adaptorredeem`, `adaptorrefund`.

Board (“API: Board, Private Offers & Fees”): `boardlistoffers`, `boardstatus`, `boardpostoffer`, `boardtake`, `boardrevoke`.

Private offers (same chapter, and “Private (Off-Market) Offers”): `makeprivateoffer`, `takeoffer`, `listprivateoffers`, `cancelprivateoffer`.

Fees (same chapter): `estimateswapfees`.

Diagnostics (“API: v1 HTLC Swaps”): `dumpswap` (secret-free per-swap record + log bundle; works for v1 and v2); `swapprogress` (live snapshot of every active swap — confirmations and feerate; v1 and v2).

27.3 Error format

Failures return a JSON-RPC error object (not an HTTP error status):

```
{ "jsonrpc": "2.0", "id": 1, "error": { "code": -1, "message": "<reason>" } }
```

code is -32601 for an unknown method (with a *did-you-mean* suggestion) and -1 for everything else. Common messages: unknown method '*<m>*' – did you mean '*<nearest>*'? (see 'help'), missing param '*<name>*', no active merchant – create or load one first. See “JSON-RPC Conventions”.

27.4 BIP32 derivation paths

All keys derive from one BIP39 seed under purpose 7228'. `coin(c)`: BTC = 0, PoCX = 0x504F4358. See “HTLC v1 (Construction)” and “v2 Adaptor (Construction)”.

Key	Path	Use
Identity	<code>m/7228'/0'/0'</code>	BIP340 x-only; signs envelopes; equals the Nostr npub. Never used in an HTLC.
Swap key, initiator (Alice)	<code>m/7228'/1'/coin(c)'/i'</code>	One compressed secp key per chain per swap (ECDSA in v1; reused as the MuSig2 x-only signer in v2), indexed by the local counter <i>i</i> .
Swap key, participant (Bob), anchored	<code>m/7228'/1'/coin(c)'/a'/b'/c'</code>	Same key type; <i>a, b, c, d</i> are the first four masked-31-bit words of <code>TaggedHash("pact/swap-key-anchor/v1", anchor)</code> (H for v1, T for v2, spec §4.2) — no counter needed, re-derivable from the seed plus the anchor alone.
Preimage source	<code>m/7228'/2'/i'</code>	$s = \text{Tagged-Hash}(\text{"pact/htlc/preimage/v1", priv}), H = \text{SHA256}(s)$ (v1, Alice-only).
Adaptor secret source	<code>m/7228'/2'/i'</code>	$t = \text{Tagged-Hash}(\text{"pact/adaptor/secret/v2", priv}) \bmod n, T = t \cdot G$ (v2, Alice-only).
Refund key (v2), initiator	<code>m/7228'/3'/coin(c)'/i'</code>	x-only single-key CLTV tapleaf, independent of MuSig2.

Key	Path	Use
Refund key (v2), participant, anchored	m/7228'/3'/coin(c)'/a'/b'/c	Same anchored derivation as the participant swap key, on branch 3'.

27.5 Tagged-hash tags

`tagged_hash(tag, msg) = SHA256(SHA256(tag) || SHA256(tag) || msg).`

Tag	Used for
pact/msg/v1	Envelope signing digest.
pact/swapid/v1	v1 swap_id (over H).
pact/swapid/v2	v2 swap_id (over T).
pact/htlc/preimage/v1	v1 preimage s.
pact/adaptor/secret/v2	v2 adaptor secret t.
pact/relay/ecdh/v1	Sealed-blob symmetric key (ECDH → ChaCha20-Poly1305).
pact/swap-key-anchor/v1	Participant's anchored swap/refund key derivation (over H or T, §4.2).
pact/rescue/dtag/v1	Opaque per-swap d-tag for a rescue snapshot event (over the swap_id).

27.6 Spec & vectors file map

File	Contents
spec/protocol.md	HTLC v1 (pact-htlc-v1): scripts, tx templates, key paths, preimage/timelock rules, the §8 message handshake.
spec/protocol-v2.md	v2 (pact-htlc-v2): Taproot/MuSig2 adaptor swaps; specifies only what changes from v1.
spec/vectors/htlc_v1.json	v1 test vectors (pinned by tests/vectors.rs).
spec/vectors/htlc_v2.json	v2 test vectors (pinned by tests/vectors_v2.rs).

27.7 Glossary

- **HTLC** — Hash Time-Locked Contract; the v1 swap output: spendable by a hash preimage (redeem) or after a CLTV timelock (refund).

- **Adaptor signature** — a signature missing a secret scalar; completing it on one chain reveals that scalar, letting the counterparty complete the other. The basis of v2 swaps.
- **MuSig2** — a two-round Schnorr multisignature; the 2-of-2 aggregate that forms the v2 Taproot internal key.
- **Tapleaf** — a script leaf in a Taproot tree; v2 uses one, the single-key CLTV refund script.
- **CLTV** — OP_CHECKLOCKTIMEVERIFY; enforces an absolute locktime, used for the refund branch. Pact requires Unix-time locktimes (≥ 500000000).
- **MTP** — median time past; the 11-block median a node uses to evaluate time-based locktimes. Refund readiness is keyed on MTP.
- **RBF** — Replace-By-Fee; rebroadcasting a tx with a higher fee. v1 redeem and refund and the v2 single-key refund are RBF-bumpable.
- **CPFP** — Child-Pays-For-Parent; a child tx that bumps a stuck parent's effective fee. Used to accelerate the **unbumpable** v2 cooperative redeem.
- **Preimage** — the secret s whose hash $H = \text{SHA256}(s)$ locks a v1 HTLC; revealing it on one chain unlocks the other.
- **Sweep address** — a fresh wallet-owned address a v2 cooperative redeem sweeps to (alice_sweep_b / bob_sweep_a).
- **Merchant** — one seed bound to one data directory; the engine's wallet analog. RPC calls target the active merchant.
- **Noticeboard** — the pluggable offer/relay transport behind the five-method Noticeboard trait (Corkboard over HTTP, or Nostr).
- **Slip** — a private (off-market) offer serialized as `pactoffer1:<base64url>`; the same signed offer envelope, never posted to a board.
- **Rescue snapshot** — an encrypted-to-self Nostr event (kind 31512) holding enough of an in-flight swap's state that a machine restored from the seed alone can adopt and finish it. Published at accepted (v1 and v2) and again at signed (v2 only); tombstoned on terminal states. See "Seeds, Wallets & Merchants".
- **PRE_FUNDING_TIMEOUT_SECS** — 15 minutes; the shared deadline for two self-healing behaviors: a v2 handshake stuck before either leg funds auto-aborts, and a board take whose signed `taken_at` is older than this is silently dropped rather than served.